Handwriting robot

Final Report

D.R. Braune
23539195

Submitted as partial fulfilment of the requirements of Project EPR400
in the Department of Electrical, Electronic and Computer Engineering
University of Pretoria

November 2026

Study leader: Prof. H.C. Myburgh

Part 1. Preamble

This report describes the engineering process followed to complete my final year, which involved creating a handwriting robot to classify and reproduce text in author's handwriting.

1. Project proposal and technical documentation
This main report contains an unaltered copy of the approved Project Proposal (as Part 2 of the report).

All the code that I developed appears as a separate submission on the AMS.

2. Project history
The project was done on my own, with advice from an ex-project student, algorithms and techniques inspired by referenced papers, like classifier structure.

3. Language editing
This document has been language edited by a knowledgeable person. By submitting this document in its present form, I declare that this is the written material that I wish to be examined on.

My language editor was _____.

_____ _____

Language editor signature *Date*

4. Plagiarism declaration

I, _____ understand what plagiarism is and have carefully studied the plagiarism policy of the University. I hereby declare that all the work described in this report is my own, except where explicitly indicated otherwise. Although I may have discussed the design and investigation with my study leader, fellow students or consulted various books, articles or the internet, the design/investigative work is my own. I have mastered the design and I have made all the required calculations in my lab book (and/or they are reflected in this report) to authenticate this. I am not presenting a complete solution of someone else or that I found somewhere.

Wherever I have used information from other sources, I have given credit by proper and complete referencing of the source material so that it can be clearly discerned what is my own work and what was quoted from other sources. I acknowledge that failure to comply with the instructions regarding referencing will regarded as plagiarism. If there is any doubt about the authenticity of my work, I understand that I may have to attend an oral ancillary examination/evaluation about the work.

5. AI usage declaration

Artificial intelligence tools assisted with code development, analysis support and report editing.

The author is responsible for checking the submitted work, understanding the implementation and acknowledging assistance in accordance with the University's requirements.

6. Software library usage declaration

The project includes first-principles implementations of the network operations, training and image-processing algorithms described in Design. NumPy provides array operations. Supporting libraries provide image handling, the web interface and hardware access. A PyTorch route provides reference training and comparisons. The evaluated checkpoint provenance is recorded in Appendix C.

7. Unaltered Project Proposal

I certify that the Project Proposal appearing as the Introduction section of the report is an unaltered copy of the approved Project Proposal.

Signature

With my signature below, I hereby certify that all statements, declarations, and information presented above are complete.

_____ _____

D.R. Braune Date

TABLE OF CONTENTS

Part 2. Project definition: approved Project Proposal

1. Project description
2. Technical challenges in this project
3. Functional analysis
4. System requirements and specifications
5. Field conditions
6. Student tasks

Part 3. Main Report

1. Literature study 1
    - 1.1. Handwriting recognition and writer identification 1
        - 1.1.1. Offline recognition and available datasets 1
        - 1.1.2. Image conditioning and text separation 1
        - 1.1.3. Neural networks and feature extraction 2
        - 1.1.4. Training and assessment 4
        - 1.1.5. Techniques for writer identification 4
    - 1.2. Handwriting synthesis in a writer's style 4
    - 1.3. Physical reproduction and pen-path mapping 5
    - 1.4. Application of the literature 6

2. Approach 7
    - 2.1. Mechanical Hardware Design 7
    - 2.2. Page condition and line segmentation 7
    - 2.3. Text and writer classification 8
    - 2.4. Style extraction and handwriting reproduction 8
    - 2.5. Integration and testing 8

3. Design and implementation 9
    - 3.1. Design summary 9
    - 3.2. System block diagram 10
    - 3.3. Hardware Design 11
        - 3.3.1. Plotter designs and mechanics 11
        - 3.3.2. Stepper motor and driver 14
        - 3.3.3. Step resolution, microstepping and calibration 16
        - 3.3.4. Pen up/down mechanism 17
        - 3.3.5. End stop switches and calibration 19
        - 3.3.6. Gcode generation and motion mapping 21
    - 3.4. Page conditioning and line segmentation 24
        - 3.4.1. Lighting normalisation and ink extraction 24
        - 3.4.2. Page orientation and skew correction 25
        - 3.4.3. Line detection and separation 25
        - 3.4.4. Assigning ink to the correct line 26
        - 3.4.5. Preparing the classifier crops 28

3.4.6. Complete processing sequence and output 29
3.4.7. Segmentation verification and limitations 29
3.5. Neural network design, training and classification 31
3.5.1. Training data, labels and evaluation separation 31
3.5.2. Shared convolutional feature extractor 32
3.5.3. Text recognition using sequence modelling and CTC 33
3.5.4. Writer identification and stroke normalisation 36
3.5.5. Training procedure and model selection 37
3.5.6. Verifcation through testing with off-the-shelf methods 38
3.6. Style extraction and handwriting reproduction 39
3.6.1. Choice of reproduction representation 39
3.6.2. Character alignment and glyph extraction 39
3.6.3. Skeleton tracing and pen path construction 40
3.6.4. Glyph selection and library quality 42
3.6.5. Writer style measurement and storage 44
3.6.6. Text layout and connected handwriting synthesis 45
3.6.7. Candidate selection for legibility and writer identity 45
3.6.8. Machine coordinate mapping and G-code generation 47
3.6.9. Reproduction design limitations 48
3.7. System integration and embedded operation 48
3.7.1. Classification and reproduction workflows 49
3.7.2. Embedded execution, interfaces and job control 49
3.7.3. Integration verification and remaining constraints 49

4. Results 50
4.1. Summary of results achieved 50
4.2. Qualification tests 50
4.2.1. Test material, configuration and measurement definitions 50
Qualification test 1: Reproduction of digital text in a trained writer's style (R1)
Objectives of the test or experiment 51
Equipment used 51
Test setup and experimental parameters 51
Steps followed in the test or experiment 51
Results or measurements 51
Observations 55
Statistical analysis 55
Qualification test 2: Reading handwritten content and identifying its writer (R2)
Objectives of the test or experiment 55
Equipment used 55
Test setup and experimental parameters 55
Steps followed in the test or experiment 55
Results or measurements 55
Observations 56
Statistical analysis 57
Qualification test 3: Time to reproduce digital text on paper (R3)

Objectives of the test or experiment 57
Equipment used 57
Test setup and experimental parameters 57
Steps followed in the test or experiment 57
Results or measurements 57
Observations 58
Statistical analysis 58
Qualification test 4: Time to decode a photographed page and identify its writer (R4)
Objectives of the test or experiment 58
Equipment used 58
Test setup and experimental parameters 58
Steps followed in the test or experiment 59
Results or measurements 59
Observations 60
Statistical analysis 61
Qualification test 5: Gantry dimensional accuracy and repeated geometric samples (R5)
Objectives of the test or experiment 61
Equipment used 61
Test setup and experimental parameters 61
Steps followed in the test or experiment 61
Results or measurements 61
Observations 62
Statistical analysis 62
Qualification test 6: Storage and use of multiple trained styles (R6)
Objectives of the test or experiment 62
Equipment used 62
Test setup and experimental parameters 62
Steps followed in the test or experiment 63
Results or measurements 63
Observations 63
Statistical analysis 63
4.2.2. Qualification under the proposal's field conditions 63

5. Discussion 64
5.1. Critical evaluation of the design 64
5.1.1. Interpretation of results 64
5.1.2. Critical evaluation 64
5.1.3. Unsolved problems 65
5.1.4. Strong points of the design 65
5.1.5. Expected failure conditions 65
5.2. Considerations in the design 65
5.2.1. Ergonomics 65
5.2.2. Health and safety 66
5.2.3. Environmental impact 66
5.2.4. Social and legal impact 66
5.2.5. Ethics clearance 66

6. Conclusion 67
6.1. Summary of the work completed 67
6.2. Summary of observations and findings 67
6.3. Contribution 67
6.4. Suggestions for future work 67

7. References 68

A. Extended design calculations and connections 72
A.1. Worked values from the implemented system 72
A.1.1. Page conditioning and line segmentation 72
A.1.2. Network calculations 72
A.1.3. Glyph extraction and reproduction calculations 73
A.1.4. System integration calculation 74
A.2. Additional recorded numerical checks 74
A.3. Extended training procedure 74
A.4. Representation and connection records 75
A.5. Controller pin and supply planning 76

B. Extended design figures and subsystem evidence 78
B.1. Recorded capture and seam checks 78
B.2. Numerical verification records 78
B.3. Extended figures from the Design section 81

C. Qualification evidence 84
C.1. Measurement records 84
C.2. Original-handwriting classifier evidence 84
C.3. Reproduced-page classifier evidence 84
C.4. Geometric comparison evidence 85
C.5. Completed reproduction timing records 88
C.6. Recorded classification stages 88
C.7. Dimensional measurement evidence 88
C.8. Stored profile evidence 89
C.9. Training log extracts 90
C.10. Complete physical-page photographs 90

LIST OF ABBREVIATIONS

ADC	analogue-to-digital converter
BiLSTM	bidirectional long short-term memory
CAD	computer-aided design
CER	character error rate
CNN	convolutional neural network
CPU	central processing unit
CTC	connectionist temporal classification
DC	direct current
GPIO	general-purpose input/output
GPU	graphics processing unit
GT2	2 mm pitch timing-belt profile
IAM	Institute of Informatics and Applied Mathematics handwriting database
I/O	input/output
LSTM	long short-term memory
NEMA	National Electrical Manufacturers Association
NLL	negative log-likelihood
PWM	pulse-width modulation
QT	qualification test
ReLU	rectified linear unit
RNN	recurrent neural network
USB	universal serial bus
VNC	virtual network computing
WER	word error rate

Part 2. Project definition: approved Project Proposal

This section contains the problem identification in the form of the complete approved Project Proposal, unaltered from the final approved version that appears on the AMS.

For use by the Project lecturer	Approved	✓	Revision required	

Feedback

For use by the Project module lecturer only

Approved - the project is clearly delineated.
However, a number of things to pay attention to in especially section 4.

Symbol awarded: B

To be completed by the student

PROJECT PROPOSAL 2026

Project no	HCM4	Revision no	0

Title	Surname	Initials	Student no	Study leader (title, initials, surname)
Mr	**Braune**	**D.R**	23539195	Prof. H.C. Myburgh

Project title (this is the title on the project concept note)

Handwriting robot

Language editor details	Language editor signature
M.T. Braune	M.T. Braune
Student declaration I understand what plagiarism is and that I have to complete my project on my own.	**Study leader declaration** This is a clear and unambiguous description of what is required in this project. Approved for submission (Yes/No)
Student signature	Study leader signature and date 20 April 2026

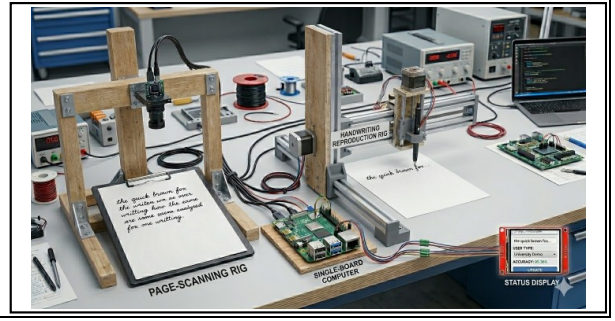
1. Project description.

What is the problem to be solved with your project? Describe in non-technical language what your project is about. What does your system have to do? Refer explictly to Figure 1 that you inserted on the right.

The need for handwriting still exists for authenticity, ease of communication and personal touches. The problem is that many individuals do not possess the time, skills or ability to write neatly and legibly. This leads to the solution presented in this proposal: a handwriting robot capable of recognizing a user's handwriting style and being able to replicate it.

The proposed solution is a handwriting robot capable of training on multiple different user's handwritten text (using a page-scanning rig to capture the handwritten page like the one seen in the image) and being able to produce any desired piece of text in any of these user's writing styles (using a text reproduction rig like the one seen in the image). The system will be able to recognize which user authored a piece of text and identify what text was written (shown to the user through a display as seen in the image). A gantry with 2-dimensional movement will perform actions with a pen to automatically reproduce the desired text.

Figure 1. Insert an image below that captures the idea with your project. Explain in the text on the left.



2. Technical challenges in this project

Describe the technical challenges that are *beyond* those encountered up to the end of third year and in other final year modules.

2.1 Primary *design* challenges

What is the intended core technical challenge in the project? Which aspect or aspects of the design of the system do you expect to be the most challenging?

1. Text image processing will be performed from first principles, including the isolation of letters from an image of various fonts and sizes of handwritten text.
2. A neural network will be designed and trained from first principles to be able to categorize text to different user's styles based on certain style elements, and identify the text written.
3. A gantry rig will be designed to be capable of performing the necessary precise 2-dimensional movements using a pen, whilst holding the pen and paper securely in place.
4. Pen path planning will be implemented, which involves the conversion of written characters to a 2D motion the gantry needs to perform, and then mapping the movement to stepper motor controls.

2.2 Primary *implementation* challenges

Which aspect or aspects of the implementation do you expect to be the most challenging?

1. Image processing and deep learning will be performed on a single board computer (instead of on a personal computer), which has limited memory, storage and power.
2. Smooth motor control will be needed to produce precise and consistent pen movement to create accurate text.
3. Near real-time classification of handwritten text and reproduction of digital text on paper will need to be achieved.

All processing will be done on an embedded platform, and not on a PC platform. Confirmed by having ticked this box. ☑

Library functions will be used for hardware interfacing only. Confirmed by having ticked this box. ☑

3. Functional analysis

3.1 Functional description

Describe the design in terms of system functions as shown on the functional block diagram in section 3.2. This description should be in *narrative format*. **DO NOT** use a bullet list.

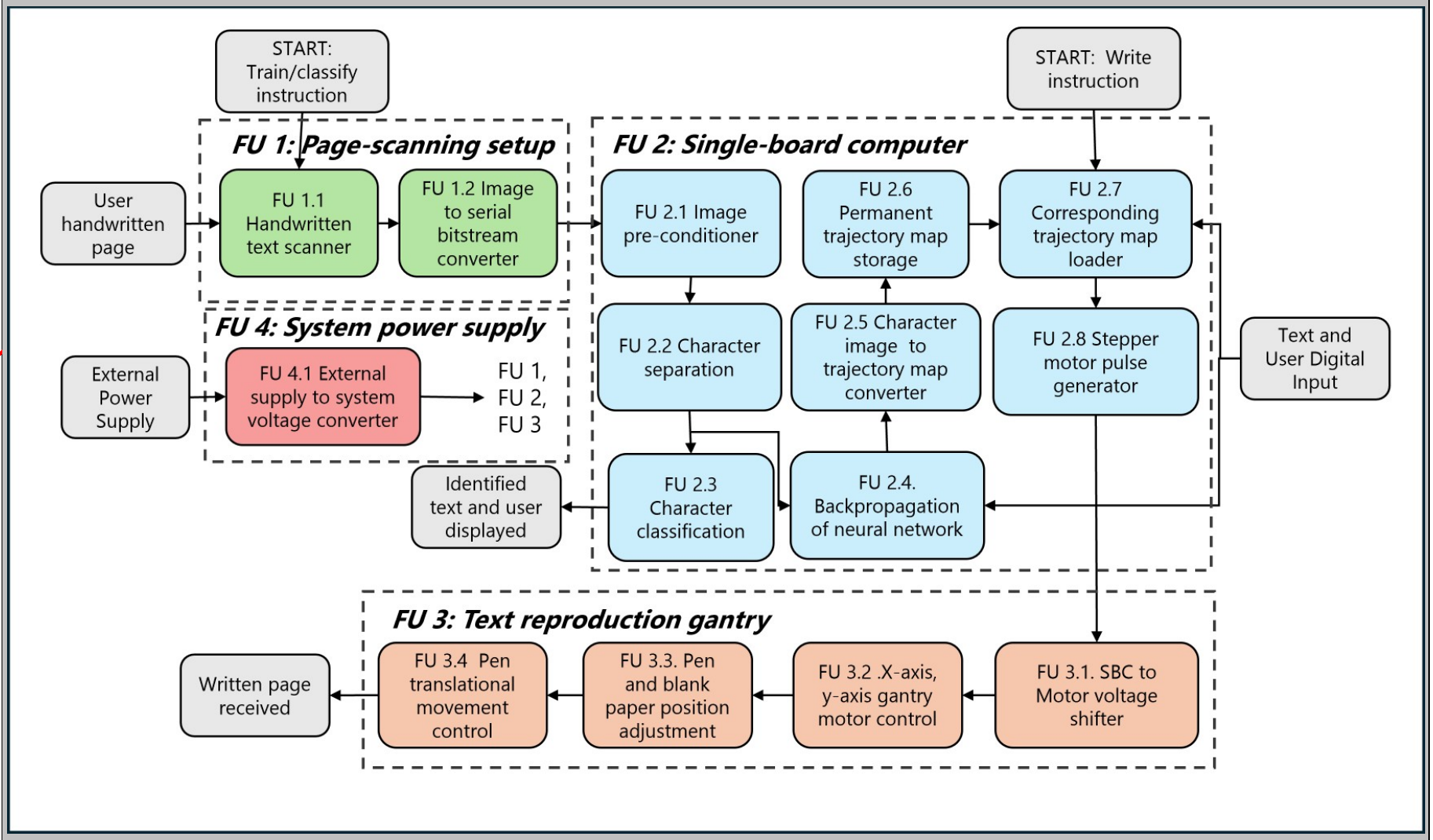
There are three main modes of operation for the handwriting robot: training, classification and reproduction

Training/Classification: First, the user will input their page of handwritten text to the scanner (FU 1.1). The user will select to either train or classify, and will input the intended digital text alongside this through an input interface. Once the page is scanned, a camera USB driver will be used to transfer the image to the single board computer (FU 1.2). The image will be conditioned first to fix warping or noise in the image (FU 2.1), and then the character separation will occur. This will separate the page characters (or groups of characters that are connected) to match the desired neural network input size (FU 2.2). The difference between training and classification occurs here: if the 'training' option is selected, the separated characters will be used to adjust the neural network neuron weightings against the desired digital text that was input (FU 2.4). Each separated character or group of characters will also be converted into a trajectory map (coordinate map of required pen motion) (FU 2.5). This trajectory map will be stored permanently within the system (FU 2.6). If the 'classify' option is selected, the separated characters or group of characters will be passed into the trained neural network classifier (FU 2.3), and the corresponding digital text and detected user will be outputted to the display interface.

Reproduction: If the user sends a 'write' instruction, the digital text and user style desired to be reproduced will be provided. The corresponding trajectory for each character/group will be retrieved from storage (FU 2.7), and will be converted into stepper motor controls required to produce the desired pen movements (FU 2.8). These stepper motor controls will be sent to the corresponding motors via a stepper motor driver to allow for forward and reverse movements (FU 3.1). The x-motor, y-motor and pen lift mechanism will be controlled through the stepper motor driver (FU 3.2/3.4). After the pen has been adjusted and is held securely by the pen holder in the robot (FU 3.3), precise handwritten imitations of each character will be written.

Remove power supply (FBD should shown system functions).
Remove the "start" blocks - this is not a flow diagram, but rather an FBD.

3.2 Functional block diagram (this should not be a flow diagram)



4. System requirements and specifications

These are the core requirements of the system or product (the mission-critical requirements) in table format **IN ORDER OF IMPORTANCE**. Requirement 1 is the most fundamental requirement.

	Requirement 1: the fundamental functional and performance requirement of your project	**Requirement 2** (Second in order of importance)	**Requirement 3** (Third in order of importance)
1. Core mission requirements of the system or product. Focus on requirements that are **core** to solving the engineering problem. These will reflect the solution to the problem.	The system must be able to accurately reproduce any desired digital text on paper in the writing style of any writer from the training set.	The system must be able to accurately decode the content of the handwritten text, and identify which writer from the training set wrote the handwritten text.	The system must reproduce text on paper at a near real time speed, at a rate that can be considered practical and useful.
2. What is the target specification (this must be in *measurable* terms) to be met in order to achieve the requirement in 1. above?	The system should reproduce desired text with a minimum character accuracy of 85% while having a high stylistic correlation of 85% to the chosen writer's style.	The system must be able to identify the writer of the handwritten text from the training set and the content of the written text with an overall accuracy of 95%.	The system must be capable of reproducing the desired text in the writing style of any writer in the training set at a rate of no slower than 4 seconds per character.
3. Explain where this specification originated : i.e., **explain** the value that you selected. I.e., *how* will meeting the specification given in point 2 above *solve the problem*?	An accuracy of 85% (averaged from Babushkin et al. results [1, p8]) is high enough to represent the input text, ~~but practical to account for the build up of precision loss in the system.~~ see class discussion on what specs should be based on.	An overall classification accuracy of 95% (similar to existing classifiers mentioned in Acta Informatica Pragensia [2, p6]) ensures practical usability and consistency.	Averaging the time required to produce text between 0% and 1% error (Ossi Teikari [3, p35]) gives approximately 4 seconds per character, allowing the system to reproduce text at a practical rate with minimal error.
4. How will you demonstrate at the examination that this requirement and specification (points 1 and 2 above) have been met? Be explicit about how you will ***prove*** these were met.	A sentence of text will be reproduced in a specific user style, and the amount of characters that can be visibly seen to match the input text (in the style of the chosen user) must be >= to 85% of the total characters produced. This is not a proper way to measure write in text	A predefined paragraph of text must be handwritten by a writer in the training set and classified by the system. The classified text and writer must match the input text and writer with an overall accuracy of >= 95% .	A sentence of x characters long will be sent to the system to be reproduced, the system must be able to reproduce it within 4*x seconds. this means "convolution"
5. Your own design contribution: what are the aspects that *you will design and implement yourself* to meet the requirement in point 2? If none, *remove this requirement.*	Accurate conversion from character image to pen trajectory map, precise motor control from pen trajectory map, and a structurally sound gantry design for accurate drawing will be designed and implemented by the student.	A rig to capture an image of written text, image conditioning and noise removal, character separation and processing, classification with a neural network, and communicating necessary data to a display will be designed by the student.	Techniques to optimize pen movements for quicker drawing, smooth motor control and robust design for efficient gantry movements will be designed and implemented by the student.
6. What are the aspects to be taken off the shelf to meet this requirement? If none, indicate "none". Clearly specify for what tasks **library functions** will be used (if relevant to the project).	Physical motors, pen, single board computer, stepper motor driver, and separate components for gantry design e.g. screws or belts will be taken off the shelf.	A camera, camera driver (library function), single board computer, separate components for image-capturing rig, and a physical device where data is displayed will be taken off the shelf.	Efficient physical motors, single board computer, stepper motor driver, and separate components for gantry design e.g. smooth ball bearings will be taken off the shelf.

the NUMBER of characters, NOT amount.

System requirements and specifications page 2

	Requirement 4 (Fourth in order of importance)	**Requirement 5**	**Requirement 6**
1. Core mission requirements of the system or product. Focus on requirements that are core to solving the engineering problem. These will reflect the solution to the problem.	The system must be able to identify the writer of the handwritten text, and decode the content of the handwritten text in a short, practical amount of time.	The gantry of the system must have precise pen movements and accurately draw the desired characters with minimal positional error.	The system should be able to be trained on multiple different writing styles and be capable of storing all the writers' necessary character trajectory maps within the system permanently.
2. What is the target specification (this must be in *measurable* terms) to be met in order to achieve the requirement in 1. above?	The system must identify the writer of the written text from the training set and decode the content of the text at a rate of no slower than 25ms per character with an extra 2 seconds for image capture/conditioning.	The gantry of the system must be capable of drawing the desired character trajectory map while maintaining an absolute positional error that does not exceed 0.5mm. 0.5 mm relative to text that is how large?	The system should be capable of training on a minimum of 10 different users' writing styles, and be capable of storing the necessary character mappings for each of these 10 user's permanently.
3. Explain where this specification originated: i.e., explain the value that you selected. I.e., *how* will meeting the specification given in point 2 above solve the problem?	2 seconds was given to image capturing and conditioning due to transmission times and complex conditioning and noise removal algorithms, and 25ms per character classification due to simple forward propagation calculations. same comment as before. Specs are selected to best solve the problem, not because "processing will take long".	According to research by T. Huang and R. Xiong [4] an absolute positional error of 0.3mm with a gantry is achievable and indicates sufficient accuracy, therefore a maximum absolute error of 0.5mm was chosen for slightly more leeway. imcorrect motivation	Having 10 trained user styles stored at the same time proves multi-user neural network recognition and multi-user storage. More than 10 is not required due to neural network complexity and storage limitations. same comment again
4. How will you demonstrate at the examination that this requirement and specification (points 1 and 2 above) have been met? Be explicit about how you will *prove* these were met.	A page with a sentence of x handwritten characters will be given to the system. The time from the initial instruction to the received classified text and user should be within 2 + x*0.025 seconds.	The system should be instructed to recreate the same word twice without varying the dataset characters, and the two words should appear identical on-top of each other, and not exceed an absolute positional error of 0.5mm.	The system should display the current amount of users in the system, and any random user should be selected out of the 10 and have text reproduced in their style to prove the system can be trained with 10 different users.
5. Your own design contribution: what are the aspects that *you will design and implement yourself* to meet the requirement in point 2? If none, *remove this requirement.*	A rig to capture an image of written text, image conditioning,optimized and efficient character separation, neural network character classifier, and calculation and display of time taken will be designed and implemented by the student.	Accurate conversion from pen trajectory map to precise stepper motor control, robust gantry system to write characters, secure pen adjuster and holder, and a secure paper holder will be designed and implemented by the student.	A rig with a camera to capture new text, sufficiently complex neural network to be able to classify minimum 10 user's styles, and a storage strategy to store 10 user's character trajectory mappings will be designed by the student.
6. What are the aspects to be taken off the shelf to meet this requirement? If none, indicate "none". Clearly specify for what tasks **library functions** will be used (if relevant to the project).	A camera, camera driver, single board computer, separate components for image-capturing rig, and a physical device where time is displayed will be taken off the shelf.	Precise stepper motors, separate components needed for gantry, pen and paper holders, and a pen will be taken off the shelf. This show repeatability, not accuracy	A camera, camera driver, single board computer, components for image-capturing rig, and permanent storage device will be taken off the shelf. NUMBER of users. Not amount.

System requirements and specifications page 3

	Requirement 7 (Seventh in the order of importance)	**Requirement 8**	**Requirement 9**
1. Core mission requirements of the system or product. Focus on requirements that are core to solving the engineering problem. These will reflect the solution to the problem.			
2. What is the target specification (this must be in *measurable* terms) to be met in order to achieve the requirement in 1. above?			
3. **Explain where this specification originated**: i.e., explain the value that you selected. I.e., *how* will meeting the specification given in point 2 above solve the problem?			
4. **How will you demonstrate at the examination** that this requirement and specification (points 1 and 2 above) have been met? Be explicit about how you will *prove* these were met.			
5. Your own design contribution: what are the aspects that *you will design and implement yourself* to meet the requirement in point 2? If none, *remove this requirement.*			
6. What are the aspects to be taken off the shelf to meet this requirement? If none, indicate "none". Clearly specify for what tasks l**ibrary functions** will be used (if relevant to the project).			

5. Real world field conditions

These are the ACTUAL, REAL WORLD CONDITIONS under which your project has to work *and will be demonstrated.*

	Real world field condition 1 (required to include)	**Real world field condition 2** (optional to include)	**Real world field condition 3** (optional to include)
Field condition requirement. In which real world conditions does the system have to operate?E.g., "lab conditions" or "outside in uncontrolled conditions". Describe the one, two or three most important field conditions.	The system is expected to perform in well-lit areas like a lab or office environment, at an operating range of 400-1000 lux, to ensure minimal noise in image capturing.	The system is expected to run on a stable, level and vibration free surface, to ensure the page scanning is accurate and the pen in the gantry can write precisely without fluctuations.	

6. Student tasks

6.1 Design and implementation tasks of the student

List your primary design and implementation tasks in bullet list format (5-10 bullets). These are *NOT* product requirements, but *YOUR* tasks.

- The student will research types of neural networks, their structure, the effect of neurons and layers, and how to extract features of text from different layers.
- The student will research image conditioning, noise removal, character separation strategies, and pen trajectory mapping.
- The student will design and implement the convolution neural network, image conditioning, and character separation algorithms.
- The student will simulate the classifier on existing handwritten text datasets with adjusted character separation strategies and neuron/layer numbers to meet requirements.
- The student will construct a rig to capture images of handwritten text on an A4 page, and implement the neural network and algorithms on a chosen single board computer.
- The student will test the new classifier implementations on the single board computer with the captured images to ensure requirements are met.
- The student will design the gantry system in CAD software and construct it with the necessary store-bought individual components.
- The student will design and implement the pen trajectory mapping algorithm and the stepper motor control.
- The student will test the constructed gantry with the implemented classifier to ensure the remainder of the requirements are met.

6.2 New knowledge to be acquired by the student

Describe what the theoretical foundation to the project is, and which *new* knowledge you will acquire (*beyond* that covered in any other undergraduate modules).

- The student will acquire new knowledge of training and classification of character images using a neural network.
- The student will acquire new knowledge involving the extraction of text features from neural network layers.
- The student will acquire new knowledge about the process of image processing with character isolation.
- The student will acquire new knowledge about pen trajectory map creation.
- The student will acquire new knowledge on 2D gantry design and implementation with precise motor control.
- The student will acquire new knowledge about the implementation of neural networks on a single board computer.

7. References

Format your reference list either in IEEE reference format or in APA reference format. Check what your study leader prefers and use consistently.

[1] V. Babushkin, H. Alsuradi, M. O. Al-Khalil, and M. Eid, “Analyzing handwriting legibility through hand kinematics,” Frontiers in Artificial Intelligence, vol. 8, Mar. 2025, doi: https://doi.org/10.3389/frai.2025.1426455.

[2] Sidar Agduk and Emrah Aydemir, “Classification of Handwritten Text Signatures by Person and Gender: A Comparative Study of Transfer Learning Methods,” Acta Informatica Pragensia, vol. 11, no. 3, pp. 324–347, Dec. 2022, doi: https://doi.org/10.18267/j.aip.197.

[3] O. Teikari, "Reproducing Handwriting with Position-Controlled Robots," Master's thesis, Dept. Pervasive Systems, Tampere University of Technology, Tampere, Finland, 2016.

[4] T. Huang and R. Xiong, "Cost-Effective Robotic Handwriting System with AI Integration," in Proc. 2024 IEEE Long Island Systems, Applications and Technology Conference (LISAT), 2024.

Part 3. Main Report

1. Literature study

1.1 Handwriting recognition and writer identification

1.1.1 Offline recognition and available datasets

The ability of a computer or system to identify written text after being trained on large amounts of data is known as HWR (Handwriting Recognition) or HTR (Handwritten Text Recognition) [1, p. 39]. One type of handwritten text recognition is offline recognition, which means the data being identified by the system is static and captured post-completion of the page of handwritten text. This is ideal for identifying the text from a fixed image of handwritten content by utilising image processing techniques and algorithms to identify the text that was written [2, p. 268]. The other type is online recognition, which is a more dynamic process that uses information recorded during the creation of the handwriting. The addition of a time variable includes data about the order of strokes as well as pen-up or pen-down movements [3, p. 18].

This project uses offline recognition because its inputs are photographed or scanned pages. The text and writer must therefore be identified from visible ink geometry. The absence of recorded stroke order also influences how the handwriting can later be reproduced: a path recovered from an image is a geometric estimate, rather than a recording of the writer's movements.

Multiple datasets have been used for training handwritten text recognition modules. The EMNIST (Extended MNIST) By_Class dataset consists of 814 255 handwritten characters and numbers, each being 28 by 28 pixels in size [1, p. 40]. There is only a single character per image, which suits isolated-character classification but does not itself supply examples of connected handwritten words. The IAM handwriting dataset provides complete handwritten English pages, lines and words with their transcriptions and writer metadata [4]. Hemanth et al. report the expanded database as 5 685 labelled sentences and 115 320 labelled words from 657 writers [5, p. 2460].

The dataset influences which variations the classifier encounters and which labels are available. For this project, line images and their transcripts support recognition without manually labelling every character boundary. Writer-labelled pages also support author classification and extraction of individual style profiles. IAM supplies public handwriting variation, while the personal dataset represents the project's photographed input conditions. A small isolated-character dataset alone would not address both requirements.

1.1.2 Image conditioning and text separation

The image conditioning of handwriting pages includes techniques performed on the handwritten text before it is passed into the classification and feature-recognition algorithms. This accounts for dimensional adjustments, removal of inconsistencies and normalisation of the data, to make the images more appropriate for text separation and classification [5, pp. 2458–2460]. Normalisation reduces the influence of unwanted variation; examples include min–max scaling and z-score normalisation [1, pp. 40–41].

Binarisation differentiates the background page from the ink of the written text, computed by pixel. A global threshold taken from the grey-level histogram uses one decision level for the image [6]. A local threshold instead uses statistics from a window around each pixel and can accommodate uneven illumination [7, pp. 2–3]. Deskewing corrects page rotation, while deslanting corrects the lean of individual characters by applying a horizontal shear. These are different transformations: aligning text lines does not require removing the writer's characteristic slant [8, 9].

For the camera input in this project, illumination correction and deskewing are needed before the ink is separated into lines. Slant must remain available as a writer-specific measurement; it can be removed for glyph storage and reapplied when the text is synthesised. The purpose is to suppress capture variation while retaining the information needed for reproduction.

Text separation is the process of breaking up a page of handwritten text into smaller segments or blocks that can be used in feature extraction and neural-network classifiers. Existing methods separate text into lines, words or individual characters. Hemanth et al. obtain lines and words through filtering, dilation and bounding boxes [5, pp. 2459–2460]. When lines are mostly straight and separated by blank rows, boundaries can be found from minima in the row ink profile. Where ascenders and descenders overlap, a boundary that bends between lines is more suitable. Minimum-cost seam methods provide a way to trace such curved boundaries [10, p. 3].

The project therefore separates whole lines before recognition. This avoids requiring a reliable cut between every pair of connected letters. Character boundaries are still needed to construct a glyph library, but there the known transcript and a trained line reader can constrain the extraction. Line segmentation and glyph extraction consequently solve different problems, even though both divide an image.

1.1.3 Neural networks and feature extraction

Many different types of neural-network models have been implemented for handwriting recognition, including artificial neural networks (ANNs), convolutional neural networks (CNNs), recurrent neural networks (RNNs) and combinations of them. ANNs consist of interconnected neurons, weights, hidden layers and biases that learn a mapping from the input to an output. CNNs use local connections and shared kernels to extract spatial patterns with fewer independent parameters than an otherwise comparable fully connected image layer [11, pp. 6–8]. RNNs maintain a state across a sequence, allowing a prediction to depend on neighbouring evidence [2, p. 269].

The convolutional layers of a CNN perform feature extraction by sliding a convolutional kernel over the image. Each local input value is multiplied by a corresponding kernel value, the products are summed, and a bias and activation function produce an output feature value. Reusing the kernel allows the same pattern to be detected at different locations [11, p. 6]. For unit stride and no padding, an N by N input with an f by f kernel returns an $(N-f+1)$ by $(N-f+1)$ feature map. Pooling then reduces spatial resolution and retains selected responses [11, pp. 6–7]. Figure 1 illustrates these operations.

The LeNet-5 model is an established example of this structure. It uses a 32×32 input and seven trainable layers, including convolution, subsampling and fully connected stages; its original output uses radial-basis units [11, pp. 7–8]. A deeper alternative is a residual network, whose

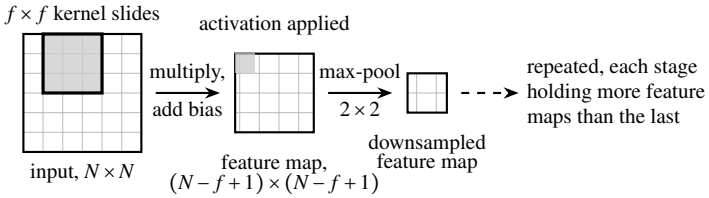


Figure 1. One convolutional layer and one max-pooling layer. The kernel slides over every position of the input, so an $N \times N$ input gives an $(N-f+1) \times (N-f+1)$ feature map, which pooling then downsamples [11, pp. 6–8]. This size expression assumes unit stride and no padding.

skip connections provide additional routes for information and gradients through the network [2, p. 270]. These examples show the choice between compact feature extraction and a more complex model; increasing depth does not guarantee an improvement for every dataset or processor.

A combination of CNNs and recurrent layers allows an entire line to be read without word or character separation. The convolutional layers convert the cropped line into feature columns, and a bidirectional recurrent layer reads the sequence in both directions, providing context from before and after each position [12, pp. 4–5]. In this offline setting, the sequence axis is the horizontal position in the image, rather than the measured time at which a pen moved. The full line crop is available before classification. Related convolutional–recurrent recognition systems use the same distinction between image features and sequence prediction [13, pp. 2–3].

Long sequences make simple recurrent networks difficult to train because gradients can diminish or grow through repeated updates. Long short-term memory (LSTM) uses an additive memory state and gates to control what is retained, replaced and exposed [3, pp. 4–5]. The convolutional feature columns usually outnumber the transcript characters, so their alignment is initially unknown. Connectionist temporal classification (CTC) adds a blank symbol and sums the probabilities of the paths consistent with the transcript. Repeated adjacent symbols are collapsed before blanks are removed [14, pp. 2–4]. Figure 2 shows the resulting recognition path.

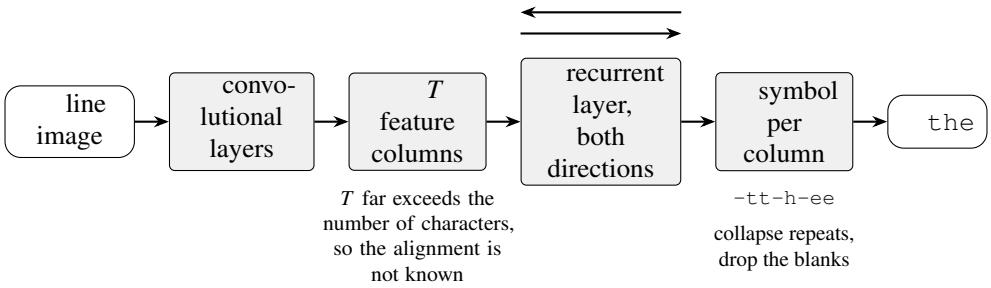


Figure 2. Reading a whole text line without cutting out characters, after the arrangement of Kizilirmak and Yanikoglu [12, pp. 4–5]. The convolutional layers produce T feature columns, a recurrent layer reads them in both directions, and a symbol including a blank is emitted per column; collapsing repeats and removing the blanks gives the transcription.

This combination is used for the project's reader because the training labels describe complete lines. The CNN extracts stroke patterns, the bidirectional LSTM retains their horizontal context, and CTC trains the network without supplied character positions. For author identification, the same convolutional architecture is useful, but its features must describe the writer across a line

rather than decode an ordered transcript. The two tasks therefore require different output heads and separately trained weights.

1.1.4 Training and assessment

The way networks are trained corresponds to the architecture they are built from. Batch normalisation subtracts the batch mean and divides by the standard deviation, with a stabilising constant and learned scale and offset [15, p. 4]. Dropout reduces overfitting by removing a random subset of a layer's outputs during training, limiting dependence on individual features [16, pp. 1931, 1934]. Data augmentation introduces small rotations, shears or smooth distortions while retaining the transcription [17]. These methods address different sources of variation and do not replace separate validation data.

Recognition is commonly assessed through character error rate (CER) or word error rate (WER), using insertions, deletions and substitutions relative to a reference transcript [14, p. 2]. An error in one character can spoil a whole word, but word edit error is not simply a count of words at matching positions. The metric and its reference population must remain explicit when comparing systems.

For this project, numerical forward passes, backward passes and optimiser updates make the learning calculations inspectable. A corresponding library implementation supports training and comparison, with stored arrays used for NumPy inference on the ODROID. The report distinguishes the selected library-trained weights from the independently trained numerical models. Character edits, writer decisions and physical drawing measurements are evaluated separately because they measure different functions.

1.1.5 Techniques for writer identification

There are multiple approaches to identifying the author of a written piece of text. One uses measured, text-independent features such as slant, height, stroke width, spacing and connectivity. Another describes the distribution of edge directions and recurring letter fragments, giving a texture and shape description of the writing [18, p. 701]. Alternatively, a neural network learns discriminative features from examples labelled by writer [19]. Such a classifier identifies a writer among its trained classes; it does not automatically identify an unfamiliar person.

This project needs an author classifier to select and assess the stored writer styles, while explicit geometry measurements are needed to reconstruct their appearance. Ink-trace width can itself contain writer information [20, p. 162], but the gantry uses its own pen. Stroke normalisation is therefore applied to reduce reliance on that cue and direct the classifier towards shape. Writer identity and stylistic fidelity remain separate measures: recognising an author does not establish that every letter or spacing feature has been reproduced accurately.

1.2 Handwriting synthesis in a writer's style

Before handwritten text can be reproduced through hardware, the desired text must be created in the selected author's style. Four approaches illustrate the trade-offs between source data, learned variation and the form of the output.

Recurrent stroke-level synthesis. A recurrent neural network predicts the next pen movement through small coordinate offsets and an end-of-stroke indicator. It models a distribution of possible movements because no letter is written identically every time [3, pp. 18–20]. Its output is naturally suited to a plotter, but training uses online recordings. This project's offline pages contain no measured stroke order, so this approach would require a different source dataset and an additional generative model.

Adversarial image synthesis. A generative adversarial network trains a generator and discriminator together: one generates handwriting, while the other helps distinguish generated samples from real examples. GANwriting also constrains content and writer style to produce text images [21, pp. 3–7]. Once trained, the generator produces an image in a forward pass. This is a relevant alternative for learning contextual letterforms, but a raster output still needs conversion into the pen paths required by this project.

Diffusion-based synthesis. This method learns to reverse a noise-corruption process and generates handwriting through successive denoising steps. One-DM uses one reference style image at generation time, while its training uses a much larger handwriting corpus [22, pp. 3–5, 9]. A small reference requirement must therefore be distinguished from the data and computation required to train the model. For this project, diffusion would add both a separate training task and image-to-path recovery, rather than directly supplying the motion representation.

Glyph-library synthesis. This method assembles words from shapes or glyphs extracted from existing handwritten pages. Haines et al. store labelled forms and select and join them using information about their shape, context and spacing [23, pp. 4–8]. Their standard data collection uses approximately four A4 pages per writer [23, p. 4]. This suits the project's small personal collections and allows explicit control of size, slant and spacing. The main limitation is that a finite collection of character shapes and joining rules only approximates the way connected handwriting changes with its neighbours.

Generated handwriting can be assessed through geometric comparisons, recognition and human judgement. Haines et al. include human evaluation, while image-generation work also uses text and writer recognition to assess the output [23, pp. 12–14], [21, pp. 8–9]. For the present robot, model scores help choose a candidate before drawing, but final physical output also needs its own read-back and geometric assessment. A classifier used to select the candidate is not an independent human judgement of its style.

1.3 Physical reproduction and pen-path mapping

There are three relevant families of mechanism for reproducing text on paper: Cartesian systems, serial-chain robots and parallel-chain robots. A Cartesian system consists of a gantry with independent x–y movement. Serial-chain robots consist of linkages connected in a chain, like a typical robot arm. Parallel mechanisms use several linked chains to support the moving platform [24, pp. 5–8]. These arrangements differ in their workspace, mechanical structure and coordinate transformations.

A Cartesian gantry is suitable here because the page is flat and the main motion requirement is two-dimensional. Each axis can be commanded and calibrated separately, with a further pen-lift mechanism for travel. This makes the conversion from an image-derived path to physical

coordinates direct, while leaving frame alignment, transmission compliance and missed steps as practical accuracy concerns.

A pen path is stored as a list of points, and a recovered stroke usually has more points than a plotter needs. Polyline simplification reduces this list while retaining the stroke's shape. The Douglas–Peucker method joins the endpoints, finds the point furthest from that line and retains it when its distance exceeds a chosen tolerance, then repeats on the resulting parts [25]. A small tolerance follows the stroke closely; a large tolerance removes more points but can cut corners. When handwriting exists only as an image, the path must first be recovered, for example by thinning the ink to a centreline [26, p. 237].

These operations are needed to turn the project's stored glyphs into manageable G-code trajectories. Simplification controls geometric detail, while the thinning and tracing determine which strokes are available to the synthesiser. Neither recovers the original writing speed or pressure from a photograph, so the controller must define its own drawing feed and pen transitions.

There are multiple methods of linear actuation that could be considered for a Cartesian plotting system, including belt drives, rack-and-pinion drives and screw drives [27]. The project uses timing belts and pulleys to convert motor rotation into movement along supported rails. This choice prioritises a straightforward, affordable page-sized mechanism with sufficient travel speed; its nominal step resolution must still be distinguished from measured positioning accuracy.

1.4 Application of the literature

The literature supports a system built around offline pages, complete-line recognition and explicit glyph geometry. Image conditioning and curved line separation prepare photographed input; convolutional features support separate text and writer outputs; transcript-guided extraction turns observed handwriting into reusable paths. A glyph library then supplies controllable geometry without training an additional handwriting generator. The Cartesian gantry provides the physical output stage. Section 2 explains how these choices were combined and developed, while Section 3 gives their calculations, implementation and verification.

2. Approach

The project was approached as two interacting systems: one interprets photographed handwriting and constructs writer-specific paths, while the other converts those paths into physical pen movement. The proposal's functional analysis provides their responsibilities; the G-code interface allows their implementation and verification to proceed separately.

2.1 Mechanical Hardware Design

Before any computer vision or neural networks were worked on, the hardware to perform the pen plotting was created. This is because the hardware does not rely on any of the coding elements to be complete to be able to be tested, and hardware can experience major delays in production due to order delays, stock shortages or component failure, adding to the reason for it to be constructed first. A Cartesian gantry with two independent axes was chosen over some sort of robotic writing arm, as system requiring only flat, cartesian movement does not need joint-dependent kinematics, and each axis can be calibrated separately and work indepedently from one another. To achieve the linear motion required, GT2 timing belts and pulleys were chosen over a lead-screws due to their lower cost, faster travel and ease of customizstion.

Two NEMA 17 stepper motors were chosen for the X and Y axes, driven by A4988 driver chips. Because a stepper motor always rotates by precisely the number of commanded steps at exact intervals, accuracy could be assured without an encoder. To allow for even more increased accuracy and minimal friction, linear bearings were used within the mechanics of the system to allow parts to move with ease and precision, instead of just having the printed supports sliding over the steel rails of the gantry. The use of microswitches on the axes allowed the system to be able to calibrate to reproduce text more accurately. The gantry was designed to be able to reproduce any G-code file that was provided, allowing for easy integration with the other subsystems in future.

2.2 Page condition and line segmentation

In parallel while working on the gantry and waiting for hardware to be delivered, work was done on the first processing stage of the software pipeline, which was performing image conditioning and segmentation of those pages into a format the future classifiers would accept. A photograph produces a wide range of inconsistencies that need to be dealt with, including uneven lighting, page rotation, printed rules, and strokes from neighbouring lines merging into each other. The image conditioning part of the code was designed to remove these undesired page inconsistencies without ruining the quality of the original writing, using methods like lighting normalization, ink separation, and skew correction.

Segmentation of the page of text was chosen to be done lines at a time instead of per word or character, as it was found that dynamic word separation that worked in all cases was difficult, and line separation has the added advantage of more context to train and classify text. The design chosen to perform the segmentation involves locating line centres first, finds curved boundaries between the lines and its ascenders and descenders, and only then uses the geometry of each ink piece to decide which line actually owns it. Deskewing, removal of image background, and artefact rejection were deliberately ordered to run before line centres are estimated, while

recovery of faint ink was left until after ownership of each piece had already been decided, since doing it earlier risked reattaching noise to the wrong line. The system fomatted each page into the ideal crop sizes to be passed into the text and author classifiers.

2.3 Text and writer classification

Once clean line crops were available, the next stage had two jobs: reading the text and identifying who wrote it. Both depend on the same stroke features, so a single convolutional backbone was built, but then the writer and author classifiers varied after that point. The text classifier uses a CNN–BiLSTM architecture with decoded CTC output, which was heavily based off existing research, and chosen because it reads a whole line without needing individual characters pre-cut, as mentioned in the segmentation section, and cursive text has no reliable character boundary to cut at in the first place.

Training data came from a mix of public handwriting (IAM) and the project's own photographed pages, and both author and text models were trained using hold-out pages or lines to allow the system to be validated against, to prove over-fitting did not occur. The writer network's input was stroke-normalised rather than using varying ink thickness from real pages, specifically because the gantry's own pen produces constant stroke thicknesses, regardless of the author.

2.4 Style extraction and handwriting reproduction

Once the previous subsystems were created, the actual text synthesis and reproduction was made. Four reproduction methods were considered, and the method chosen instead was a stored set of normalised glyph trajectories. The other were not chosen because an online dataset of pen strokes was not available, and the chosen IAM dataset was not extensive enough to train a rpeproduction model, neither was the necessary GPU processing power available for it. Using the glyph trajectory method simply resused data and models the project already had, but at the cost of weaker handling of how letters join and vary with context compared to a true generative model especially for heavy cursive styles. The new text to be created is then assembled from the stored glyphs rather than generated from scratch, giving direct, inspectable control over scale and spacing before anything is sent to the gantry. Extracted features like slant, spacing etc. are also applied to the synthesised text.

2.5 Integration and testing

The integration stage formed a complete pipeline of the system through a web server, with G-code remaining the constant fixed interface between the software reproduction output and the gantry hardware. The web interface was designed to be able to preview and inspect generated G-cocdes before any gantry operation was called. By previewing the gantry before reproduction, the precision of the gantry could be measured, and any errors with gantry movement isolated. Safety features were also included as part of the integration, which included a Emergency abort feature on the web server to manually stop any gantry command, as well as end-stop detection with switches to ensure the motors did not stall if the end of the rail was ever reached.

3. Design and implementation

The design follows the hardware, page processing, classification and reproduction stages, then their integration.

3.1 Design summary

This section summarises the project design tasks and how they were implemented (see Table 1). The tasks follow the deliverables and student design work in the approved Project Proposal. The qualification tests in Results assess numerical compliance.

Deliverable or task	**Implementation**	**Completion and report location**
Investigation of methods	Handwritten text recognition, author recognition, image conditioning and reproduction methods were compared, and design choices were made.	Completed, seen in Literature Review, Sections 3.5, 3.6.
Page capture and image conditioning	A fixed USB camera module captures handwritten pages, and an image-processing pipeline conditions the page before segmentation	Completed, seen in Sections 3.3.1 and 3.4.
Separation of handwritten text	The segmenter creates boundaries between lines while preserving the quality of the text, to produce line crops.	Completed, seen in Sections 3.4, 3.6.2.
Text and writer classifiers	Separate networks recognise line text and identify the writer from convolutional features, bidirectional LSTM laters and CTC decoding.	Completed, seen in Section 3.5.
Network training implementations	Training and backpropagation implemented from first principles calculates the network outputs, loss derivatives and parameter updates to train the networks.	Completed, seen in Sections 3.5.5–B.2.
Writer-specific trajectory maps and storage	Character glyphs are extracted, converted to pen paths are stored with each writer's style features.	Completed, seen in Sections 3.6.2-3.6.5.
Handwriting synthesis and G-code	The synthesiser lays out glyphs, connected portions of the sentence, selects a candidate and converts its paths to machine coordinates.	Completed, seen in Sections 3.6.6-3.6.8.
Gantry, pen mechanism and motion control	The gantry combines a two-axis belt drive, switched pen lift, end stops and calibration to create precise cartesian movements.	Completed, seen in Section 3.3.
Processing and integration	Web server display connects capture, processing, stored profiles and execution.	Completed, seen in Section 3.7.
Subsystem verification	Numerical checks and offline image and path experiments verify the stages. Results report the integrated qualification.	Completed, seen in Sections B.2, 3.4.7; Appendix B.

Table 1. Design summary.

3.2 System block diagram

Figure 3 groups the hardware into five parts. External power (1) supplies the ODROID and motor drivers. External interfaces (2) provide page restraint, camera images and operator access. Stepper control (3) converts GPIO commands into winding currents. The belt-driven gantry (4) moves the pen in x and y. The pen mechanism (5) switches contact with the page and reports its state. Table 19 in the appendix gives the individual connections.

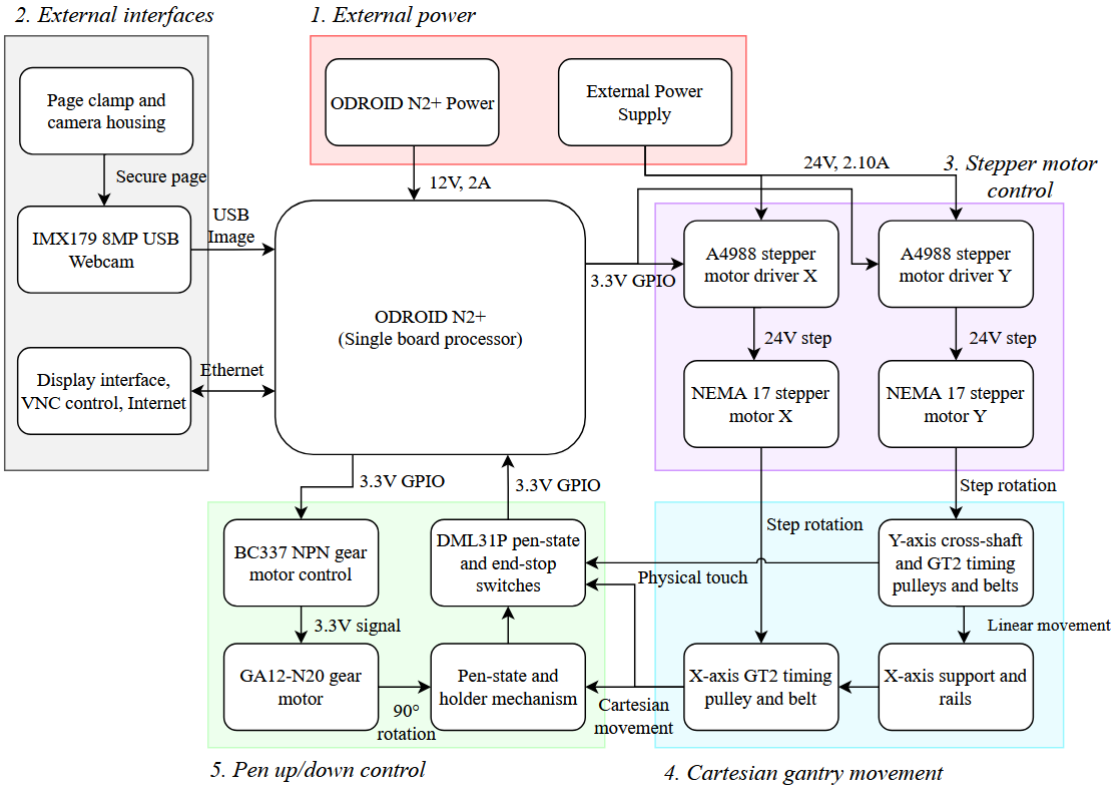


Figure 3. Hardware system block diagram.

Figure 4 shows four software groups. Segmentation (1) conditions a page and returns line crops. Reproduction (2) assembles the selected profile into trajectories and G-code, then executes the path after calibration. Training (3) uses labelled crops to fit the text or writer model and builds stored glyph profiles from labelled handwriting. Classification (4) recognises text or identifies the writer, applying stroke normalisation before writer inference. The diagram groups these functions, while their data dependencies and separate training stages are detailed below.

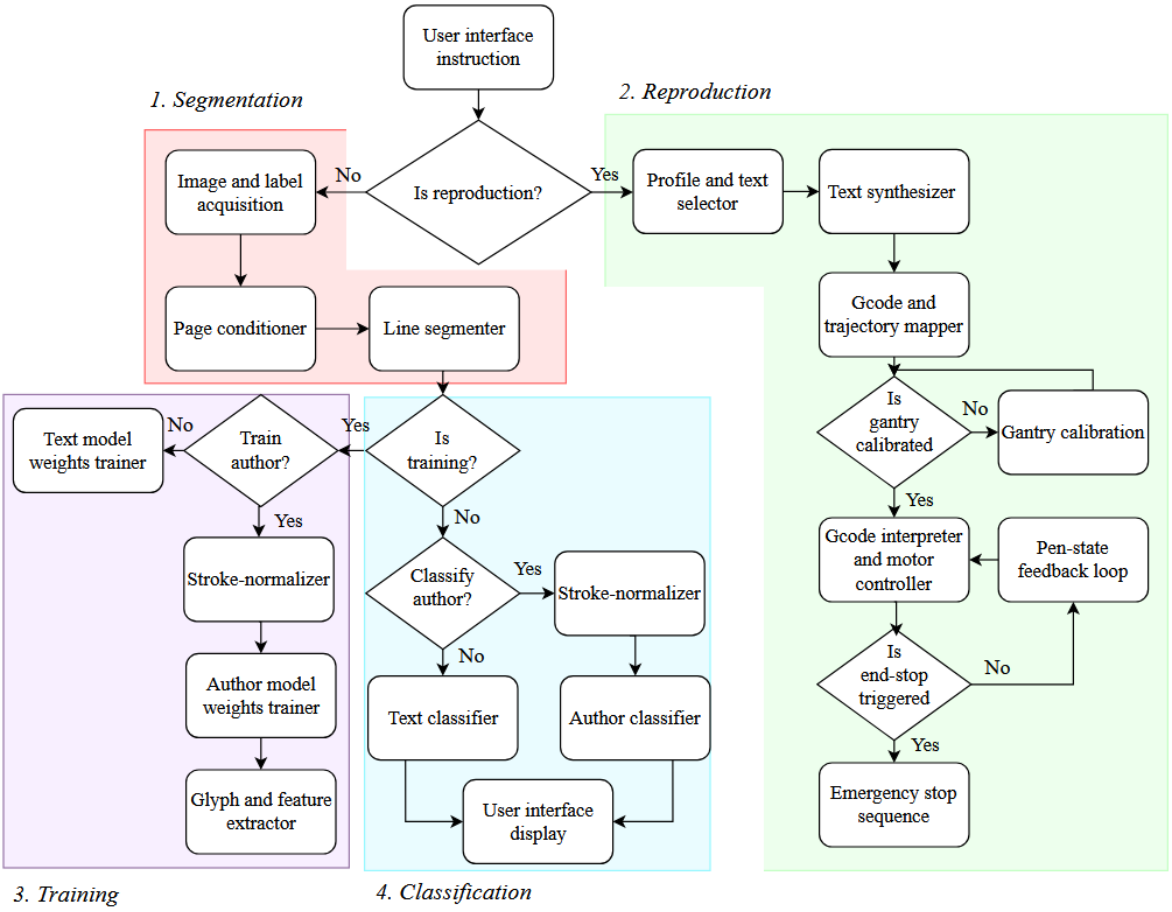


Figure 4. Software system block diagram.

3.3 Hardware Design

3.3.1 Plotter designs and mechanics

The chosen method of text plotting is a Cartesian gantry system with 2 independent axes of movement with a pen up/down mechanicsm. Complex kinematics equations and error dependencies are avoiding by using 2 isolated x and y movement methods for reproduction [24]. Linear GT2 timing belts and pulleys are used to convert rotary motion from stepper motors and axles to the desired cartesian movements. Timing belts are used for their reliability, cost effectiveness, and higher speed travel [27].

A Cartesian gantry consists of a stationary rail that controls movement in one axis, and a movable rail that controls the other axis of movement. For this system and future explanations, the stationary rails control y-axis movement, while the x-axis rail moves along the y-axis. Figure 5 shows the fixed rails guiding *y* movement, the moving *x* rail and the separate pen lift.

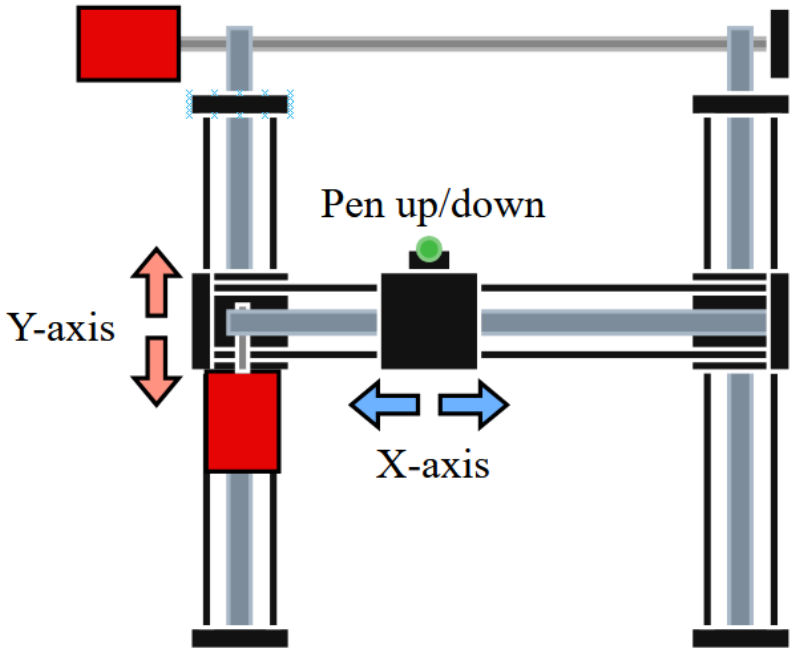


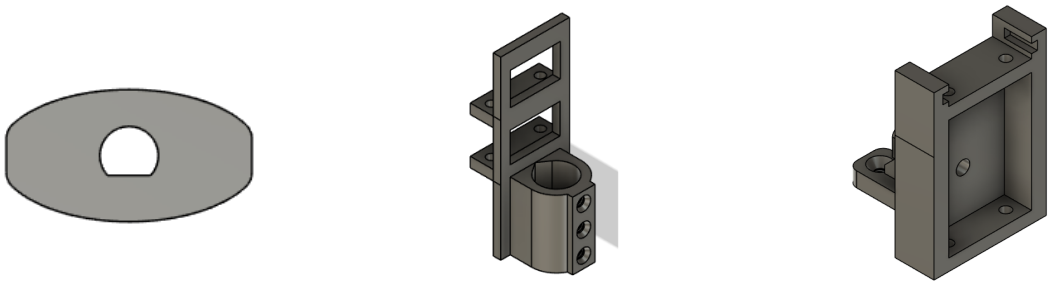
Figure 5. Cartesian gantry axes and pen up/down mechanism.

The x-axis rail needs to move with stability and consistency, which is why it is supported on both ends by dual-rails on the y-axis. A singular stepper motor was chosen to drive both pulleys on both y-axis rails to ensure synchronicity and save costs. 2 independent stepper motors driving one axis creates synchronicity issues because any difference in timing, missed steps or load torques could offset the movement between the two motors. The challenge with using one motor to drive two timing pulleys, is finding a method to extend the shaft of the motor to connect to both timing pulleys without overloading the motor and applying too much tortion or twist to the new axle. A rigid coupler was used to extend the axle of the stepper motor to a chromed steel rod.

To reduce strain on motors, and allow for increased accuracy and precision, linear bearing were fitted around the chromed rod rails to allow all parts to slide with very little friction. To verify the stepper motor could handle the weight of the x-axis rail further, the torque required to move the rail at the desired speed was calculated. It is described in Section 3.3.2 with the descriptions of the stepper motor type chosen and its product specifications.

All CAD models for this project were designed from first principles using Autodesk Fusion360, with no off the shelf housings used, including all designs for the gantry system as well as the camera housing module. Designs were created by identifying off-the-shelf hardware components that were available and had to be acquired, and designing around these components with decent tolerances to allow for easy contruction and precision.

The three pen-lift parts in Figure 6 work together: the cam rotates by 90 degrees at a time to control the lift, while the mounting pieces hold the pen in position relative to its up/down state, with springs constantly applying a downward pressure to act as dynamic pen pressure control.



(a) (b) (c)

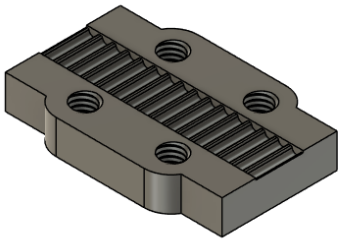
Figure 6. CAD models of the cam and pen-mounting parts.

Figures 7 and 8 show the different mounting part used for the drive chain, used to hold the timing belt pulleys as well as the stepper motor in place. Chamfered threaded screw holes are added so the design maybe be screwed fastly into the correctly position on the base with M4 bolts.

(d) (e)

Figure 7. CAD models of the drive support brackets.

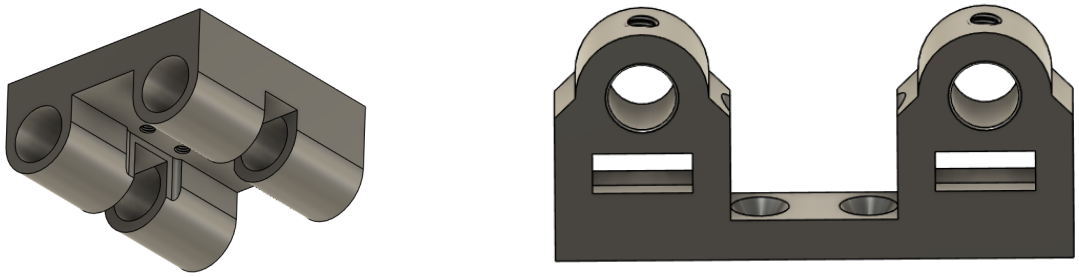
The toothed belt grip shown in Figure 8 secures the belt to the moving assembly, transferring belt motion to the gantry. It is designed to be removable and adjustable to allow for tightening or loosening of the timing belt.



(f)

Figure 8. CAD model of the toothed-belt grip.

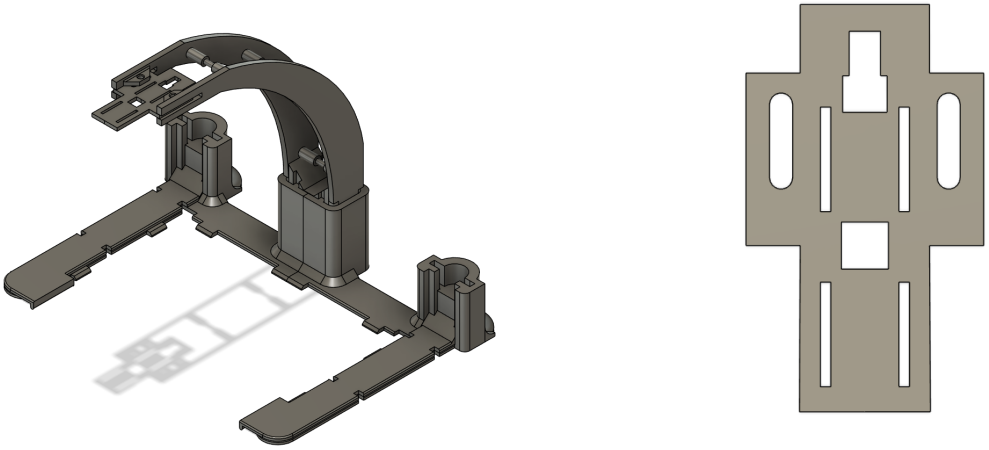
The bearing and rod supports in Figure 9 locate the rails and guide the moving parts along them.



(g) (h)

Figure 9. CAD models of the bearing and rod supports.

The camera housing was designed to hold the IMX179 USB webcam module securely. Its single curved supporting arm is intended to reduce shadows on the page. Page clamps hold each page securely in place to minimise rotation and skew errors. Figure 10 shows the camera housing and page-clamp designs; panel (a) shows the assembled housing and clamps, and panel (b) shows the camera housing plate that slots into the support structure.



(a) Camera housing, curved support arm and page clamps.

(b) Page-clamp component.

Figure 10. CAD designs for the camera housing and page clamps.

3.3.2 Stepper motor and driver

A stepper motor does not continuously turn with a PWM input, but rather turns at specific fixed angles each time there is a switch in the windings (the coils of the stepper motor). The A4988 regulates the winding current through a process called chopping, which switches the current supply on the winding until the current rises to a certain point, and then switches off at the target current. This occurs at high frequencies and effectively keeps the current going to the coils within a specific range so as to not overshoot, even when the motor is not actively stepping. A step is created by slighty adjusting the current value between 2 coils just enough to allow the rotors teeth to make a small jump or 'step'. The stepper motor is perfect for the precise movements required

for handwriting reproduction with the gantry, as it produces steps at exact, known intervals, with position tracked from commanded steps rather than an encoder. This assumes that the motors do not miss steps.

Two stepper motors are used in the gantry plotter, one for x and one for the y-axis direction. The type of stepper motor chosen was the 42BYGH40-01 Nema 17 bipolar stepper motor, with a resolution of 200 full steps per revolution and a current rating of 1.68A per phase [28].

The following calculations were done to determine if the Nema 17 stepper motor would be strong enough for the use case of the gantry plotter, and if a singular motor could control both y-axis belt drives safely.

Dziedzic et al. examined samples written at 10–60 mm/s [29]. The maximum handwriting speed mentioned was 3600mm/min, which was rounded up to 4000mm/min for headroom and approximately 10% tolerance.

For the 20-tooth GT2 pulleys with 2mm belt pitch that were used for the gantry, the belt travel during one rotation is calculated as follows gives [30]

$$\ell_{\text{rev}} = N_t p = 20(2) = 40 \text{ mm/rev}, \qquad r_p = \frac{N_t p}{2\pi} = 6.37 \text{ mm}. \tag{1}$$

N_t refers to the number of teeth on the pulley, p is the belt pitch, and ℓ_{rev} is the belt travel during one full pulley revolution. The pitch radius r_p is the effective radius of the belt path around the pulley.

Speed-to-torque margin check. The chosen max gantry speed to achieve is converted to a motor speed and compared with the torque required to move the assembly.

$$n = \frac{v}{\ell_{\text{rev}}} = \frac{4000 \text{ mm/min}}{40 \text{ mm/rev}} = 100 \text{ RPM} \tag{2}$$

where n is the motor speed in revolutions per minute, $v = 4000$ mm/min is the max gantry speed decided upon as previously mentioned, and $\ell_{\text{rev}} = 40$ mm is the belt travel per revolution previously calculated. To then calculate the pull-out torque the system experiences at this speed, interpolation is done on the NEMA 17 pull-out torque curve found in SOURCE

$$T_{\text{po}}(n) = T_1 + \frac{n - n_1}{n_2 - n_1}\left(T_2 - T_1\right) = 7 + \frac{100 - 60}{120 - 60}(21 - 7) \approx 16.3 \text{ Ncm} \tag{3}$$

where $T_{\text{po}}(n)$ is the interpolated pull-out torque estimate at speed n, and the measured curve readings are $(60 \text{ RPM}, 7 \text{ Ncm})$ and $(120 \text{ RPM}, 21 \text{ Ncm})$.

To then calculate the torque required at constant velocity, the following formula and calculation is done, based off of bearing friction, mass, and the previously calculated effective radius of the belt path. SOURCE

$$T_{\text{req}} = r F_f = r \mu m g = 0.006366 \times 0.004 \times 0.950 \times 9.81 \approx 0.0237 \text{ Ncm} \tag{4}$$

where T_{req} is the torque needed at constant velocity, $r = 6.37$ mm is the pulley pitch radius , $\mu = 0.004$ is the original assumed bearing-friction coefficient, $m = 0.950$ kg is the complete moving assembly and $g = 9.81$ m/s^2.

The calculated torque needed at constant velocity to move the 0.95kg x-rail, is only 0.0237 Ncm, which is significantly lower than the interpolated pull-out torque of the NEMA stepper motor at the maximum use case travelsal speed. (16.3N)

Figure 11 shows the selected motor and its A4988 driver.

Figure 11. NEMA 17 motor and A4988 driver.

3.3.3 Step resolution, microstepping and calibration

Three properties of a stepper drive fix the resolution of this machine. The motors used here take 200 full steps per revolution, so one full step is 1.8°; because position is simply the number of steps that the motor was signlalled to produce, with no encoder or feedback loop required to verify this due to the consistent nature of the stepper motors. The driver subdivides each full step by microstepping: rather than switching the two windings fully on or off, it holds them at intermediate current ratios, placing the rotor at intermediate angles in the stepper motor. At the 1/16 setting on the driver, one full step becomes 16 microsteps, so a revolution is 3200 microsteps. This maximum microstepping option was used for this project because it has increased resolution and smoother motion. The only downside is that at higher load torques, steps could slip due to the intermediary position of the motors.

The following equations are used to determine if the stepper motor resolution will be fine enough to fulfill the 0.5mm gantry precision tolerance requirement specified in the project proposal. Equation 5 gives the step resolution from 200 full steps [28], 16 microsteps [31, p. 6] and $\ell_{\text{rev}} = 40\,\text{mm}$,

$$K = \frac{200 \cdot 16}{\ell_{\text{rev}}} = \frac{3200}{40} = 80\ \text{steps/mm}, \qquad \Delta = \frac{1}{K} = 12.5\ \mu\text{m}, \tag{5}$$

where K is the steps/mm resolution of the stepper motor driver, and Δ is interval step size of the stepper motor. The Quantisation error is at most $\Delta/2 = 6.25\ \mu$m per axis, 8.8 μm combined over both axes as a pythagorean measurement, which is significantly smaller than the 0.5mm tolerance of R5. Multiple other factors could still affect the gantry precision including belt stretch, backlash, missed steps, switch repeatability and flexibility in the pen housing.

Coil current and its derating. To calculate the appropriate V_{ref} to set with the potentiometer on the A4988 stepper motor driver, the datasheet relation [31, p. 9] between the the carrier's two sense resistors and the regulated current of the stepper motor and driver gives

$$I_{\text{coil}} = \frac{V_{\text{ref}}}{8\,R_S} \quad , \quad V_{\text{ref}} = 8\,R_S\,I_{\text{coil}} = 8 \cdot 0.1 \cdot 1.68 = 1.34\ \text{V} \tag{6}$$

for the 1.68 A motor rating [28], conditional on the R10 sense-resistor marking meaning 0.1 Ω. This gives gives the safe maximum operating V_{ref} for the system to operate at, and the potentiometer on the A4988 stepper motor driver was tweaked to match the V_{ref}.

3.3.4 Pen up/down mechanism

The pen-lift mechanism itself is a design choice justified against the alternative of a third stepper or a servo motor. A servo motor expects a continuous pulse-width-modulated position command and typically needs its own timer a precisely timed software loop running concurrently with the gantry's own step timing, competing for the same processor cycles that generate STEP pulses. An N20 geared d.c. motor run in one direction only, with a cam that toggles the pen up or down every 90° of rotation, needs only an on/off drive pulse of roughly known duration and a microswitch to act as the feedback loop to verify the pen up position. It is cheap, widely available, has high torque at low speed because of its internal gearbox, has an incredibly small profile, and has no accuracy or positioning cost due to the use of a switch for the feedback loop.

Figure 12 shows the cam holding the pen in its two positions.

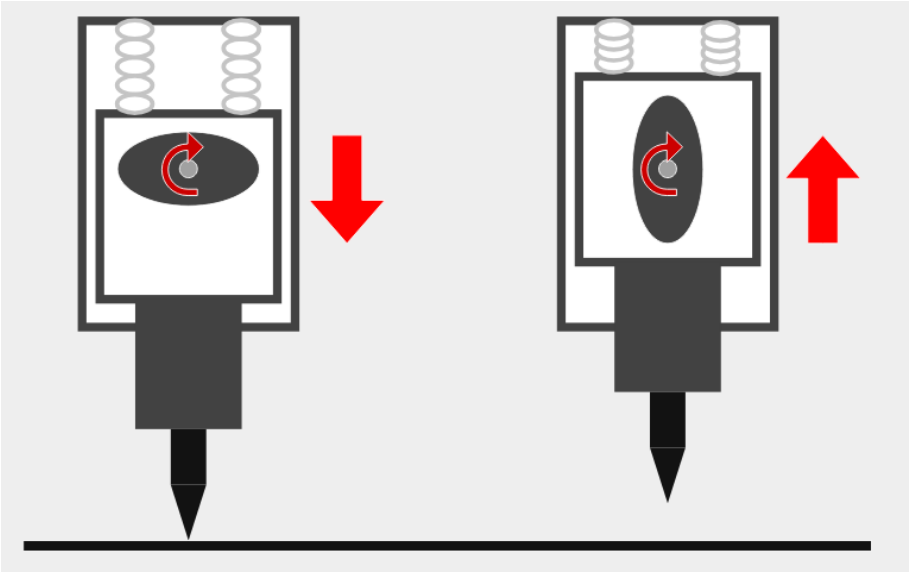


Figure 12. Pen lowered (left) and raised (right).

The implemented lift, planned in the proposal as a stepper, gives only pulses, so the pen state must be tracked in software, and the pen switch is feedback only (high = up). The pen-drive routine of Algorithm 1 must therefore test the switch before energising the motor, since the cam turns one way only and an unnecessary pulse would toggle the pen to the wrong state rather than leave it alone. The controller runs the motor until the requested switch state is confirmed or a timeout occurs, with a 40ms overrun after pen-up confirmation. The generator's separate settling

dwell is 350ms default setting, done through visual verification to make sure the pen was at its highest state when the up command was triggered.

Pen motor driver and base-resistor selection. The GA12-N20 geared motor was selected to turn the pen cam with a compact, low-speed geared output. The listed 3 V, 150 rpm variant has a specified voltage range of 1.5–5 V, a no-load current of 10 mA, a rated current of 30 mA, and a stall current of 230 mA [32]. The supplied circuit uses the Odroid's 3.3 V GPIO signal (max 4mA output SOURCE) to drive the base of a BC337 NPN transistor. The transistor is a low-side switch: motor current flows from the 3.3 V DC motor rail, through the motor and collector-emitter path to the shared ground, while the GPIO supplies only base current [33]. The source symbol in the supplied diagram should therefore be read as a DC supply, not an AC source.

The stated 4 mA maximum GPIO output current sets a *minimum*, not a maximum, base-resistor value. Assuming a base-emitter voltage of approximately 0.7 V, the minimum resistance is

$$R_{B,\min} = \frac{V_{\text{GPIO}} - V_{BE}}{I_{\text{GPIO,max}}} = \frac{3.3 - 0.7}{0.004} = 650\,\Omega. \tag{7}$$

A standard 1 kΩ resistor was selected, which is above this minimum. Its nominal GPIO/base current is

$$I_B = \frac{V_{\text{GPIO}} - V_{BE}}{R_B} = \frac{3.3 - 0.7}{1000} \approx 2.6\,\text{mA}. \tag{8}$$

The nominal 2.6 mA base current is below the stated 4 mA GPIO output limit. Using a forced gain of 10, as in the BC337 saturation test [34], gives a conservative 26 mA collector-current check. This is below the listed 30 mA motor rating, so saturation at rated motor current is not established by the base-resistor calculation alone.

Figure 13 shows the transistor circuitry, with connections to the motor that spins the pen mechanism, a 3.3V source, ground, and the GPIO pin of the ODROID N2+.

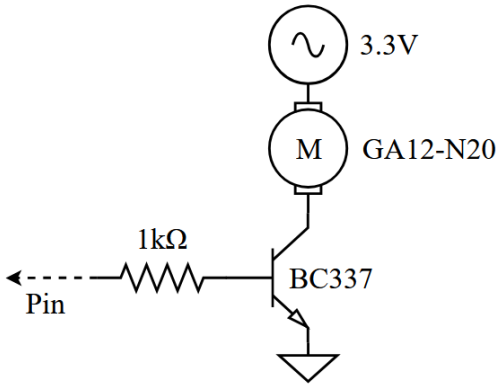


Figure 13. BC337 pen-motor switching circuit.

During tuning the clicker sometimes over-rotated and the pen did not push down fully before the switch read as triggered, requiring the switch timing to be adjusted experimentally. The pressure of the pen different surfaces like a soft clamp surfaces (exam pad) or a hard wooden base surface

Algorithm 1 Pen toggle

Require: desired state wantUp
if any end-stop is currently triggered **then**
  refuse the request
else if the pen switch already reads wantUp **then**
  return {the motor must *not* run; the cam turns one way only}
else
  energise the pen motor
  while before the deadline and the pen switch does not yet read go UP **do**
    wait
  end while
  if the switch now reads up **then**
    wait 40 ms more to seat the cam
  end if
  de-energise the pen motor
end if

produced variations in the ink thickness, so a rigid clipboard with a single page clamped flat was decided to be used, for consistent pen pressures.

3.3.5 End stop switches and calibration

All four axis end-stop switches are checked before every major axis step, and every G-code line. Debouncing on the pen switches was imlemented digitally to ensure end-stops were never triggered from noise. A switch is only accepted to be triggered after three consecutive high reads 6 ms apart (Algorithm 2). The pen switch is read separately by the pen-control routine.

The DML31P microswitch was selected as a compact lever microswitch for detecting gantry end-stop contact. It is as an SPDT (single-pole double throw) switch rated for 1 A at 125 V AC [35]. In this circuit and for the use case of this project it is used only as a low-current logic contact detection, not to switch on a motor. The diagram uses the common and normally-open contacts so that actuating the switch connects the Odroid N2+'s 3.3 V logic rail to the GPIO input; the source symbol represents DC. A standard 10 kΩ resistor was a pull-down, to ensure the GPIO pin of the ODROID N2+ was provided with a current comfortably than the max 4mA the pin can handle [33]. The circuit holds the input low while the switch is open, preventing a floating input, and draws a small current when the switch closes.

With the switch closed, the pull-down current and resistor dissipation are, proving the current flowing through the system is significantly lower than the 4mA max of the GPIO pins, ensuring safety of the SBC.

$$I_{\text{closed}} = \frac{V_{\text{rail}}}{R_{\text{PD}}} = \frac{3.3\,\text{V}}{10\,\text{k}\Omega} = 0.33\,\text{mA}, \tag{9}$$

Figure 14 shows the switch contact and pull-down that define the GPIO input state.

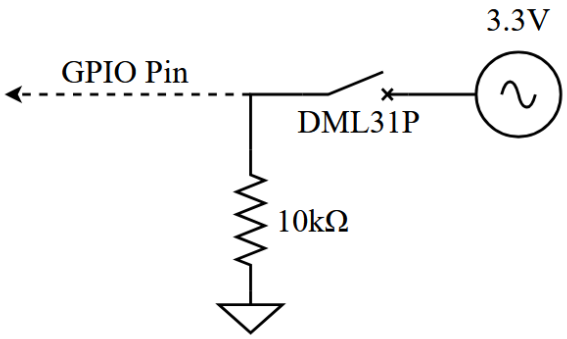


Figure 14. End-stop input and pull-down resistor.

Algorithm 2 Debounced end-stop read, called before every major-axis pulse

Require: set of end-stops to ignore (already triggered on this axis)
1: **for** each end-stop not in the ignore set **do**
2: **if** it does not read closed **then**
3: **continue** to the next end-stop
4: **end if**
5: re-read it twice more, 6 ms apart {three reads in total, debounce of Equation 36}
6: **if** it read closed on all three reads **then**
7: **return** that end-stop as triggered
8: **end if**
9: **end for**
10: **return** none triggered

When an end-stop press is detected, the program enters a stop state by setting both STEP pins low, retracting off every triggered switch and stopping the current gantry task. For the emergency stop to be reset, the end-stop switches must no longer detect contact, for the safety of the system incase the gantry attempts to move further in the direction of the pressed-down switch. A manual force-stop command can also be triggered by the user when the current gantry command must be stopped.

Calibration is also completed using the end-stop switches. The movement of the gantry depends of the knowledge of its current position to ensure it performs operations in the available movement area of the gantry, but the position after power-up is unknown, so each session begins with a calibration sequence that will automatically reposition the pen starting position to the top-left corner independent of where its existing position was on startup. It will also confirm the available page space that can be drawn on. It drives one way at 1.6 ms per step until switch of that axis triggers, reverses, and counts the steps N to the *other*. With the hand-measured travel L, the resolution, the initial step counter and the position in millimetres are as follows:

$$K_{\text{axis}} = \frac{N}{L},, \qquad x_{\text{mm}} = \frac{n}{K} - 5, \tag{10}$$

The 5mm is added for a safe distance away from triggering the end-stop switches accidentally, so the usable area is 190.6×251.3 mm, and each axis retracts 5 mm off its last switch. Using the nominal 80 microsteps/mm, a 1.6 ms homing-step period corresponds to 7.8125 mm/s. Algorithm 3 states the procedure.

Algorithm 3 Dynamic calibration of one axis

drive one way at 1.6 ms per step until either switch of this axis triggers
reverse; count steps N until the other switch triggers
$K_{\text{axis}} = N/L$ {L is the hand-measured travel}
if the second switch to trigger was the maximum-side one **then**
    $n_0 = N$
else
    $n_0 = 0$
end if
retract 5 mm off the switch that is still closed
if an already-triggered switch, an unexpected switch on the other axis, or 200 000 steps pass with no trigger **then**
    abort calibration; refuse all jobs until it is re-run
end if

3.3.6 Gcode generation and motion mapping

G-code are used to store movement and pen-state commands in a text file. The hardware controller reads each command and its parameters, then converts the coordinates that it must go to into motor pulses. Table 2 describes the commands and parameters used in this projects G-cocde files. The standard G-code commands to travel between coordinates do not include commands to lift or lower the pen. The meanings of `M3` and `M5` are the new commands used in this software, where they operate the pen-lift mechanism.

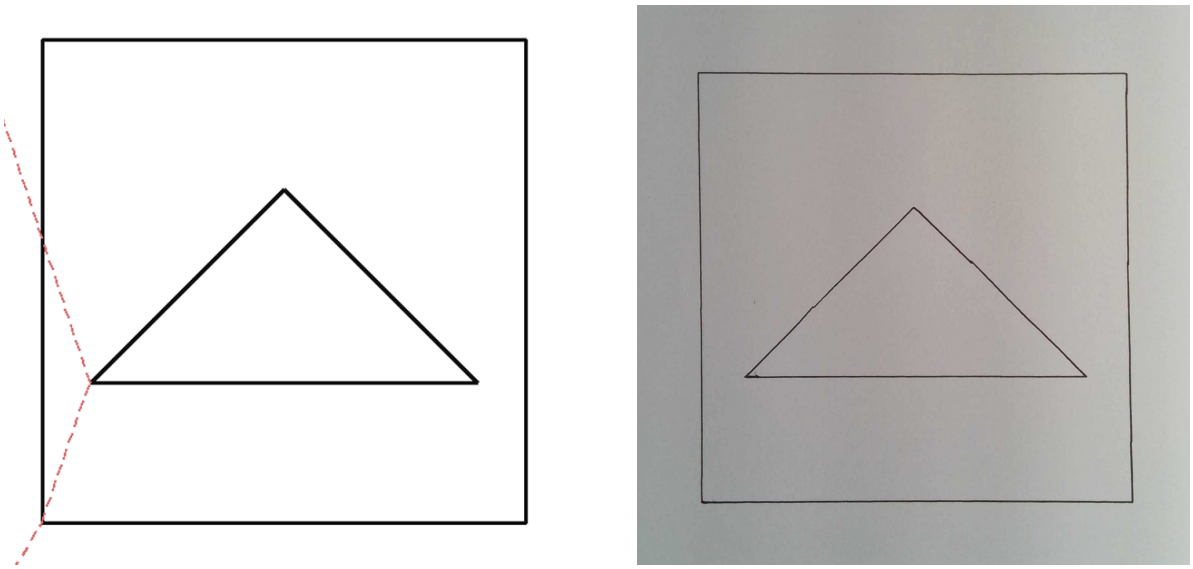
Command or parameter	Meaning in this plotter
`G21`	Select millimetres as the units for coordinates and feed rates.
`G90`	Use absolute positioning, so each X/Y coordinate is measured from the machine origin.
`G0`	Move rapidly without drawing, used for positioning and travel with the pen raised.
`G1`	Make a straight, feed-controlled move; with the pen down, this draws a line.
`G4 P`	Dwell for the duration specified by P in seconds; used here to let the pen-lift motor finish moving.
`M3`	Lower the pen using the plotter's one-way pen-lift motor.
`M5`	Raise the pen using the plotter's one-way pen-lift motor.
`M2`	End the G-code program.
`X`, `Y`	Target coordinates in millimetres when used with the active positioning mode.
`F`	Feed rate for movement, in millimetres per minute with `G21` active.
`P`	Dwell duration in seconds when supplied to `G4`.

Table 2. G-code commands and parameters used in the square-and-triangle test file.

The following extract shows the G-code commands used to draw a square. Coordinates are in millimetres and use absolute positioning, which means that the pen is lowered before the drawing moves and raised after the square is complete.

```
G21
G90
G0 F2400
G0 X45.000 Y76.000
M3
G4 P0.350
G1 F900
G1 X145.000 Y76.000
G1 X145.000 Y176.000
G1 X45.000 Y176.000
G1 X45.000 Y76.000
M5
G4 P0.350
```

Figure 15 compares the generated path preview of the generated square, as well as an isometric triangle, with the corresponding physical plot. The test file commands a 100 mm square and an 80 mm-base triangle, providing both horizontal and diagonal movements for visual comparison.



(a) G-code path preview (b) Reproduced drawing

Figure 15. Comparison of the G-code preview and the physical square-and-triangle plot.

Interpolating a straight line. Stepping the two axes independently would draw an L-shaped path, so the program interpolates with Bresenham's integer line algorithm [36, p. 28], which involves an error term e that is carried along the major axis of Δx and Δy steps, the major axis is pulsed with a step command every loop, and the minor axis whenever e would go negative. Keeping e in the range of $[0, a_x)$ gives the minor-axis m_k count after k major steps as

$$m_k = \left\lceil \frac{k\,a_y - \lfloor a_x/2 \rfloor}{a_x} \right\rceil, \tag{11}$$

Algorithm 4 states the loop.

Algorithm 4 Coordinated line movement with an integer error accumulator

Require: signed target changes in microsteps

set axis directions; choose major count N_M and minor count N_m
$e = \lfloor N_M/2 \rfloor$
for each of the N_M major-axis steps **do**
  if abort requested or an end-stop is confirmed **then**
    return interrupted
  end if
  pulse major axis; update its count; $e = e - N_m$
  if $e < 0$ **then**
    pulse minor axis; update its count; $e = e + N_M$
  end if
end for

Figure 16 Shows a working example of the commanded microsteps for a separate generated move of $(+6, -28)$ microsteps. Its highest difference from the ideal line is 0.5 microsteps, or $0.5/80 = 0.00625$ mm. This plot shows how the major axis gets pulsed every iteration (the y microsteps), while the error adds up in the minor axis and steps when the error gets too large (the x microsteps)

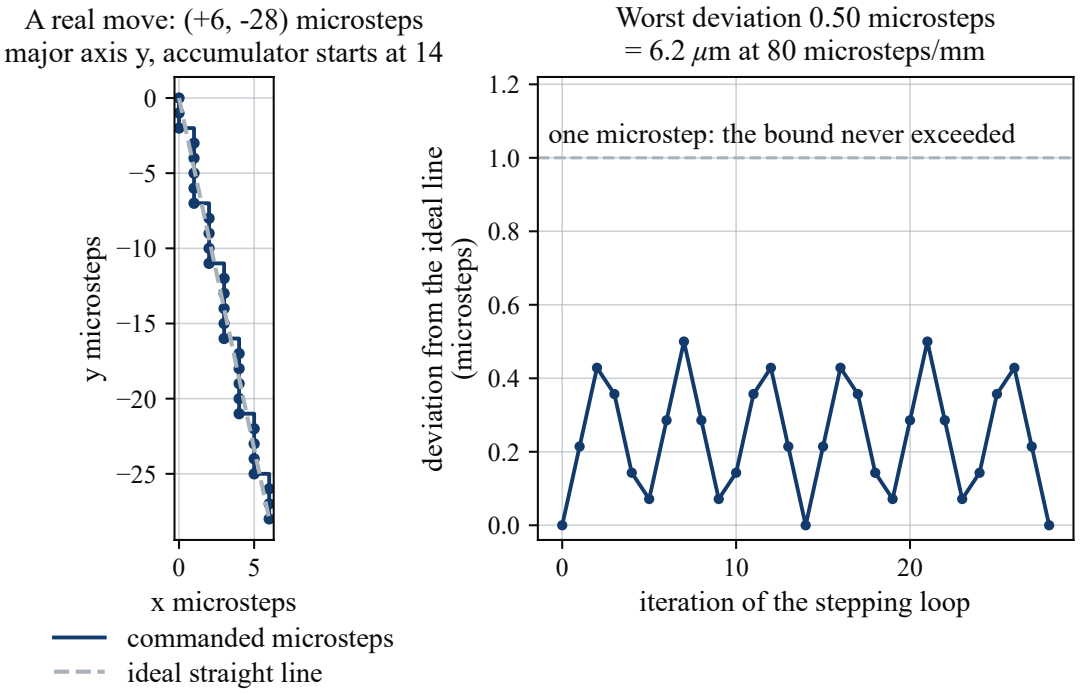


Figure 16. Integer axis stepping and command-space deviation for a generated move.

3.4 Page conditioning and line segmentation

The recogniser needs one line of text at a time. A photographed page can contain shadows, rotation and strokes extending between lines. This stage prepares the page and returns ordered line crops while preserving the shape and spacing of the writing.

3.4.1 Lighting normalisation and ink extraction

Using a single global brightness threshold can mistake a shadow for ink if the page lighting differs. The design compares each pixel with its local background, following the local comparison used in adaptive thresholding, by calculating the background level for an area around each pixel [7, pp. 2–3]. Image analysis uses a 0-255 grey image at half the resolution in each dimension by averaging 2×2 blocks. This is to ensure thin pixel lines get do not get missed. The full-resolution page is kept for the final crops.

Estimating the local background. The local background light level B_r is the average light level of the background of a square window of radius r around each grey pixel. The implemented radius is $r = \max(8, \min(40, \text{round}(H/41)))$, where H is image height. A summed-area table S, also call an integral image is a pre-calculated grid where each pixel stores the sum of all pixels up and to the left of it. It is used to calculate each window sum using four different image integral computations to isolate the window size in the middle. [7, p. 2].

$$\Sigma_{[y_0,y_1)\times[x_0,x_1)} = S(y_1,x_1) - S(y_0,x_1) - S(y_1,x_0) + S(y_0,x_0). \tag{12}$$

The window area divides the sum that was calculated to give B_r. Edge replication is done to avoid darkening or lightening the image boundary with artificial padding. This calculation is independent of window size once the pre-computed image integral is available.

Separating ink from paper. Dividing the grey intensity of each pixel G by its local background calculation gives the ratio R. Ink has a lower ratio than paper. The binary mask M retains pixels below threshold T, to effectively only store ink values according to a certain threshold, and exclude the background. The y and x in each case refers to the position of the pixel in the image.

$$R(y,x) = \frac{G(y,x)}{\max[B_r(y,x), 1]}, \qquad M(y,x) = \mathbf{1}[R(y,x) < T]. \tag{13}$$

The denominator floor avoids division by zero to prevent a maths error. Otsu's method is a technique which analyses the ratio values between the grey intensity of the pixel and the local background calculation, and finds the optimal dividing line to get the correct threshold. Otsu's criterion selects the threshold by maximising the separation between two histogram classes [6]. The class masses are ω_0, ω_1, with mean bin indices μ_0, μ_1.

$$T = \arg\max_t \ \omega_0(t)\omega_1(t)\big[\mu_0(t) - \mu_1(t)\big]^2. \tag{14}$$

The software uses 256 bins to store the ratio values in a range of 0-1.2. Applying the threshold to a brightness ratio makes the decision less sensitive to gradual shadows than applying it directly to the grey image, hence the reason that local background calculation is used. SOURCE

This process was verified through a test example image from the dataset, to verify if an ink pixel was kept or rejected by the threshold. The pixel has an intensity of 100 and local background 161.43, giving $R = 0.62$. Its selected threshold is 0.80, so the pixel is kept as ink. The prefix values and threshold calculation are retained in Appendix A.1.

3.4.2 Page orientation and skew correction

A page that is rotated at 90° has a completely incorrect axis line, meaning text segmentation could not occur, while a small rotation changes the slope of lines on that axis, and importantly still remains in the same line axis. To automatically know when to rotate a page, the software first compares the fraction of nearly empty rows before and after a 90° rotation. A turn requires a score improvement by a factor of 1.35, which will avoid an unnecessary rotation when the evidence is weak and the facotr is not high enough.

Fine deskewing is a process that automatically straightens a document image with a small tilt until it becomes horizontal. It tests seventeen angles from −4° to +4° in 0.5° steps. For each angle $p_\theta(y)$ is the projected count of how much row ink there is. When the ink is rotated and concentrated into its own fixed row, it increases the squared-profile objective [8].

$$\theta^* = \arg\max_\theta \sum_y p_\theta(y)^2, \qquad p_\theta(y) = \sum_x M_\theta(y,x). \tag{15}$$

The search will map the ink coordinates onto candidate rows in a further reduced mask. The selected rotation is then applied to the full and working grey images, and then the binary mask over the page dividing pixels between ink or background is rebuilt. The example test page in the appendix selects +1.0°, with the trial scores retained in Appendix A.1.

3.4.3 Line detection and separation

To properly separate a line, a line centres is used to locate the writing, while curved boundaries separate overlapping ascenders and descenders. A straight horizontal cut can split these strokes even when they belong to different lines, which is not ideal, hence the reason for the curved boundary cuts over ascenders and descenders.

Removing marks and locating line centres. A preliminary line pitch P (the estimated spacing between text lines) sets the scale for filtering the surrounding background artifacts on a scanned page image, and printed rule lines (the straight horizontal lines going across the page). Horizontal and vertical run limits are 1.25 and 1.4 pitches. The pitch is estimated again after these marks and artifact components have been cleared out. A long handwritten underline can also meet a rule-removal condition if it remains within the specified pitch angles.

The final smoothed row profile is created and consists of candidate peaks above 6% of its maximum. Median peak spacing gives the preferred pitch of the lines, with a lower bound based on component height. Autocorrelation and component height provide fallbacks. Centres are selected by peak strength (the max text height within its local range) with at least $0.6P$ separation.

Finding a boundary between lines. A boundary follows a low-cost path across each gap determined by the line centers and pitch. The local cost combines a strong penalty for crossing ink with a weaker attraction towards midpoint y_m, scaled by half-gap height h_g.

$$C(y,x) = 4M(y,x) + 0.012\left(\frac{y - y_m}{h_g}\right)^2. \tag{16}$$

Here M is the mask after morphological closing, which is a dilation operation (pixel expand) followed by erosion operation (pixel shrink), which is a method of expanding then shrinking pixel boundaries to bridge gaps or extend lines [37, p. 541]. Closing with radius $\max(2, \lfloor 0.07P \rfloor)$ fills narrow breaks that could otherwise let the boundary enter a stroke. The path is then computed dynamically using the accumulated cost D [10, p. 3]. The equation that follows is a dynamic path-finding algorithm, used to track the the curves of the text.

$$D(y,x) = C(y,x) + \min\{D(y+d, x-1) + 0.02|d|\}. \tag{17}$$

$C(x,y)$ refers to the local cost of passing through the pixel at x,y, and d refers to the vertical displacement and step size.

Figure 17 shows one gap from Abhinav01. It was selected for the largest reduction in crossed ink on that page. The seam crosses one working-image ink pixel, compared with 64 for the straight midpoint cut. The lower panel shows the cost field that makes the seam bend around the writing. The second example and complete original figure are in Appendix B.3.

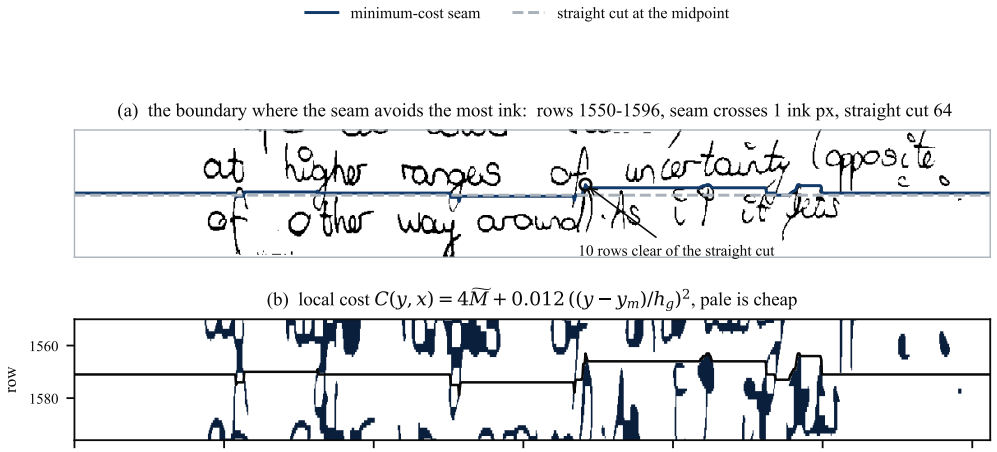


Figure 17. Seam path and its cost field.

3.4.4 Assigning ink to the correct line

A seam divides the page into bands along the calculated boundaries. Dots, ascenders or descenders can still fall into its neighbour's crop. According to research, eight-connected components, found using row runs and union operations, are used to do a check of ownership of extended ink fragments [38, pp. 476–479].

For each component, a band containing at least 15% of its in-band ink is considered important. The uppermost substantial band supplies the reference owner for eligible small-piece corrections.

If none qualifies, the largest overlap in text features is used. Large collections of text generally keep their ownership of text, except for the narrow-tail and small-piece rules.

For example, an `i` dot may lie above the seam while its stem supports assignment to the lower line. Raw components are revisited because closing may join the dot to an unrelated stroke. Enclosed-ring and residual-fragment checks handle further detached pieces. Algorithm 1 gives this order, with O storing each pixel's line index.

Algorithm 1 Refining seam bands into ink ownership

Require: Ink mask M, components $\mathcal{C}$, seams, centres and pitch P.

for each component $C \in \mathcal{C}$ **do**
  Count overlap $a_{C,n}$ with each seam band n
  $S = \{n : a_{C,n} \geq 0.15 \sum_j a_{C,j}\}$
  Choose topmost S as reference owner, or maximum overlap if S is not 0
  Correct eligible narrow tails and small pieces, reject remote little specks
  Associate compact dots with plausible stems using pitch-scaled guards
end for
O = corrected ownership, preserving large-piece seam assignments
for each raw component near a neighbouring line **do**
  if detached-dot or enclosed-ring guards pass **then**
    Transfer eligible pixels to the supported stem's line, protect correction
  end if
end for
Reassign residual fragments lacking their own body-band support
return O, excluded pixels keep no line index

Figure 18 shows the three largest corrections on an example page written by author A. The boxes mark the affected pieces and colour shows the assigned line. The complete page has 462 reassigned pixels. This count actually shows all the rules in action by the colour of their text, with all text of the same colour belonging to the same crop. The first row shows the line before correction, while the second row corresponds to the after correction according to the rules.

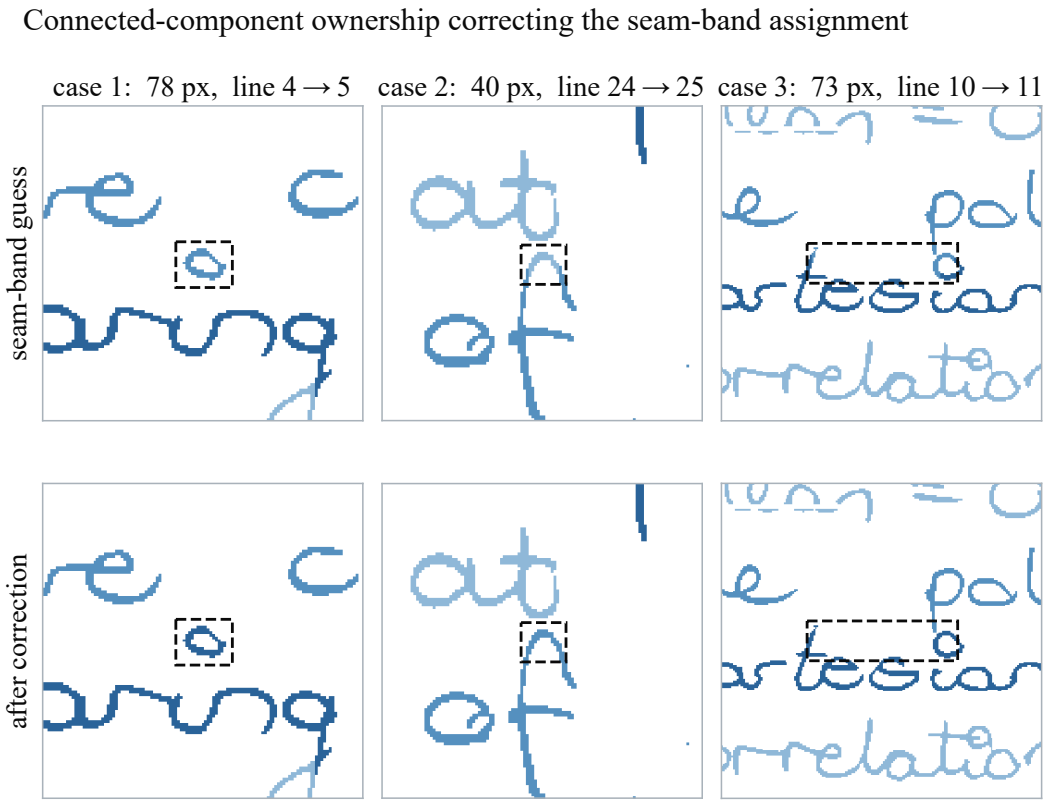


Figure 18. Ink ownership before and after correction.

3.4.5 Preparing the classifier crops

Crops are constructed from the original resolution picture after ownership is settled. A soft ink mask stays away from jagged edges, while a separate nearest-neighbour ownership exclusion keeps neighbouring ink out. Contrast stretching and a darkness exponent of 0.55 strengthen faint alows strokes to be kept.

A guard in place to ensure better crops reject a line only when its dark-pixel fraction (the number of ink or shadow pixels over the number of pixels in the crop) exceeds both 15% and is 3 times the amount of the page median. If every crop would be rejected, the input list is kept. The guard therefore is able to reduce solid-shadow crops without actually checking that every remaining crop is stil right.

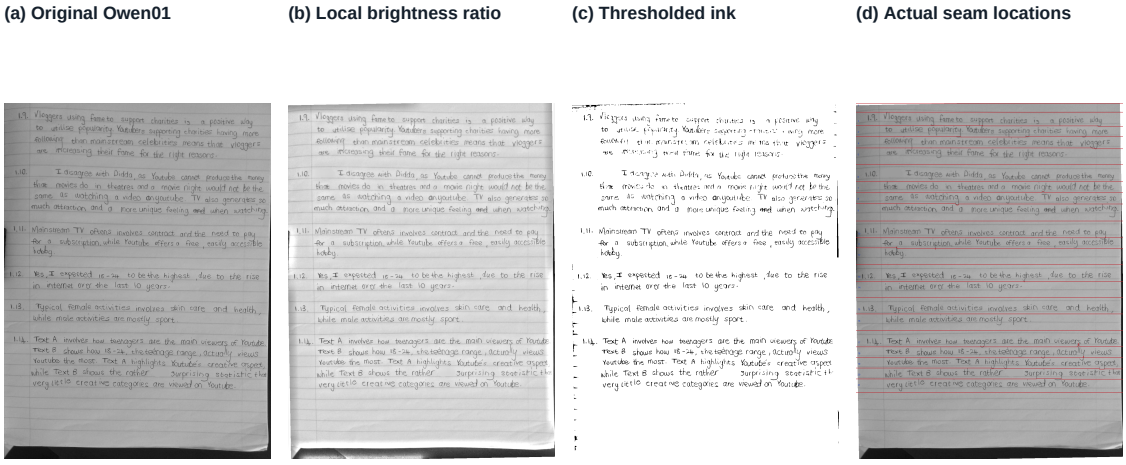
To prepare the cropped image for the classifier, it must be fitted to the desired input size. The classifier canvas is 64×640 pixels. A common scale preserves the proportions of a crop of height h and width w, to ensure an image is not stretched, and that its aspect ratio is kept.

$$a = \min\left(\frac{64}{h}, \frac{640}{w}\right), \qquad (h', w') = (ah, aw). \tag{18}$$

For the archived 156×2170 dataset line, $a \simeq 0.295$ gives a 46×640 image. It is left-aligned, vertically centred and padded white. A very long line loses vertical detail when the width limit controls this scale. INCLUDE IMAGE

3.4.6 Complete processing sequence and output

Figure 19 follows the page from lighting correction to ink extraction, line boundaries and final crops. These views show where a stroke was removed or assigned to another line. Algorithm 2 gives the full sequence after the individual operations described above.



Native input: 2448 x 3264; 20 crops; deskew +1.0 deg.
Output line 1

1.9. Vloggers using fame to support charities is a positive way

Output line 2

to utilise popularity. Youtubers supporting charities having more

Figure 19. Page preparation and line extraction.

In the algorithm, G_f, G_h are the full and half-resolution grey images, R is the brightness ratio and M is the ink mask. Pitch P, centres c and seams b define the lines. The output $\mathcal{L}$ contains the ordered crops, classifier views and bounding boxes.

After cropping, the reader compares up to three sampled lines in both directions to distinguish upright from upside-down writing. It chooses the higher mean frame log score, reuses those readings and reverses line order when needed. This heuristic can still fail when the reader is confidently wrong.

3.4.7 Segmentation verification and limitations

The native Owen01 page produces 20 crops for 20 labelled lines. Figure 20 tests a harder sparse page photographed three times. The striped-shadow image gives three crops, the diffuse-shadow image four, and the changed-angle image 28. In the last case, gap scores of 0.379 and 0.400 give only a 1.054 ratio, below the 1.35 turn threshold. Fine deskewing cannot repair the missed quarter turn.

Algorithm 2 Photographed page to ordered line crops

Require: Grey page on its chosen line axis.
Ensure: Ordered full-resolution crops, coordinates and diagnostics.

$(G_f, G_h, M, R, \theta) \leftarrow \text{Deskew}(G)$
$M \leftarrow \text{RemovePrintedStructure}(M, G_h)$
$(M, \mathcal{C}) \leftarrow \text{DropArtifactPieces}(M, R)$
$(P, c) \leftarrow \text{LineCentres}(M)$
$b \leftarrow \text{SeamCuts}(M, c, P)$
$O \leftarrow \text{AssignOwnership}(M, \mathcal{C}, b, c, P)$
$\mathcal{L} \leftarrow \varnothing$; normalise full-resolution brightness
for each line index n **do**
    $C_n \leftarrow \text{BuildLineCrop}(n, O, G_f, P)$
    if C_n has sufficient ink and height **then**
        Append C_n, its fixed-canvas view and bounding box to $\mathcal{L}$
    end if
end for
$\mathcal{L} \leftarrow \text{DropSolidCrops}(\mathcal{L})$; renumber retained crops
return $\mathcal{L}$, seams, ratio image and diagnostics

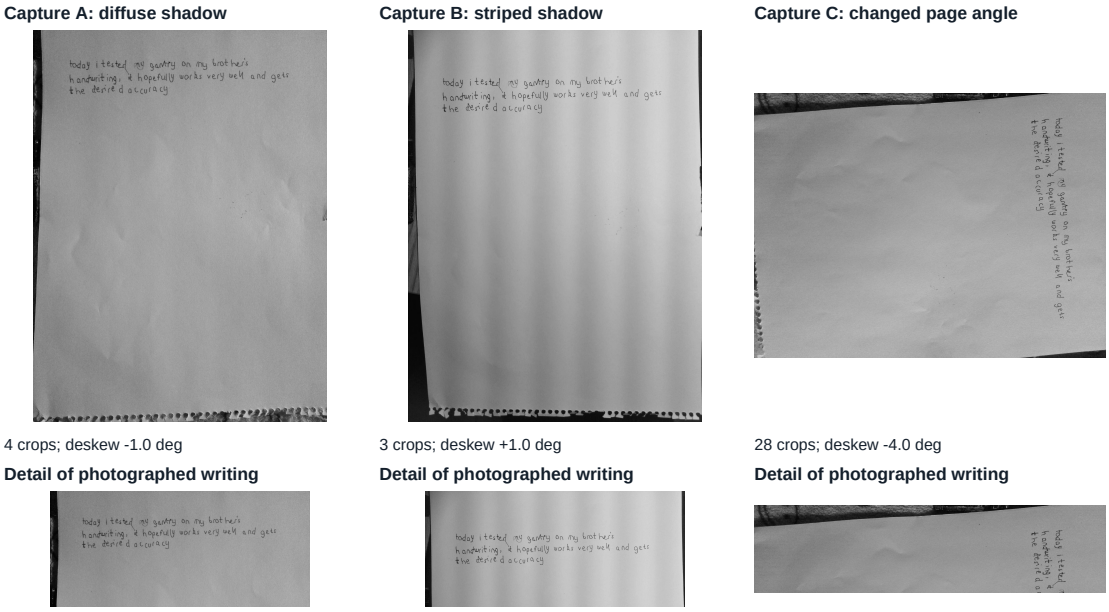


Figure 20. Three captures of the same sparse page.

These captures show failure conditions rather than a measured lighting range. Irregular spacing weakens pitch estimation, touching strokes can defeat ownership, and sparse pages provide little orientation evidence. A fixed mount and restrained page reduce perspective and curvature that a global rotation cannot correct. Appendix B retains the capture records and extended checks.

3.5 Neural network design, training and classification

Two trained networks networks are used to identify the text and writer of each line. Both extract features of the text with the same convolutional architecture, but their trained weights and output heads are separate. A 'head' is the last section of the model which converts the abstract features learnt by the model into the network's final prediction. The text reader using a CNN and BiLSTM sequence model based off of research, implemented from first principles. [12, pp. 4–5]. The writer head combines features across the line, also implemented from first principles. The literacture review explores explanations about the different types of classifiers and the layers within it.

3.5.1 Training data, labels and evaluation separation

IAM lines [4] and photographed samples provide transcripts and writer labels. The evaluated set to train author classification contains six IAM writers (150, 152, 153, 384, 552 and 588) and five personal contributors, to meet R6 of the project proposal which states the system must be able to store minimum 10 author's glyphs and styles at once. The personal pages were written on paper, photographed with the USB camera model setup and paired with line transcripts. The eleven-writer set defines the reported classifier and stored style profiles. The text classifier however was trained off of an extended Teklia IAM dataset with cut crops that are pre-labelled. The author classifier could not use the Teklia IAM dataset due to the fact that it was not labelled with its author.

The writer dataset contains 517 IAM and 669 personal lines. One IAM page per writer is held out, while a seeded 15% split selects which personal lines are used to validate the training. This gives 1031 training and 155 hold-out lines. The final common evaluation contains 156 re-extracted lines, so it is reported separately from training validation. The text reader uses a joint corpus of Teklia pre-cut IAM and personal lines.

The main difference between the full page IAM dataset used for author classifier training, and the Teklia full line version of the IAM dataset, is that the full page IAM dataset was split by author, therefore ideal for author training, yet a labelled transcript per page was not available. Manual verification had to be done to ensure the labelling and segmentation was done correctly on the full page IAM dataset. The Teklia IAM set was split by line, and had transcriptions of the cropped line, yet did not have the author who wrote it. This was ideal for the text classifier as the transcriptions did not have to be manually verified to ensure accurate text to label data, and the author was not required since it is used for training the text classifier, which is independent of who wrote it.

Blank or unusable lines and printed prompts are excluded before pairing crops with labels. Augmentation stays within the training partition, to reduce overfitting on lines not trained on. Both the author classifier and the text classifier were verified against their own respective hold out pages, since they were not necessarily the same pages or lines, since the text classifier was trained without knowledge of the author who wrote it, while the author classifier had to.

3.5.2 Shared convolutional feature extractor

The input is a batch of grey 64×640 line crops, shaped $(B, 1, 64, 640)$ for batch size B, as described in the previous section, without stretching the image disproporionately. Intensity scaling to $[-1, 1]$ makes dark ink positive and white padding negative. A learned kernel W (the weights) and bias b calculate local responses from input X [11, pp. 6–8]. The following equation describes how the a kernal slides across an input tensor (X) to compute a local feature map Z.

$$z_{o,y,x} = b_o + \sum_c \sum_{i,j} W_{o,c,i,j} X_{c,y+i,x+j}. \tag{19}$$

The sums include the input channels and kernel positions. Input patches are rearranged into a matrix for multiplication by flattened kernels, to not have to use standard Python calculation loops for every pixel operation [39].

Batch normalisation uses a channel mean μ_B and variance σ_B^2 during training, followed by learned coefficient γ and offset β [15, pp. 4–5].

$$\widehat{z} = \frac{z - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}, \qquad y = \gamma \widehat{z} + \beta. \tag{20}$$

The constant ε is used to stabilise division. Inference uses stored running statistics. The activation function, ReLU, applies $\max(0, y)$, used to keep positive responses with a simple nonlinear operation. Max pooling keeps the strongest response from each local window. Table 3 describes the features of the architecture.

Stage	**Operation**	**Output excluding batch**
Input	Resized crops to input size and intensity scaling	$1 \times 64 \times 640$
1	Two 3×3 convolutions, each with batch normalisation and ReLU, and 2×2 max pool	$32 \times 32 \times 320$
2	Four convolution/normalisation/ReLU blocks, and 2×2 max pool	$64 \times 16 \times 160$
3	Six convolution/normalisation/ReLU blocks	$128 \times 16 \times 160$
Height reduction	Maximum over the remaining height	$128 \times 1 \times 160$

Table 3. Convolutional stages and feature dimensions.

Two spatial poolings make the width smaller to 160 columns. The final max over height leaves 128 features per column. Figure 21 shows the input Owen line and three strong feature channels before the first spatial pool. Their scales are independent, so darkness cannot be compared as an absolute value between channels. The complete layer sequence is retained in Figure 42 in the appendix.

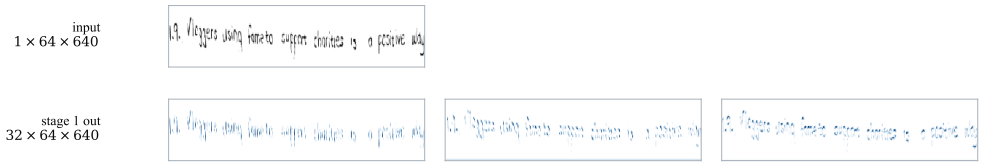


Figure 21. Input line and first convolutional features.

3.5.3 Text recognition using sequence modelling and CTC

Figure 22 shows the architecture of the text classifier. It consists of two bidirectional LSTM layers, each with 256 hidden units per direction, process the 160 feature columns. The first receives 128 features per frame. The second receives 512 after both directions are concatenated. A linear layer calcultions and determines scores for 74 characters and blank. The writer classifier does not use recurrent layers (the bidirectional LSTM layers are recurrent).

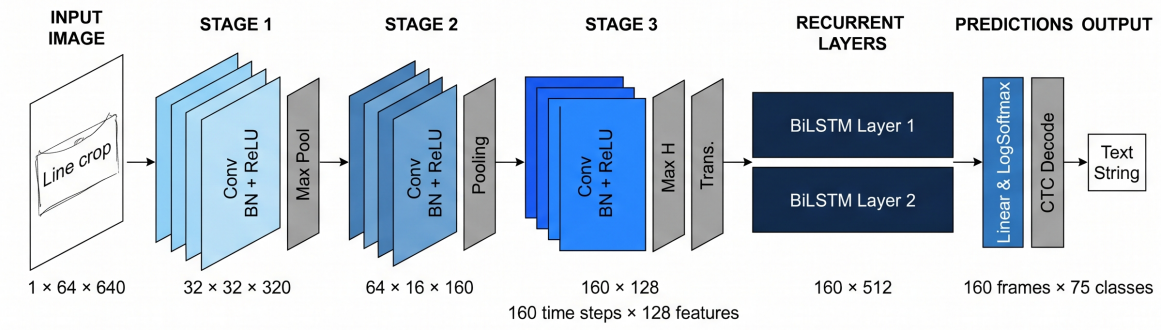


Figure 22. Text-reader architecture.

The LSTM cell state c_t is used to carry and transfer memory and weights between frames. Input, forget and output gates are i_t, f_t, o_t, with desired update variable g_t and hidden output h_t [3, p. 5].

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t, \qquad h_t = o_t \odot \tanh(c_t). \tag{21}$$

The gates depend on a vector of the current feature, and the previous hidden state. Sigmoid $\sigma(a) = 1/(1+e^{-a})$ is used to limit the gates between zero and one, while tanh bounds the desired update change. The reverse direction (bidirectional LSTM) supplies context from the other end of the line, to allow the entire line to contribute to the context while classifying. This is particularly useful for cursive words, where all characters in a word are connected and influence the appearance of the word.

CTC (Connectionist Temporal Classification) is used to train from a line transcript without having the actual labelled character positions [14, pp. 2–3]. It gets the total of the frame paths π whose collapse B produces a sentence of text ℓ, using the probabilities p_t over 160 frames.

$$L_{\text{CTC}} = -\log \sum_{\pi : B(\pi) = \ell} \prod_{t=1}^{T} p_t(\pi_t). \tag{22}$$

The collapse of the scattered text merges consecutive repeats of characters before removing blanks. Thus a,a,BLANK,b,b becomes “ab”, while a,BLANK,a gives “aa”. The dynamic collapse and merging system uses blank-interleaved target states and log probabilities. For L labels and r adjacent repeated pairs, at least $L + r$ frames are required to compute.

A method called ’greedy inference’ chooses the most likely class at each frame and then performs the collapse on the text. No language model is ever used to help it. Figure 23 compares greedy decoding with alignment constrained to the known Owen transcript. This line is a worked example containing adjacent gs, which require separated emission runs. The greedy result matches its transcript and blank carries about 46% of mean frame probability.

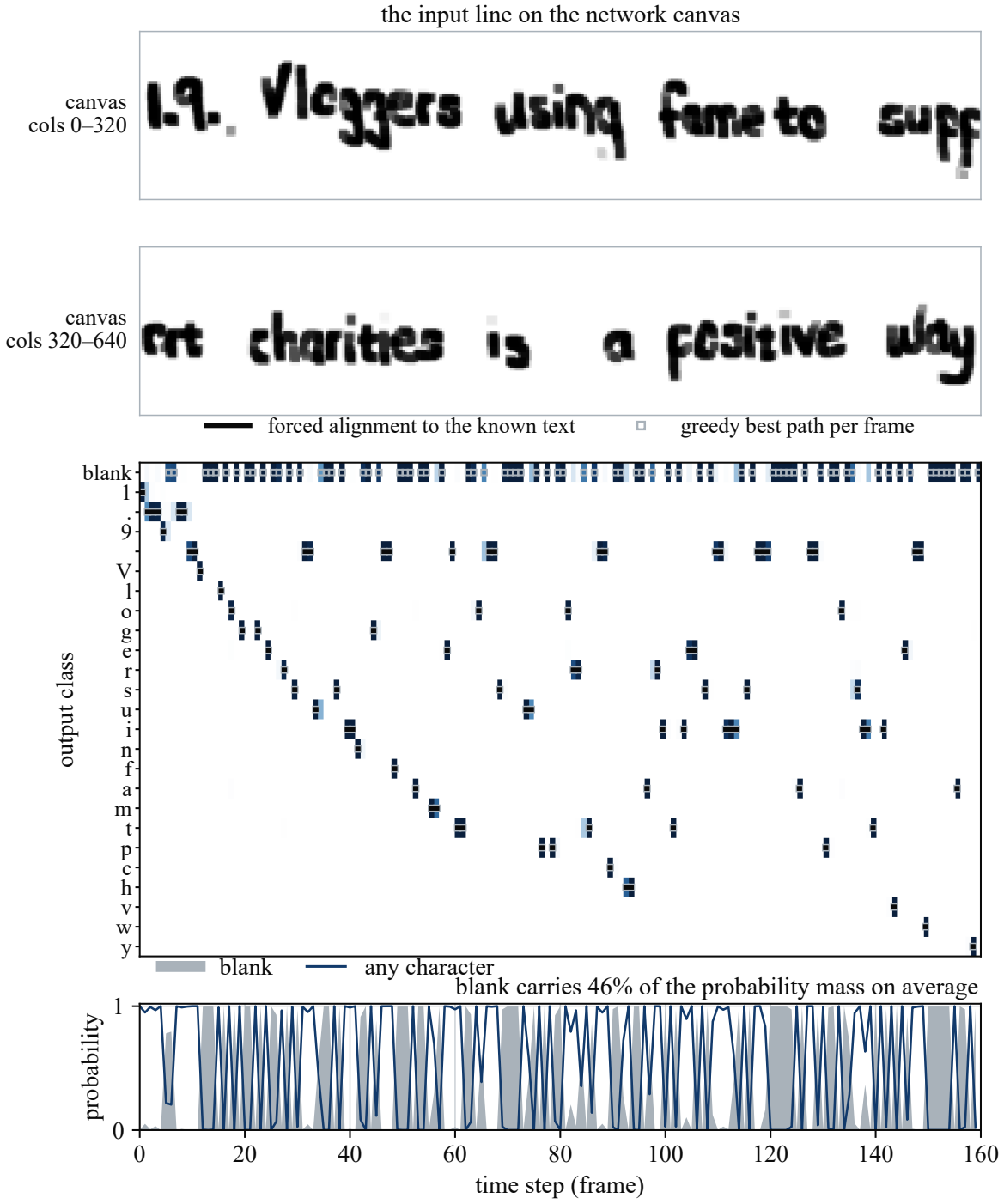


Figure 23. CTC decoding and forced alignment.

The numerical backward pass makes use of a target state occupancy $q_{t,k}$ and total path likelihood Z and logits $z_{t,k}$ (which represent the score of the final character guesses). Forward score α includes the current emission (the current network's character guess score) and moves forward

though the sequence, while backward score β includes only later emissions (future time steps and the prediction at that point) and moves backwards through the sequence [14, pp. 4–6].

$$q_{t,k} = \sum_{s=k} \exp(\alpha_{t,s} + \beta_{t,s} - \log Z), \qquad \frac{\partial \mathcal{L}}{\partial z_{t,k}} = p_t(k) - q_{t,k}. \tag{23}$$

The batch average is then applied without target-length division in the trainer, which means that the normalization over target length is skipped and every sequence has equal weight in the calculation regardless of how many characters there are.

3.5.4 Writer identification and stroke normalisation

The writer head averages the 160 feature columns into 128 values and does not use a recurrent layer, because the writers depend directly on the extracted features, and not time units or extended like context. A 96-unit hidden layer, ReLU and a K-class output identify the writer. Dropout probability of 0.2 is only used during the training of the network [16, pp. 1931, 1934].

The first classifier used the ink styles and thickness from the page scans directly to classify authors. Ink width is a useful writer identifier [20, p. 162], but the gantry uses its own pen, and keeps its ink size consistent, therefore this kind of system is inadequate to be tested with reproduced text. The later classifier therefore uses normalised strokes (with the ink levels standardized for all authors) so that original and reproduced writing are compared more through their geometry. Binarisation, Zhang-Suen thinning and square dilation form are used to form that input, as the following equations describe respectively: [26, p. 237], [37, p. 533].

$$I_s = 255\left[1 - \delta_k\big(\text{thin}(M)\big)\right], \quad w = \text{clip}\big(\text{round}(\rho h_x), 2, 9\big), \quad k = 2\lfloor w/2 \rfloor + 1. \tag{24}$$

where h_x is estimated x-height, ρ is the width fraction and k is the dilation-kernel size. The selected fraction is 0.17, which was obtained from a sweep through various different ρ values to identify the value that produced the best writer identification accuract per page, using the same eleven writers and held-out partition, with a 50-epoch budget for each width. Important caveat here compared to final results, the the results here only did a full page identification test, not a per line test as will be done in the final qualification test, hence the reason the numbers may appear inflated. The important distinquishing factor is the comparison between the four and which is the most accurate. Table 4 gives the best saved epoch and validation score.

Width fraction ρ	Best saved epoch	Held-out writer accuracy per line
0.06	38	98.06%
0.09	43	97.42%
0.11	39	98.06%
0.17	43	99.35%

Table 4. Stroke-width selection using the same eleven-writer population and held-out partition.

The best observed width was 0.17. The final author classifier trained from first principles used for results was trained with the 0.17 stroke normalization width value. The sweep is simply considered an experiment to find the width coefficient that performed the best, and is not part of the final qualification tests.

Figure 24 shows author O and A to illustrate different letter shapes under the same preparation. The original ink is reduced to a centreline before a controlled width is restored. Thresholding or thinning can still damage faint and touching strokes.

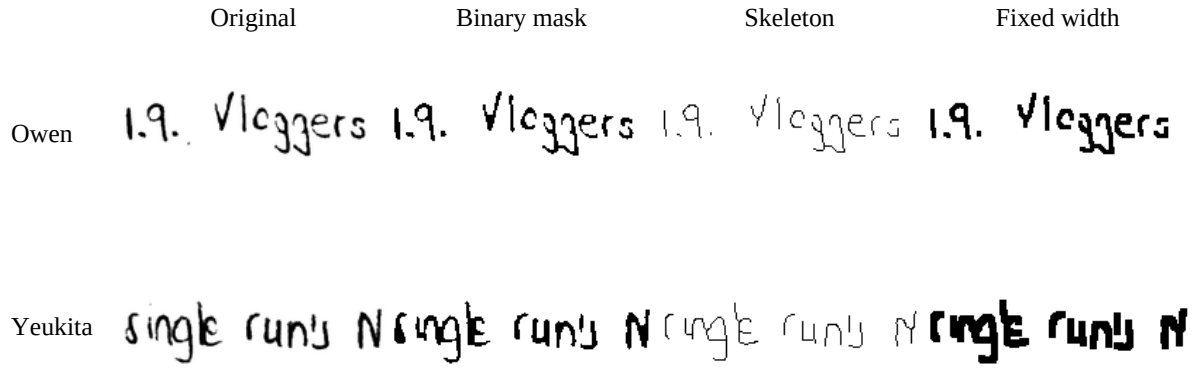


Figure 24. Stroke normalisation on two writers.

3.5.5 Training procedure and model selection

The backbone and task head are updated together. Training uses convolution weights that are scaled with variance, and initial normalisation scales of one with zero offsets. Augmentation is applied to the training data to create richer model with minimal overfitting, in the form of shear, small rotations, perspective changes and elastic distortion changes to the training images while preserving labels. Validation uses a fixed preparation path without augmentation or dropout.

The text classifier minimises CTC loss, while the author classifier minimises cross-entropy for the true writer y, with a logit (score) derivative $p_k - \mathbf{1}[k = y]$ [3, p. 6]. Table 5 shows the default rates, optimisers and other methods and coefficients used.

Setting	Text classifier	Author classifier
Objective	CTC loss	Cross-entropy
Optimiser	RMSprop	AdamW
Initial learning rate	10^{-3} default	10^{-3} default
Weight decay	10^{-5}	10^{-4}
Learning rate changer	Plateau reduction, factor 0.5, patience 5, minimum 10^{-5}	Cosine annealing over the configured run
Gradient norm clipping	Not applied	Maximum norm 5

Table 5. Training settings implemented in the first-principles trainer. The table describes implemented defaults rather than a recovered record of every completed training run.

The text classifier training supports controlled personal-sample representation, with default fraction 0.20, to ensure there is not biasing between the IAM dataset or personal dataset. Text classifier checkpoints are selected and saved by validation loss and author classifier checkpoints

by validation accuracy. Checkpoints were vital to the training, due to the long time it took to train first principles classifiers without libraries or GPU usage capabilities, and a SBC with a mediocre CPU. In case of crashes or power outages, checkpoints were always saved as safety feature, so the run could be continue from the checkpoint. Forward caches kept the quantities needed for differentiation. Algorithm 5 in Appendix A.3 gives the update procedure. The supplied first-principles training records are retained in Appendix C.9.

3.5.6 Verifcation through testing with off-the-shelf methods

The first principles route implements forward propagation, backward propagation, losses and optimiser updates using NumPy array mathematics. This was verified through a comparison against a PyTorch-implementation of the model. Both used the same image conditioning, data, segmentation, architecture, gradients etc, so they were expected to perform relatively similarly

Table 6 compares trained models on common original validation material. The text classification scores differ by 0.33 percentage points in favour of the text classifier, and the author classifier differs by 0.57 points in favour of the PyTorch implementation. The PyTorch implementation was expected to perform better than the first principles one, due to the fact that the PyTorch method is optimized, and runs of 64 bit precision floats, while the first principle ones used 32 bit floats to save computation time because they already took many hours to run. A higher precision on float calculations creates more precise error calculations in training. The fact that both methods proved similar results proves the implementation of the model and layer calculations etc. was correct. The difference could be attributed to random augmentation differences between the runs.

Common validation material	**Library route / %**	**First principles / %**
Reader, public IAM lines	94.12	94.39
Reader, photographed lines	93.33	94.42
Reader, pooled character accuracy	94.06	94.39
Writer, 156 original lines	98.08	97.51

Table 6. Training-route comparison on common data.

The error metrics between the PyTorch and first principles versions were recorded aswell, to further prove the consistency between the two implementations to prove the first principles implementation was correct. A numerical comparison was done using identical inputs, parameters and precision, and float64 checks were done between the maximum logit difference (score metrics), the absolute loss difference, and the worst ranked gradient relative difference, show in Table 7. All errors and differences between the two implementations were incredibly small, all less than 10^{-13} error margin. Over time these miniscule errors could add up, causing differences in the classifications of each implementation.

Recorded comparison metric	Reader	Writer classifier
Maximum logit difference / maximum magnitude	2.12×10^{-14}	3.04×10^{-15}
Absolute loss difference	1.14×10^{-13}	1.78×10^{-15}
Worst ranked gradient relative L_2 difference	2.80×10^{-13}	6.01×10^{-14}

Table 7. Archived float64 operation comparison.

3.6 Style extraction and handwriting reproduction

The reproduction stage extracts single character, or glyphs from labelled handwriting, stores each writer's shape and spacing measurements, and then synthesizes them into new text.

3.6.1 Choice of reproduction representation

A glyph profile reuses the few labelled pages available for each author in the personal or IAM datasets. Haines et al. used approximately four A4 pages in their glyph-based experiments [23, p. 4]. The new sentences can be synthesized and created without the need for a heavy weight classifier trained on large amounts of data for reproduction, and is then mapped to the gantry.

The photographs contain no measured pen order, speed or pressure, unlike the trajectories required by recurrent synthesis [3, p. 18], because the system used in this project was an offline system, performing computer vision on static images. Text synthesis that is based on image generatoration can work real time with very few existing style references as data, but the issue is an entire model still needs to be trained on much data, and requires a significant amount of computation to train the model well, for example on IAM-based synthesized image generator used 339 writers and four RTX 3090 GPUs [22, p. 9]. The chosen glyph approach fits the available data and compute, but has an expected weakness of performing glyph extraction on heavily connected authors. Various methods were implemented alongside the glyph extraction and synthesis, to improve the quality of reproduced writing.

3.6.2 Character alignment and glyph extraction

A source line has a known transcript but no labels on where exactly each character is positioned. The existing trained text classifier supplies frame probabilities, and uses CTC forced alignment to select the highest-scoring path consistent with that ideal label or transcript. It uses the CTC transition rules, replacing path summation with maximisation [14, pp. 4–5].

$$V(t,s) = \log p_t(\ell'_s) + \max_{r \in P(s)} V(t-1,r). \tag{25}$$

where ℓ' is the text label interleaved with blanks and $P(s)$ is the allowed predecessor set for state s. Backtracking gives each character's frame interval. A frame edge t maps about to the source position $(640t/T)/a$, using the inverse canvas scale a. Multiple acceptable boundaries can still give the correct transcript.

The horizontal ink profile guesses the main box of the text. Its first and last rows above 45% of the maximum calculate the band top and baseline b, with difference h_x giving estimated

x-height, and bands below three pixels are not used. Initial character cuts are refined towards thin connectors or blank columns using the project-specific cost

$$C_x = \frac{p_x}{q_{0.60}(p_j : p_j > 0)} + 1.15\,\mathrm{clip}\left(\frac{b - y_{\mathrm{top}}(x)}{h_x}, 0, 3\right). \tag{26}$$

where p_x is smoothened column ink, the bottom of the equation is its positive-count 60th percentile, and $y_{\mathrm{top}}(x)$ is the highest ink row. The height term prefers lower connectors that are closer to the baseline. A true blank column has no cost. Cuts remain ordered, with minimum cell width being $\max(1, 0.08h_x)$, so neighbouring boundaries can't collapse onto the same gap.

Figure 25 follows the archived first Owen line because its crop and transcript can be traced through recognition and extraction. It shows the initial frame locations, refined cuts and resulting glyphs. The worked path and cut-cost substitutions are in Appendix A.1.

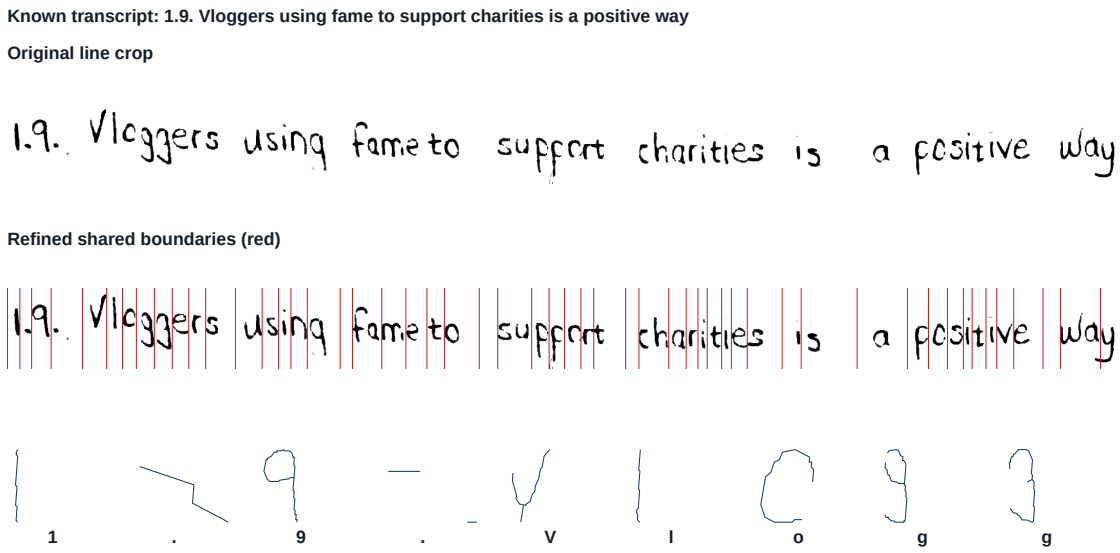


Figure 25. Character alignment and refined glyph cuts.

3.6.3 Skeleton tracing and pen path construction

The process of thinning is used to convert an ink area into a one-pixel centreline, effectively reducing the thickness of any ink feature to one pixel width [26, p. 237]. The complete line is thinned first before its cut into cells, to avoid an artificial thinning boundary generated at every letter. A graph tracer is used to divide paths at intersections, and eliminates short text artifacts and rejoins compatible ends that are below a 75° continuation angle. The Douglas-Peucker simplification removes unnecessary vertices within a bounded deviation [25]. It works by drawing anchor lines betweem two ends of a line of text to create, and then finds the point of the text further away from the anchor. If the point is further than a limit value, the point is kept and made to be a new vertex. The Douglas-Peucker algorithm is mentioned in the literature review.

Dots that are on their own are recovered as short paths, and eligible detached crossbars (like for a 't' shape) are assigned to their letter rather than split across cells or counted as cursive joins

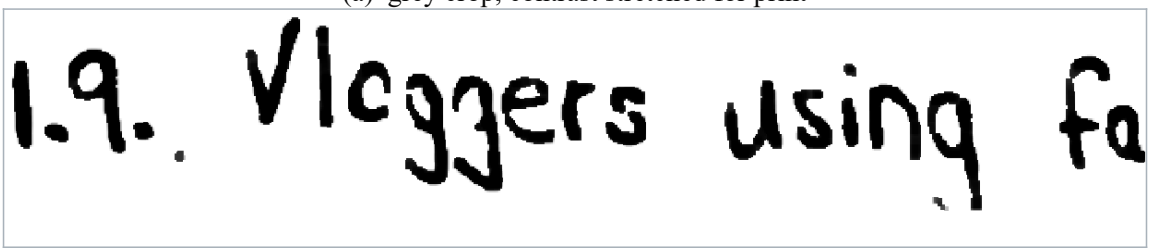
between two letters. The resulting points are stored relative to left cut x_L, baseline b, x-height h_x and slant ϕ, using a shear transform [9].

$$v = \frac{b - y}{h_x}, \qquad u_0 = \frac{x - x_L}{h_x} - v \tan \phi, \qquad u = u_0 - \delta. \tag{27}$$

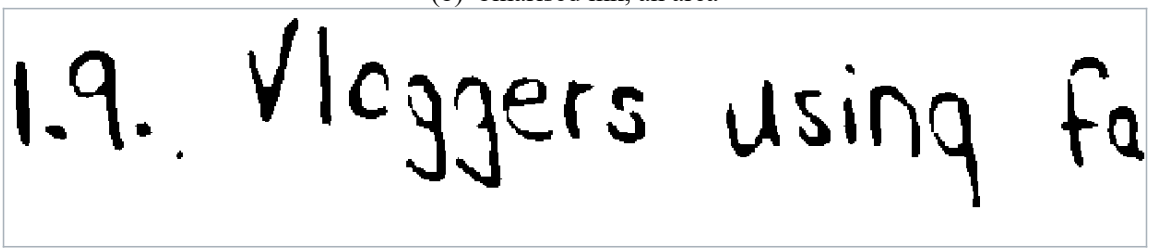
The vertical coordinate increases upwards. The trim shift δ is removed from the stored horizontal path and kept as the lead-in spacing. Slant is removed for storage as a 'author feature' and is added again at placement. This separates the letter shape from its source size and cell spacing.

Figure 26 shows the Owen fragment before and after thinning and tracing. Across its full line, simplification reduces 3738 vertices to 974 at 0.6 source-pixel tolerance. The graph gives geometric paths, while the writer's original stroke order remains unknown. The coordinate substitution is retained in Appendix A.1.

(a) grey crop, contrast stretched for print



(b) binarised ink, an area



(c) Zhang–Suen centreline, one pixel wide

(d) 85 traced paths over the whole line, simplified from 3738 to 974 vertices at ε=0.6 px
a dot marks each path start, and a change of shade marks a new path

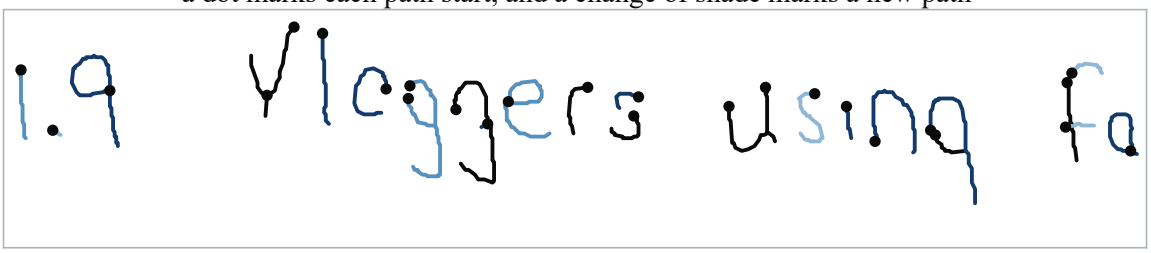


Figure 26. Ink, skeleton and simplified pen paths.

3.6.4 Glyph selection and library quality

Alignment can place a cut inside a letter even when the text is correct, making the letter unusable, and is then an unreliable glyph. The profile therefore filters out and removes unreliable lines and glyphs using width, height, stroke count, path length and allocated character advance. This is why it is beneficial to have more lines and pages of training data to extract glyphs from, as it presents a higher chance of producing a decent looking glyph. Ordinary widths are limited to 2.1 x-heights and wide letters to 2.9. Distributions that are specific to certain characters provide further checks without forcing every letter to the same size.

A normalised 12×12 occupancy descriptor compares shape. A pooled character reference identifies cuts that are done incorrectly, while pairwise distances within a writer's pool identify a representative real variant, called the medoid. A mediod is effectively the geometrical average of multiple extracted glyphs, to produce the 'average' glyph for a specific character. A separate denoised prototype combines compatible strokes. The ordinary pool holds up to twelve variants per character. Algorithm 3 shows where these checks are done when creating a new profile. The descriptor distance is $d(g,c) = 1 - v_g^T r_c$ for unit-normalised glyph descriptor v_g and class reference r_c. The medoid is used to minimise the differences betweem the glyph distances per class.

Algorithm 3 Labelled handwriting to a persistent writer profile

Require: Labelled original lines L, trained reader and writer identifier.

$G = null$; $S = null$
for each $(G, l) \in L$ **do**
    A = ForcedAlign(G, l); estimate baseline, x-height and slant
    if alignment or body band is unusable **then continue**
    Refine shared increasing cuts, thin full line, and trace skeleton paths
    Split paths at cuts, assign crossbars, recover untraced dots
    Normalise glyph coordinates, and record advance, lead and boundary crossings
    Append glyphs to G and line measurements to S
end for
Cache raw extraction; reject low-confidence lines relative to this writer
for each character's glyph pool G_c **do**
    Apply size, height-class, path-length and cell-consistency gates
    Refine size range, and apply cross-writer prior and within-writer consensus
    Rank and smooth retained variants, and construct medoid and eligible prototype
end for
return persistent profile containing G and robust summaries of S

Figure 27 compares the nine stored `a` variants for Owen, Dylan and Yeukita. The cloud shows their variation, the medoid is an actual example, and the prototype is a combined shape. The distance bars explain which variant is most typical within each writer. The exact descriptor and medoid calculations are retained in Appendix A.1.

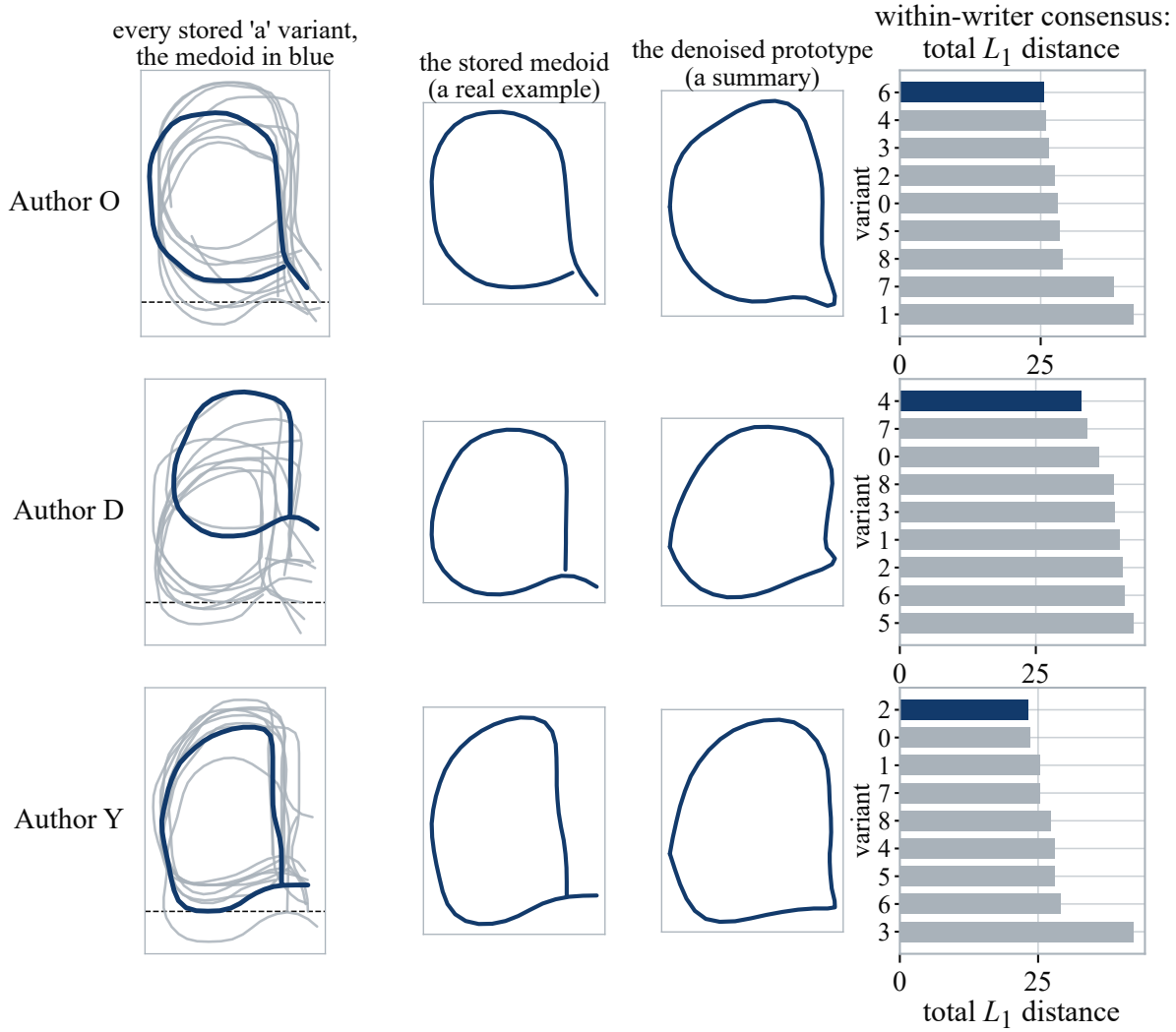


Figure 27. Glyph variants and representatives for three writers.

Missing characters can use case-related shapes, similar forms or fallback glyphs, known as 'anchor glyphs'. When a character is missing, or is too incomplete to be used, the anchor glyph will be pulled instead (different cases exact for separate and cursive styles). They allow text generation to continue, but a fallback does not establish that the character was captured in that writer's style, but since certain author's features are still extracted and applied afterwards, the anchor glyph still gets certain author's style traits applied to it.

3.6.5 Writer style measurement and storage

A profile of each author's features is created per author, which stores slant, x-height, ascender and descender extent, character advance, word gap and percentage connectivity coefficient. Slant is estimated by finding the shear that concentrates ink into columns, and dimensions of the text are normalised by x-height to reduce the dependency on the resolution of the image taken.

Connectedness is the percentage amount of available adjacent non-space boundaries crossed by a traced centreline, which is a method motivated by contour-based writer features [18, p. 701].

$$c = \frac{N_{\text{cross}}}{\max(1, N_{\text{adj}})}. \tag{28}$$

3.6.6 Text layout and connected handwriting synthesis

The synthesiser selects glyph variants, applies the stored style and places words within the available line width. Character advance, which the effective width of characters based on author style, and word gap come from the profile. Words will move onto the next line when the remaining width is insufficient.

For stored point (u, v) and lead l, physical x-height h_x^{mm} converts the glyph to millimetres. Slant shear s and baseline drift d modify the generated text about placement origin (p_x, p_y).

$$x = p_x + h_x^{\text{mm}}(l + u + sv), \qquad y = p_y + h_x^{\text{mm}}(v + d). \tag{29}$$

Small linear shifts, rotations and scale changes provide some variation to the text that is created. Drift changes gradually with persistence 0.85 and is bounded to ±0.25 x-heights, and 0.05 x-height clearance limits collisions. Variants that are better are sampled more often, with two variants with similar entry heights preferred for joining, since they would look more natural being next to one another.

The value of the automatic anchoring strength λ depends on median stored prior distance d.

$$\lambda = \text{clip}\left(\frac{d - 0.24}{0.32 - 0.24}, 0, 1\right). \tag{30}$$

Lower-distance attempts tend to keep more personal variants. Larger distances increase the dependence on an acceptable representative of the available glyphs, and then a generic form from the anchor set if needed.

Joining uses the writer's measured connection fraction, but is suppressed below 0.45, due to the fact that connections could be detected incorrectly during forced alignment and boundary cutting, adding too many unnecessary connections for simple writing that is not cursive. Authors about the threshold tend to be somewhat heavy and connected cursive writers, so more connections is preferred. A join also requires a compatible source and gap no greater than 0.80 x-heights, to prevent long, unnatural looking line connections between distance characters. A shallow connector joins the body strokes without attaching to dots or crossbars. A rendered probe applies bounded slant and extent corrections before synthesis. The placement and anchoring examples are retained in Appendix A.1.

3.6.7 Candidate selection for legibility and writer identity

The different selections of glyph choices can change the readability and writer identity. Each candidate is rendered at constant ink width and scored by the reader and stroke-normalised author classifier. Character score A and requested-writer probability P form joint score J.

$$A = \max\left(0, 1 - \frac{d_{\text{edit}}(\widehat{t}, t)}{|t|}\right), \qquad J = \begin{cases} 2AP/(A+P), & A + P > 0, \\ 0, & \text{otherwise.} \end{cases} \tag{31}$$

The harmonic mean punishes a candidate weak in either score and will give priority to another attempt. The recorded Author O candidate has $A = 18/19 = 0.95$ and $P = 0.58$, giving $J = 0.72$. Algorithm 4 gives the bounded search and repair sequence. A repair is retained only when the complete candidate score improves.

Algorithm 4 Joint candidate search and bounded legibility repair

Require: Text, writer profile, reader, shape classifier, seed s and trial budget K.

$J^* = -1$; $R = 0$; determine anchoring strength
for $k = 0$ **to** $\max(1, K) - 1$ **do**
  $T = \text{SynthesizeText}(\text{text}, \text{profile}, s + 977k)$
  $I = \text{RenderUniformInk}(T)$; $(A, P, J) = \text{Score}(I)$
  if $J > J^*$ **then** $(T^*, I^*, J^*) = (T, I, J)$
  if $J^* \geq 0.99$ **then break**
end for
for at most three configured repair rounds **do**
  if $J^* \geq 0.995$ **then break**
  W = up to two weak aligned positions not already in R
  if $W = 0$ **then break**
  $R' = R \cup W$; regenerate with stronger anchors at R' and seed s
  Render and rescore the complete trial trajectory
  if $J > J^*$ **then** commit R' and the new best trajectory
end for
return T^* with selection and accepted-repair metadata

Figure 28 compares the three recorded trials of “Writing with style.” using the same profile and settings. Candidate 1 is selected because its writer score is stronger while retaining the same character score as candidate 3. The full values are in Appendix A.1.

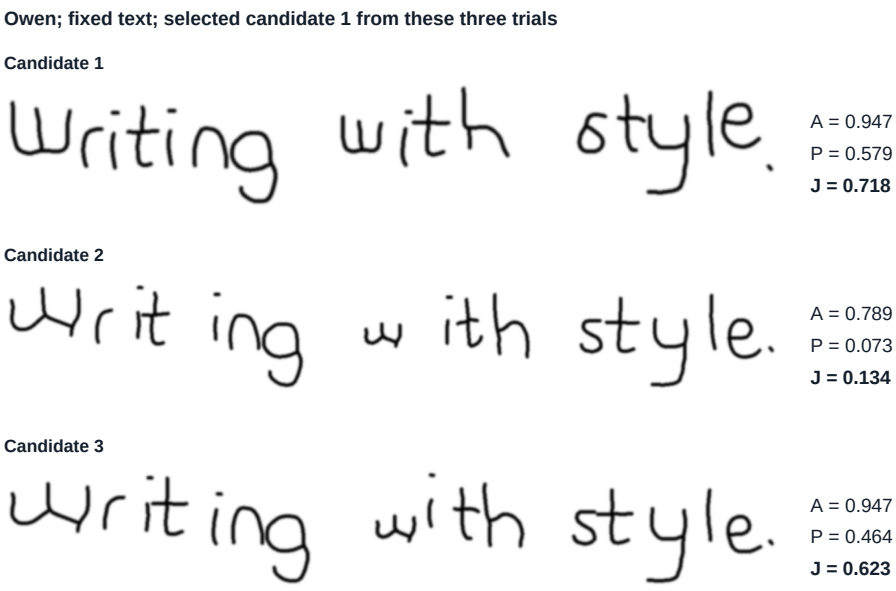


Figure 28. Candidate paths and their selection scores.

The scoring models only guide the choice of glyphs used, so their confidence does not independently prove physical style similarity. More trials also add inference time and cannot recover a missing source letterform, so there is a hard limit to the effect of running more trials, the system is always limited by the quality of the glyph.

3.6.8 Machine coordinate mapping and G-code generation

The selected trajectory is fitted inside the calibrated workspace. The system contains a ruled placement option, where the user can select the line to begin the placement of the trajectory, while free placement retains its synthesised pitch. Character size and line spacing remain separate so that a tall descender need not change both.

Trajectory bounds (x_0, y_0, x_1, y_1), origin (o_x, o_y), margins m_x, m_y and scale a affect how the mapping is generated. Offset d_y selects a later starting line and Δ_y adjusts line spacing.

$$x_m = o_x + m_x + a(x - x_0), \qquad y_m = o_y - m_y - d_y - a(y_1 - y) + \Delta_y. \tag{32}$$

Both stored and machine coordinates use upward-increasing y. Clamping is a final travel limit and can distort a letter. Robust extent fitting and margins should keep the generated text inside the bed before execution. Nearby path segments may be simplified and compatible strokes joined, using a default join distance of 0.6 mm.

Figure 29 gives the opening commands of the generated Author O example. It selects millimetres, absolute coordinates, a movement where the pen is lifted, and the first drawing movements. The full point mapping and file interpretation are extended in Appendix A.1.

```
G21
G90
G0 F2400
G0 F2400 X15.000 Y240.557
M3
G4 P0.350
G1 F900
G1 X15.001 Y240.207
G1 X15.002 Y239.857
G1 X15.003 Y239.506
G1 X15.005 Y239.156
G1 X15.006 Y238.806
G1 X15.007 Y238.456
```

Figure 29. Opening commands of the Owen example.

Travel feed is restated after drawing because feed is modal. Waiting for the pen to reach its UP state adds 0.35 s per transition to the switch-confirmed actuation. For drawing and travel distances D_d, D_u, feeds F_d, F_u and N_p alternating pen transitions, the nominal budget is

$$t_{\text{nom}} = \frac{60 D_d}{F_d} + \frac{60 D_u}{F_u} + \frac{N_p (t_{\text{up}} + t_{\text{down}})}{2}. \tag{33}$$

The earlier traced file gives 32.82 s, including 19.60 s dwell. Figure 30 shows a separate four-profile calculation for the same sentence. Waiting for the pen between state transitions makes up about 61 to 65% of the total time. This identifies a motion-side improvement to test, while actual completion also includes synthesis, actuation and pulse-loop overhead.

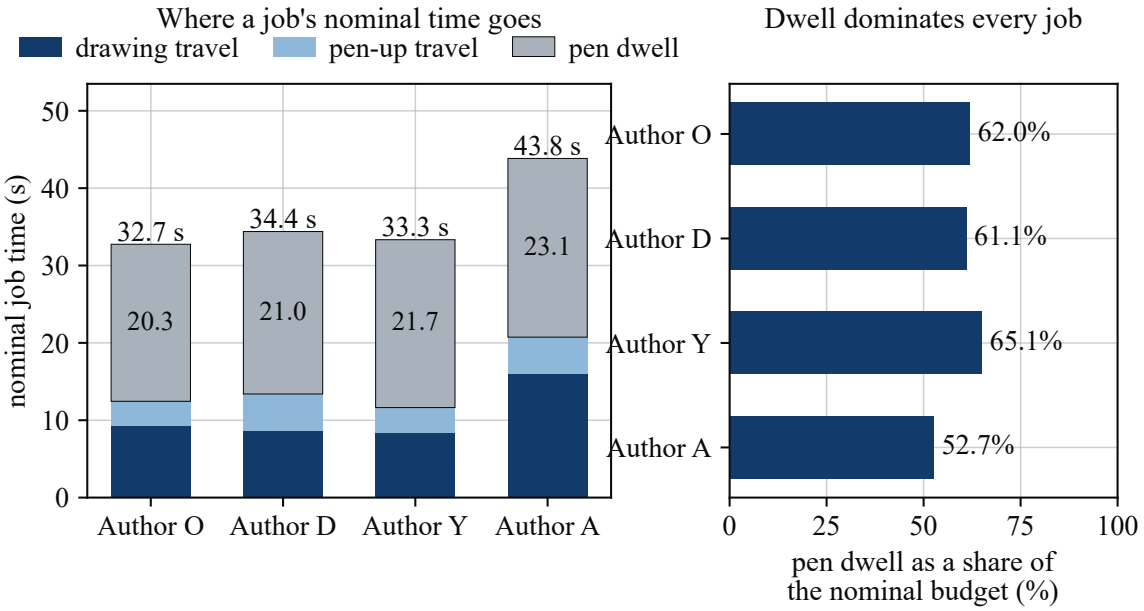


Figure 30. Drawing, travel and pen-dwell budgets.

3.6.9 Reproduction design limitations

Most of the design limitations with the software part of text reproduction come from the engineering decision to use glyph-based text synthesis over a generative model. The reasons why a generative model was not used was discussed at the beginning of the section. The fundamental limitation with glyph extraction is the difficulty faced when attempting to perform forced alignment and single character glyph extraction on a heavily connected or cursive style. Through techniques like the glyph anchor usage when extracted glyphs fail, the connectedness coefficient, and reapplication of extracted author's features, allow some clean cursive variants to perform well. Heavy cursive is still a barrier and if more data and computational power was available, a generative model should be looked into to perform sentence generation.

3.7 System integration and embedded operation

A web server was created to connect page processing, models, profiles and motion control to the browser. Training was done through the ODROID N2+ for the text and author classifiers, but was not part of the website, as there was no use to start a new training run in a user interface if the models and weights were already trained. Adding new authors through a user interface was out of the specifications of this project, and new glyphs extractions, feature extractions and author classifiers are trained through the terminal or by running a script on python, all on the ODROID N2+.

3.7.1 Classification and reproduction workflows

Captured and uploaded pages use the same preparation and recognition path. The image conditioning and segmentation code is run when a new classify request is started, and then the page and its crops are passed into the text and author classifiers alongside its expected transcription and author, and a percentage accuracy metric is displayed for it.

Each generated job stores paths, preview, G-code and metadata. Drawing requires connection and calibration, allowing preview inspection first. Table 18 records the data interfaces.

3.7.2 Embedded execution, interfaces and job control

The pulse loop checks aborts and confirmed end stops. A separate request thread accepts stops during drawing. End-stop faults trigger the configured recovery. Job records distinguish generation, refusal, completion and interruption. Software stops do not independently disconnect motor power.

3.7.3 Integration verification and remaining constraints

Interface checks cover input consistency, profile mapping, G-code bounds and interruptions. Diagnostic retracing bypasses recognition and synthesis. Comparing its preview with the drawing separates extraction faults from execution faults.

Stage timings separate processing, inference, generation and drawing. Reproduction totals include generation. Results evaluates accuracy and embedded runtime.

4. Results

The six qualification tests follow the Project Proposal. Tables and figures give the measurements, while Section 5 discusses their engineering meaning. Each test points to supporting evidence in Appendix C.

4.1 Summary of results achieved

Table 8 summarises the measured outcomes. Geometric agreement relative to natural writing is reported separately from character accuracy.

Requirement	Recorded result
R1. Reproduce text and retain writer style	Reader score on physical output 84.41%. Writer identification 86.41% of lines and 14/14 page pools. Normalised geometric agreement 86.68% on the repeated-word subset.
R2. At least 95% text and writer accuracy	Reader 94.06%. Writer identification 153/156 lines (98.08%).
R3. At most 4 s per reproduced character	All 25 completed ODROID requests pass. Largest recorded value 3.875 s per character.
R4. Classification within $2+0.025x$ s	Recorded totals 46.26 and 55.49 s, against 14.50 and 26.55 s allowed.
R5. At most 0.5 mm dimensional error	All 70 feature lengths from ten samples are within the limit. Largest absolute error 0.49 mm.
R6. At least ten stored styles	Eleven stored profiles, all represented in physical output.

Table 8. Results against the proposal.

4.2 Qualification tests

4.2.1 Test material, configuration and measurement definitions

The evaluation covers six IAM writers and five personal writers, labelled A, B, O, T and Y. Fourteen reproduced pages give 184 transcript-paired lines. Physical motion and request timings are recorded by the ODROID. Saved-image verification uses the retained photographs and original handwriting. The checkpoint identities and measurement setups are listed once in Appendix C.1.

The reader score uses Levenshtein character edits, following the character-error convention [14, p. 2]. Writer identification counts correct line labels, with page pooling reported separately.

Wilson intervals summarise observed line decisions [40, pp. 209–212]. Shared pages and writers limit their interpretation as independent observations.

$$A_c = 100\left(1 - \frac{\sum_i d_{\text{edit}}(\hat{t}_i, t_i)}{\sum_i |t_i|}\right). \tag{34}$$

The direct geometric check compares word skeletons. Precision counts reproduced skeleton points within the distance tolerance of the original, and recall reverses the two sets. Their harmonic mean follows the point-set F-score definition [41].

$$F = \frac{2PR}{P+R}, \qquad Q_k = 100\frac{\overline{F}_{\text{robot},k}}{\overline{F}_{\text{genuine},k}}. \tag{35}$$

Here Q_k is a project-defined comparison with writer k's measured natural variation. Averaging the eleven Q_k values gives equal weight to writers. It is a relative geometric score, rather than a character-accuracy percentage or a correlation coefficient.

Qualification test 1: Reproduction of digital text in a trained writer's style (R1)

Objectives of the test or experiment

Test the reproduced pages with the text classifier and author classifier, then compare their geometry with the original handwriting.

Equipment used

The gantry, ODROID N2+, IMX179 camera, stored profiles, recorded classifier predictions and saved original and reproduced word crops.

Test setup and experimental parameters

The fourteen pages contain a common paragraph, a pangram and source text. The text classifier measures readability against the known transcript. The stroke-normalised author classifier tests whether the reproduced writing retains its requested identity. Geometry uses 83 repeated words from all eleven writers. Each word is scaled uniformly to 48 pixels in total ink height, with at most five pixels of translation and a 4.8-pixel stroke-distance tolerance.

Steps followed in the test or experiment

Draw and photograph the pages, pair their lines with transcripts and retain the reader edits and writer labels. For geometry, compare a genuine word and its robot reproduction against the same references from other pages by that writer. Exclude the genuine probe's own page. Average all reference comparisons, then normalise each writer's robot score by that writer's genuine-to-genuine score.

Results or measurements

Table 9 summarises the three checks. Table 10 gives the text and author classifier results for every writer. Figure 31 shows which writers are confused, with each row normalised to 100%. Appendix C.3 retains the test-block breakdown. Appendix C.10 contains the photographed pages.

Recorded quantity	Result
Physical reader character score, $100(1-853/5471)$	84.41%
Correct physical writer labels	159/184 (86.41%)
Correct pooled page identities	14/14
Writers above 85% physical character score	7/11
Normalised geometric agreement, equal writer mean	86.68%
Repeated-word verification coverage	83 words, 11 writers

Table 9. Physical-output classifier and geometry results.

Writer	Lines	Text classifier / %	Author classifier / %	Word match / %
Y	13	97.01	92.31	74.16
150	13	96.75	53.85	76.40
O	16	94.25	100.00	58.51
B	16	93.11	93.75	68.24
384	16	86.93	87.50	43.18
552	15	86.17	80.00	48.39
152	15	85.74	80.00	49.44
A	27	77.29	100.00	18.09
153	19	75.72	57.89	31.46
588	18	73.19	94.44	22.99
T	16	60.97	100.00	13.41

Table 10. Text and author classifier scores on reproduced pages.

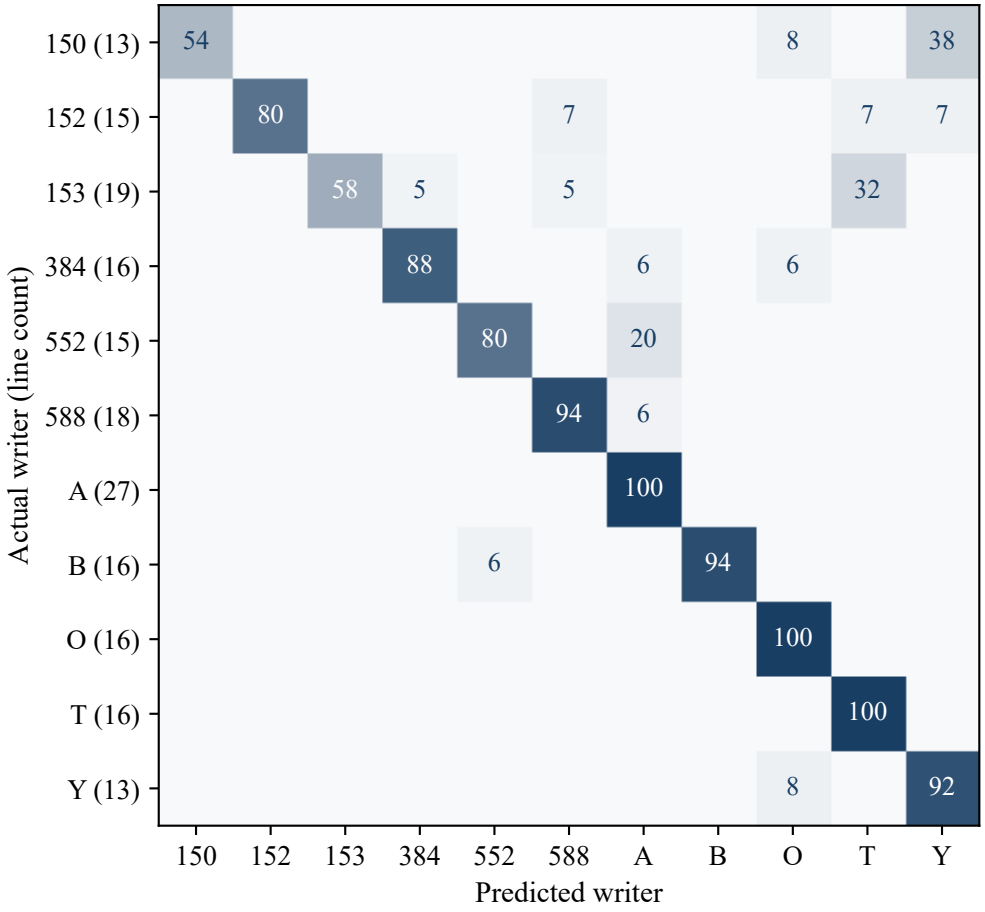


Figure 31. Author predictions on reproduced lines, normalised by actual writer.

Figure 32 gives the geometric result relative to each writer's own variation. Its 100% reference means equal agreement to the measured natural baseline. Figure 33 shows original and robot word crops with their registered skeletons. Appendix C.4 gives the supporting writer averages and further original-to-reproduced examples. The complete crop banks and individual scores remain in the accompanying measurement archive.

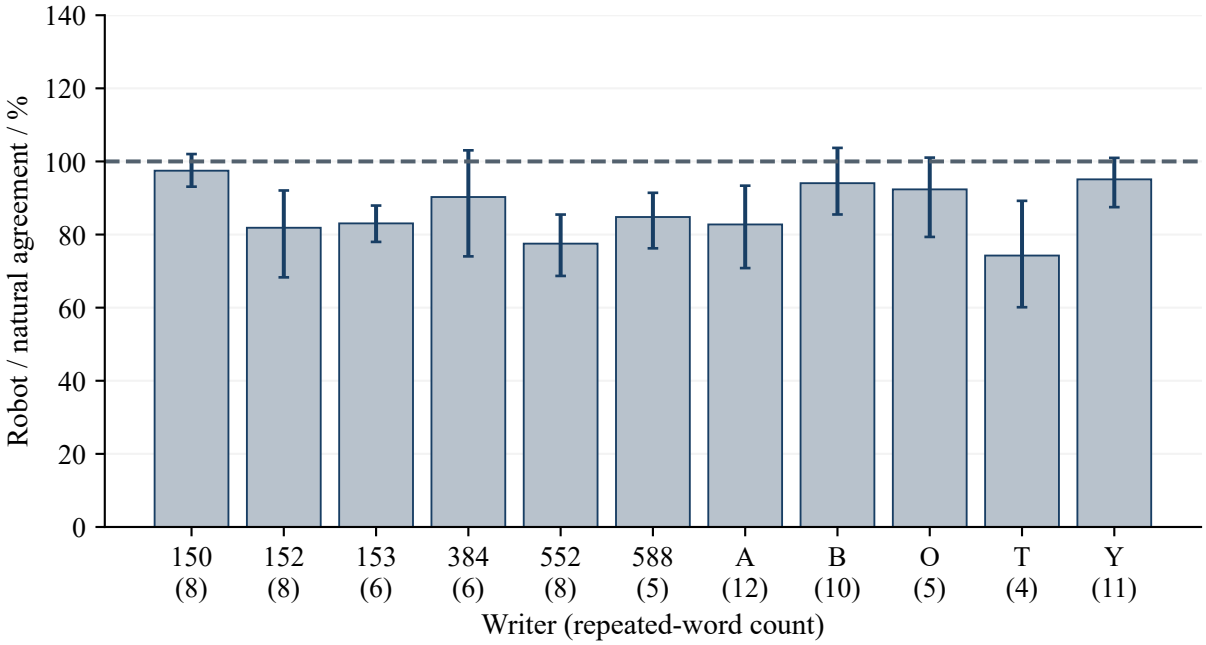


Figure 32. Geometric agreement relative to natural writing. Counts are repeated words.

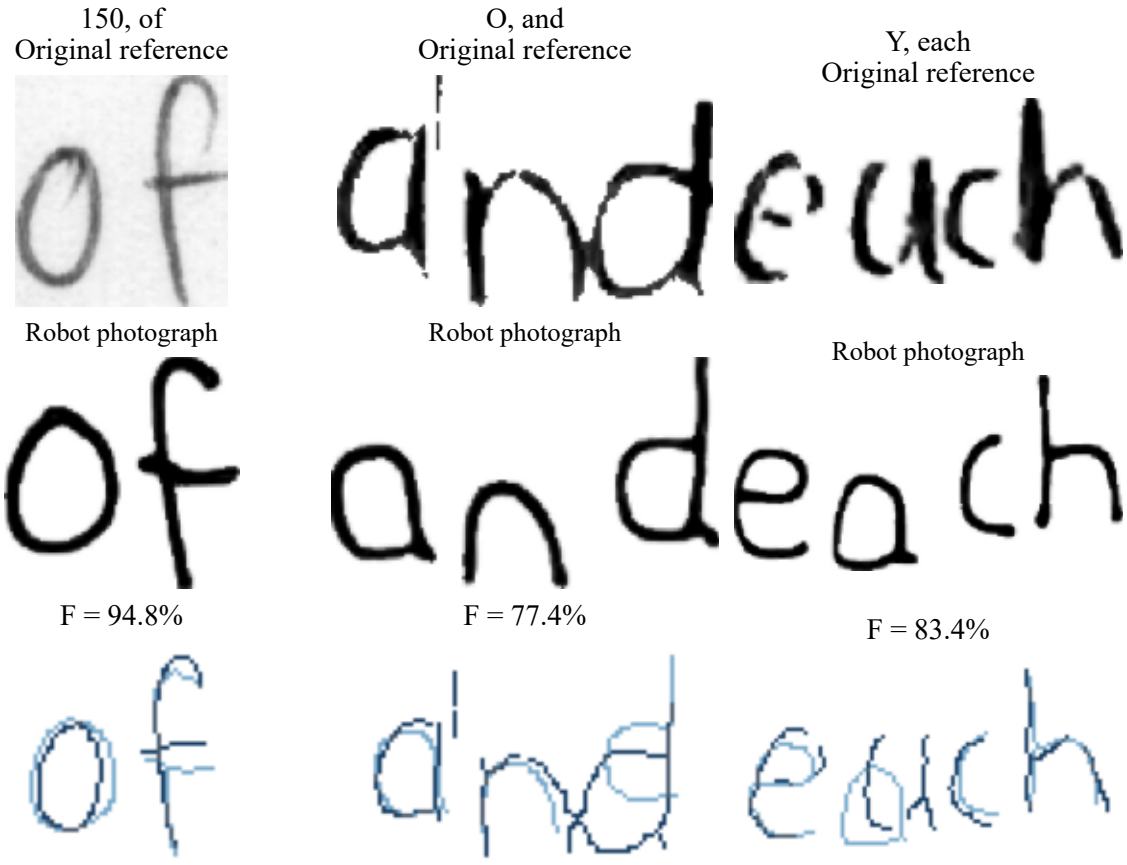


Figure 33. Measured word examples. Dark blue is original, light blue reproduced, grey overlap.

Observations

Character scores range from 60.97% to 97.01%. Geometric agreement relative to natural writing ranges from 74.26% to 97.47% using ordinary reference averages. Twenty-five of 166 eligible source-passage words lack an exact reference. A further 58 of the 141 comparable words lack cross-page repeats for a natural baseline.

Statistical analysis

The 159/184 writer decisions have a descriptive 95% Wilson interval of 80.7% to 90.6%. Figure 32 uses 10 000 paired word-type resamples within the fixed writers and reference banks. The equal-writer range is 83.65% to 89.65%. The existing closest-reference sensitivity check gives 87.89%, compared with the ordinary-reference mean of 86.68%. The baseline subset contains shorter words than the full geometric population.

Qualification test 2: Reading handwritten content and identifying its writer (R2)

Objectives of the test or experiment

Evaluate reading and writer identification separately against 95%, using the recorded validation partitions.

Equipment used

Saved IAM and personal line images, reference transcripts, selected checkpoints and the scoring tools. This experiment does not require gantry movement.

Test setup and experimental parameters

The reader population has 976 public validation lines (42 072 characters) and 100 photographed lines (3 675 characters). The writer population has 156 lines, using held-out IAM pages and a seeded personal-line split. Personal validation can share page backgrounds with training. These are validation results because checkpoint selection may have used the same split.

Steps followed in the test or experiment

Check the corpus partitions, read each line with greedy CTC decoding, classify the normalised writer crop and retain the edits, reference counts and predictions. Aggregate the reader populations and writer confusion counts separately.

Results or measurements

Table 11 reports 94.06% pooled character accuracy for the selected joint reader. The scratch comparison reaches 94.39% on the same 1 076 lines. The training-route comparison is in Table 6. Table 23 retains the individual personal-writer scores.

Population	Lines	Characters	Joint / %	Scratch / %
IAM validation	976	42072	94.12	94.39
Personal validation	100	3675	93.33	94.42
Pooled	1076	45747	94.06	94.39

Table 11. Reader character accuracy on validation lines.

The writer model correctly identifies 153/156 lines (98.08%) and all eleven pooled writer groups. Figure 34 shows the row-normalised predictions, with actual line counts beside each writer. The only three errors are predictions of Y.

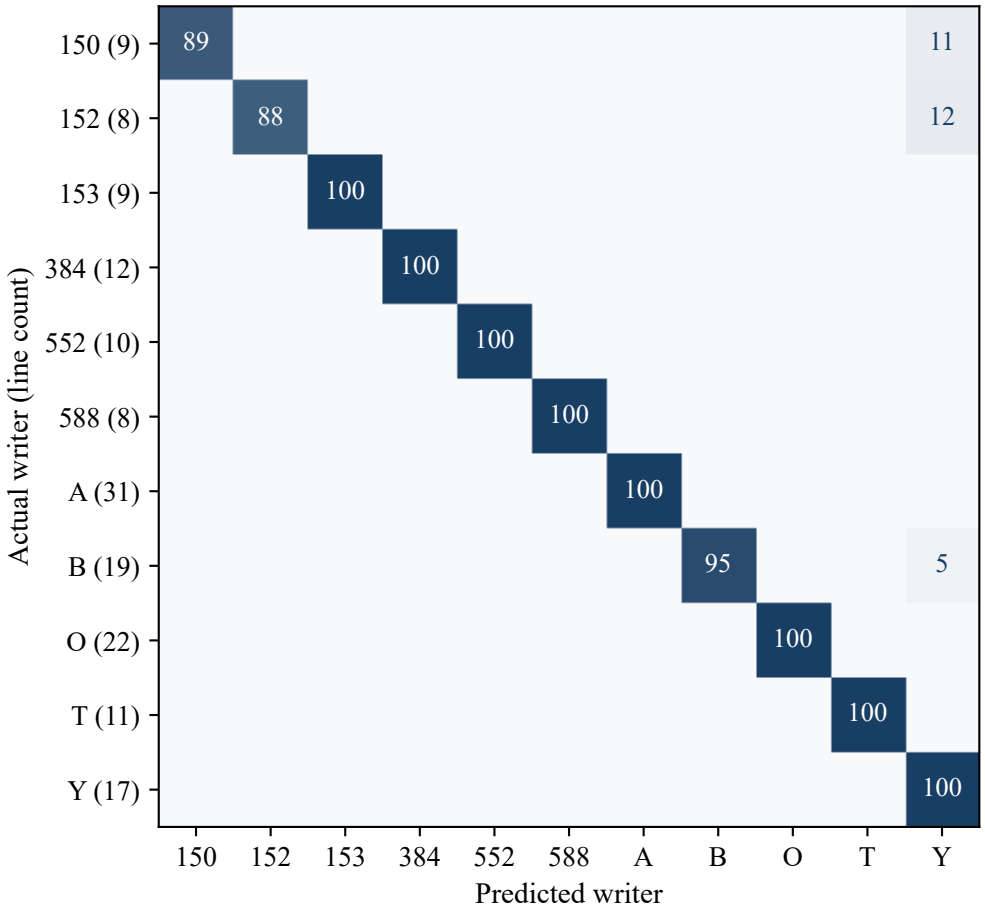


Figure 34. Writer validation predictions, row percentages.

Observations

The writer point estimate exceeds target by 3.08 percentage points. The selected reader is 0.94 points below target and recovers 221 complete lines without edits. One incorrect writer prediction has 83.8% confidence, showing that a high posterior alone cannot guarantee identity.

Statistical analysis

The writer Wilson interval is 94.5% to 99.3%. The result demonstrates a point estimate above 95%, with a stronger pooled decision across the available lines. Character accuracy is an edit-distance ratio, so a binomial interval on individual characters is not assigned.

Qualification test 3: Time to reproduce digital text on paper (R3)

Objectives of the test or experiment

Test whether a completed request containing x characters takes at most $4x$ seconds.

Equipment used

ODROID N2+, gantry, request timer and preserved execution log.

Test setup and experimental parameters

Retain positive-character drawing requests with explicit program-end confirmation. The total includes generation and physical drawing. Stopped jobs and commissioning shapes are excluded.

Steps followed in the test or experiment

Associate each timing record with its completion marker, divide total time by character count and group the retained records by request length.

Results or measurements

Table 12 gives the completed requests. Figure 35 shows every observation and the measured range at each length. All 25 complete records are in Appendix C.5.

Characters	Records	Allowance / s	Worst total / s	Worst / s per char
43	9	172	166.61	3.875
75	1	300	188.60	2.515
82	3	328	260.33	3.175
91	2	364	222.63	2.446
132	1	528	277.90	2.105
378	9	1512	1085.01	2.870

Table 12. Completed ODROID reproduction jobs by text length.

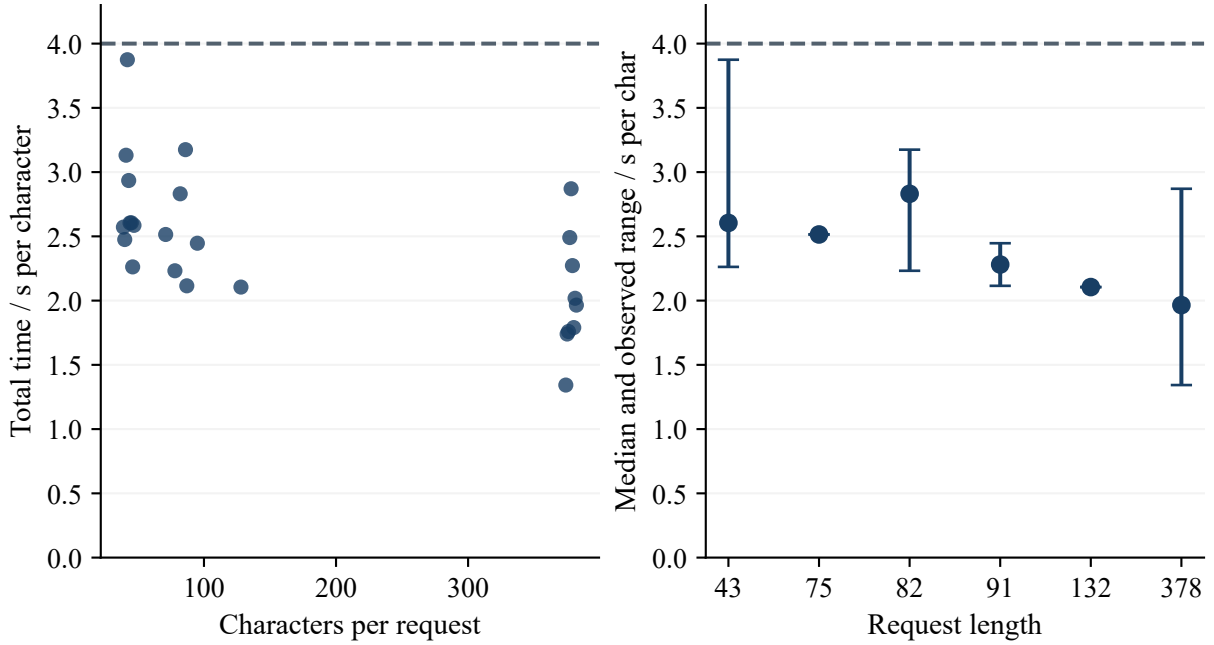


Figure 35. Completed reproduction requests and observed timing ranges.

Observations

All 25 requests are below 4 s per character. The largest value is 166.61/43 = 3.875 s per character. Nine 378-character requests take 507.57 to 1085.01 s.

Statistical analysis

There are one to nine observations at each of six lengths, from 43 to 378 characters. The all-pass fraction has a descriptive Wilson interval of 86.7% to 100%. The records do not sample all possible text and path combinations.

Qualification test 4: Time to decode a photographed page and identify its writer (R4)

Objectives of the test or experiment

Compare complete embedded classification with the allowance $2 + 0.025x$ seconds.

Equipment used

ODROID N2+, request-stage timer and preserved classification log. The laptop records use Windows 11, an Intel Family 6 Model 126 processor with eight logical cores, Python 3.14.3 and NumPy 2.4.3. A separate network benchmark compares sequential numerical inference with batched library modules.

Test setup and experimental parameters

The two embedded records contain 500 characters over 16 lines and 982 over 22 lines. The timer includes loading, conditioning, segmentation, cropping and both networks. Camera acquisition is outside this boundary. The laptop experiment contains thirteen photographed pages, each timed three times. L1 to L8 are writer Y pages and L9 to L13 are writer B pages.

Steps followed in the test or experiment

Read the complete totals and stage times, calculate the character-dependent allowances and compare the measured and allowed values.

Results or measurements

Figure 36 shows the measured stages and complete totals. Table 13 gives the numerical comparison. Appendix C.6 retains the recorded stages. Supporting acceleration records remain in the accompanying measurement archive.

Characters	Measured / s	Allowed / s	Overrun / s	Measured / allowed
500	46.26	14.50	31.76	3.19
982	55.49	26.55	28.94	2.09

Table 13. Embedded classification totals.

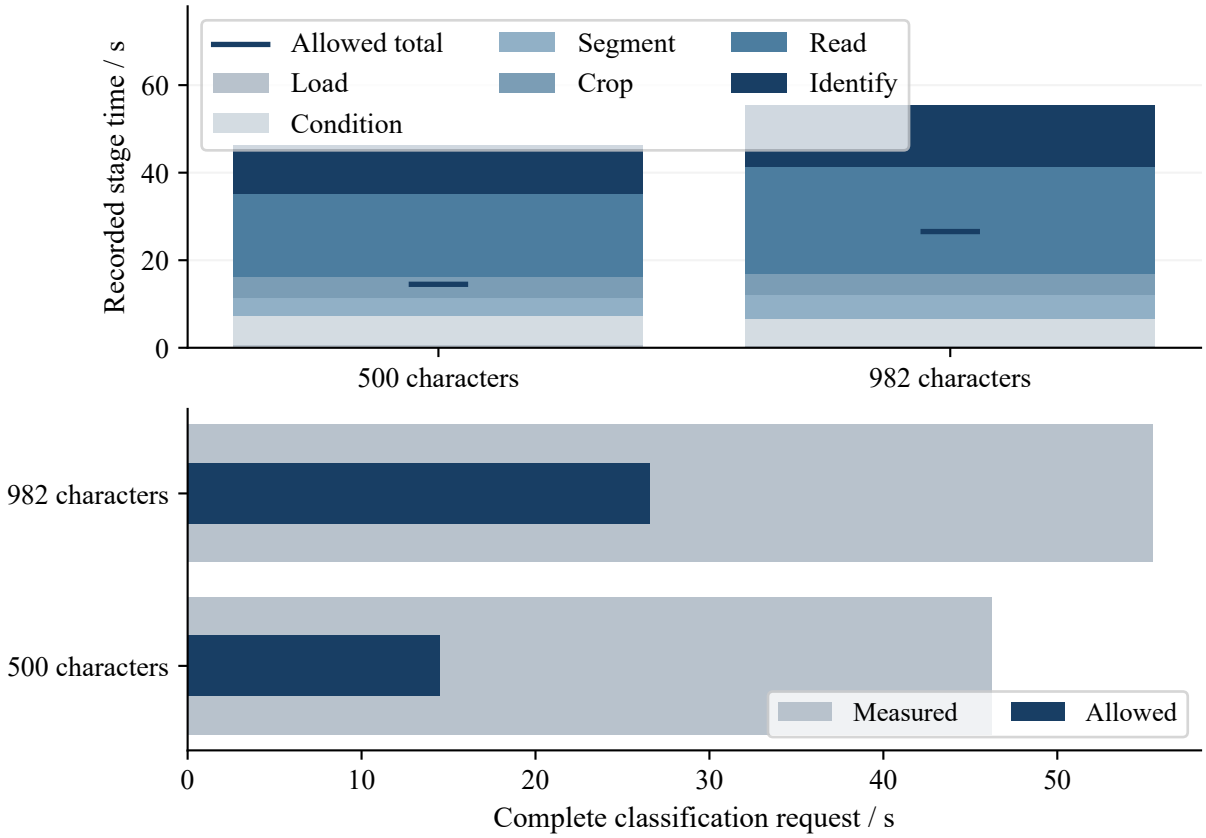


Figure 36. Recorded classification stages and time allowances.

The laptop measurements in Table 14 separate text recognition, author identification and image preparation. Each total is the mean of three recorded requests. All thirteen page means exceed their complete-request allowances.

Page	Characters	Image / s	Text / s	Author / s	Total / s	Limit / s
L1	976	34.77	7.77	5.93	48.48	26.40
L2	1168	28.94	8.44	6.60	43.99	31.20
L3	1291	25.15	8.48	6.65	40.28	34.27
L4	1399	30.00	8.38	7.04	45.42	36.98
L5	1677	33.45	9.60	7.66	50.71	43.93
L6	1301	30.94	8.64	6.74	46.32	34.52
L7	1316	31.28	8.57	6.67	46.52	34.90
L8	1388	31.71	9.38	7.43	48.52	36.70
L9	1186	37.30	9.14	6.97	53.41	31.65
L10	1296	57.94	15.92	11.65	85.51	34.40
L11	1262	69.84	17.36	12.87	100.06	33.55
L12	1040	53.73	14.14	10.66	78.53	28.00
L13	1186	53.56	14.03	11.88	79.47	31.65

Table 14. Laptop classification times, means of three runs per page. Image includes loading.

The existing fourteen-page network benchmark takes 359.8 s sequentially and 130.3 s using batched library modules, a factor of 2.76. Reader output is identical on every evaluated line. Both batching and backend change, so their individual gains and author-label equivalence are not established.

Table 15 applies that factor to the combined network stages of selected laptop records while retaining their measured image times. The estimate uses $t_{\text{image}} + (t_{\text{text}} + t_{\text{author}})/2.76$. This would bring L3, L4 and L5 inside their budgets if the factor transfers. L1 remains above its limit because image processing alone takes 34.77 s. These are conditional estimates from a separate benchmark, rather than measured passes.

Page	Characters	Recorded / s	Limit / s	Estimated / s	Estimate
L1	976	48.48	26.40	39.74	Above limit
L3	1291	40.28	34.27	30.63	Within limit
L4	1399	45.42	36.98	35.59	Within limit
L5	1677	50.71	43.93	39.70	Within limit

Table 15. Conditional laptop totals with unchanged image time and 2.76 times faster networks.

Observations

Both ODROID requests and all thirteen laptop page means exceed their allowances. Networks account for 67.6% of the combined ODROID time. The laptop records also show substantial image-processing cost, so network acceleration alone cannot bring every page within specification.

Statistical analysis

The laptop set has three repetitions per page and the embedded log has two complete requests. The library benchmark measures a network gain on fourteen saved pages. Its transfer to other requests or the ODROID is a projection. Halving image resolution is not an equivalent remedy, since the recorded trial lost lines and changed the recognised text.

Qualification test 5: Gantry dimensional accuracy and repeated geometric samples (R5)

Objectives of the test or experiment

Compare repeated square and triangle feature lengths with the 0.5 mm error limit.

Equipment used

Final calibrated gantry, pen, paper and the author's metal non-digital vernier caliper. The instrument model and least count are not recorded.

Test setup and experimental parameters

Ten supplied samples each contain four nominal 100.00 mm square sides, an 80.00 mm triangle base and two nominal 56.57 mm slopes. These give 70 readings. Errors use the supplied two-decimal nominal lengths.

Steps followed in the test or experiment

Measure all seven features on each sample, subtract the nominal length and retain signed errors. Group the readings by feature and sample without treating seven sides of one drawing as seven independent drawings.

Results or measurements

Figure 37 contains every signed error, so individual samples can be checked against the limit. Table 16 summarises the complete set. Appendix C.7 retains the feature statistics. All 70 supplied readings remain in the accompanying measurement archive.

Recorded quantity	Value
Geometric samples	10
Measured feature lengths	70
Lengths within 0.5 mm	70/70
Mean absolute length error	0.297 mm
RMS length error	0.327 mm
Largest absolute length error	0.49 mm

Table 16. Ten-sample dimensional summary.

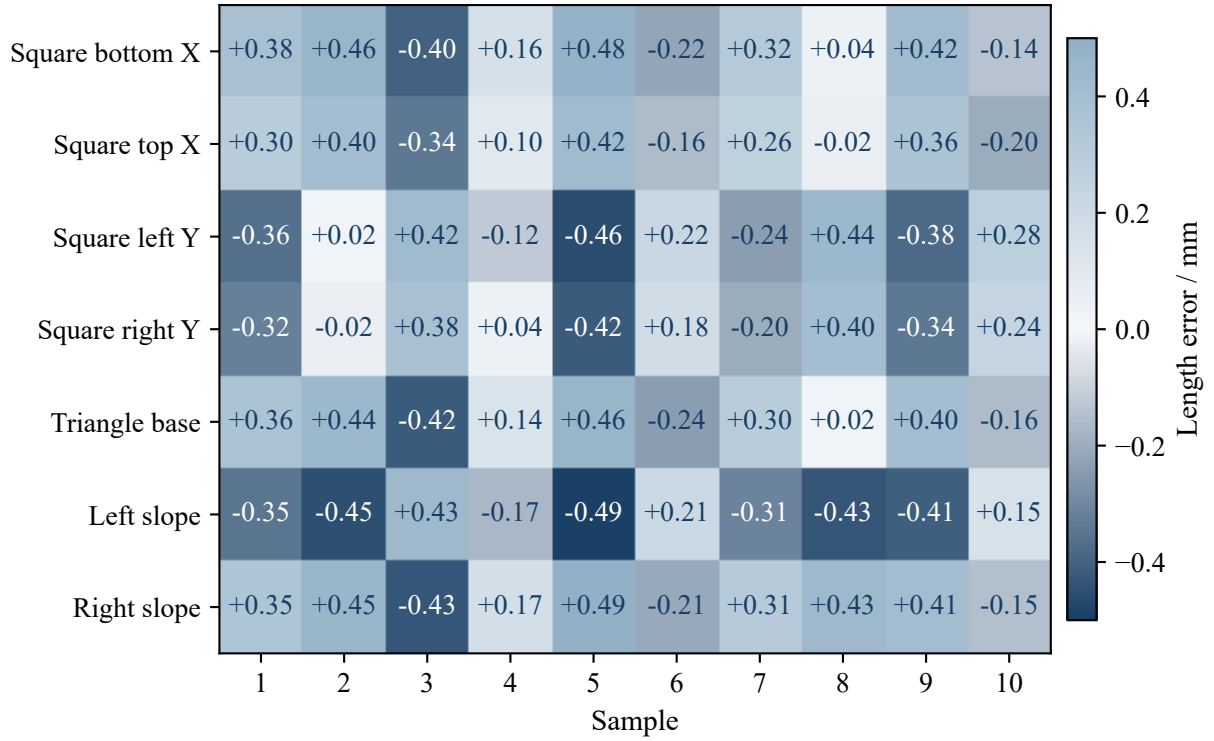


Figure 37. All 70 signed feature-length errors in millimetres.

Observations

All 70 reported lengths are within 0.5 mm of nominal. The largest errors are the two triangle slopes in sample 5, at −0.49 and +0.49 mm. Square-side errors change sign between samples. The supplied measurements describe feature lengths rather than absolute bed coordinates.

Statistical analysis

Each feature has ten readings. Sample standard deviations describe variation among those readings. The minimum observed margin to the limit is 0.01 mm. Measurement uncertainty cannot be assigned from the retained record because caliper resolution and the nib-edge reading convention are unspecified.

Qualification test 6: Storage and use of multiple trained styles (R6)

Objectives of the test or experiment

Check the required minimum of ten stored and usable writer styles.

Equipment used

Writer mapping, stored glyph profiles, generated jobs and fourteen physical output pages.

Test setup and experimental parameters

The evaluated set contains eleven identities. Each must have a nonempty stored glyph mapping and physical output.

Steps followed in the test or experiment

Compare model identities with profile keys and confirm the corresponding reproduced pages.

Results or measurements

All eleven stored styles have physical output, exceeding the required minimum of ten. Appendix C.8 lists their mapping and page counts, followed by the photographed pages.

Observations

The style count exceeds the required minimum by one. Character coverage differs between profiles.

Statistical analysis

This is a complete inventory of the evaluated profiles, rather than a sample estimate.

4.2.2 Qualification under the proposal's field conditions

The recorded tests use the fixed indoor capture and gantry arrangement. The capture variations already shown in Design extend the visual evidence. No calibrated illuminance or vibration series is assigned to these results.

5. Discussion

5.1 Critical evaluation of the design

5.1.1 Interpretation of results

The system reproduces eleven stored styles, with 84.41% physical character recovery and correct author identification on 86.41% of lines. All fourteen page pools are identified correctly. Seven writers exceed 85% character recovery, showing that performance depends on the selected style. Pooling provides more writer evidence than a short line, but does not correct unreadable letters.

The geometric test gives 86.68% agreement relative to the writers' own natural variation. Its 83 repeated words provide a separate physical-shape check, alongside classifier results. This ratio is not character accuracy or a correlation coefficient. The shorter-word subset and its 83.65% to 89.65% conditional interval limit a claim that all output meets the stylistic target. Recognition and direct shape measures remain complementary because recognition-based evaluation depends on its model [42, p. 2].

Original-line recognition reaches 94.06%, narrowly below 95%, while author identification reaches 98.08%. These are validation scores, since the partitions also support checkpoint selection. The results support the selected architectures without establishing performance on every unfamiliar page.

All 25 completed drawing requests meet the speed requirement. Classification remains the main timing shortfall. Both ODROID requests and all thirteen laptop page means exceed their allowances. The laptop stage records show why faster networks alone cannot solve every case. Some pages already spend their complete allowance on image preparation.

The measured batched-library benchmark reduces combined network time by a factor of 2.76. Applying that gain to unchanged laptop image times would bring L3, L4 and L5 within their limits. This demonstrates a practical route for some workloads, rather than a measured embedded pass. Output-preserving image optimisation and an ODROID rerun are still needed. Lowering resolution is unsuitable where it removes lines.

All 70 dimensions are within 0.5 mm, with a maximum error of 0.49 mm. This supports the calibrated transmission, but the 0.01 mm minimum margin is small. Unrecorded caliper resolution prevents an uncertainty-qualified pass. Side lengths also do not establish absolute pen position or return accuracy.

5.1.2 Critical evaluation

Line-based CTC recognition avoids requiring correct character cuts before reading [14, pp. 2–3]. Its main dependency is correct line ownership. Clipping an ascender or including a neighbouring line affects the sequence supplied to the reader.

The glyph library suits static samples and limited writer data [23, p. 4]. It gives direct control of geometry without requiring the recorded pen chronology used by trajectory generators [3, p. 18]. Its weakness is contextual shape. Cursive letters change with their neighbours, so combining isolated glyphs cannot recover every join. Contextual variants address this more directly than additional random trials.

The numerical implementation makes the network calculations inspectable. Similar validation scores and operation checks support its implementation, while batching offers a way to reuse the arithmetic more efficiently. The shared gantry shaft similarly reduces independent alignment decisions. Open-loop control still cannot detect a missed step after calibration. Controlled acceleration, consistent pulse timing and a firmer holder are appropriate improvements. The approximately R4 000 prototype cost excludes purchase of the borrowed ODROID, so a repeat build would need its replacement cost.

5.1.3 Unsolved problems

The priorities are faster classification with unchanged crops and predictions, better contextual glyphs for connected writers, and repeated coordinate and word-overlay measurements. Each change should be checked against the same inputs before a new integrated qualification.

5.1.4 Strong points of the design

The project connects offline samples to physical output in eleven styles. Source crops, stored paths, G-code and drawings make the process inspectable. Classifiers, direct geometry and dimensional measurements provide different checks on the same system.

5.1.5 Expected failure conditions

Strongly connected writing, missing glyphs, touching lines and curved pages remain difficult. An unknown writer can receive a confident known label. Slipping couplers, changed belt tension or disturbed paper can also invalidate the calibrated relationship between commanded and actual motion.

5.2 Considerations in the design

5.2.1 Ergonomics

Capture, writer selection and preview reduce repeated setup. The operator must still secure the sheet, calibrate the axes and check pen clearance before execution. Clear job and interruption feedback is necessary.

5.2.2 Health and safety

Moving belts and carriages create pinch points, while drivers dissipate heat. Accessible motor-power isolation is needed alongside software aborts. The pen circuit would benefit from the inductive protection described in Design [43].

5.2.3 Environmental impact

The computer will be returned and motion components can be reused. Replaceable printed supports and previews reduce discarded parts and trial pages. Training and motor holding consume energy, but their environmental impact was not metered.

5.2.4 Social and legal impact

Personalised writing may assist correspondence, but can also imitate another person's handwriting. Use should be authorised and machine output identified. Samples and profiles require controlled access under the personal-information considerations discussed in the literature [44, pp. 8–9, 11]. IAM data remain subject to their dataset terms.

5.2.5 Ethics clearance

All five personal contributors supplied signed consent. An institutional clearance or exemption reference is not established by the supplied records and remains separate from participant consent.

6. Conclusion

6.1 Summary of the work completed

A handwriting robot was designed and built to recognise photographed handwriting, identify a known writer and reproduce new text in that writer's style. Page conditioning and line segmentation feed the text and author classifiers. Labelled samples supply stored glyph trajectories, which are arranged into G-code and executed by the calibrated Cartesian gantry. Eleven profiles were demonstrated in physical output.

6.2 Summary of observations and findings

Original-line validation gives 94.06% character accuracy and 98.08% author identification. Physical output gives 84.41% character recovery and 86.41% line identification, with all fourteen page pools identified correctly. The separate geometric comparison retains 86.68% of measured natural agreement on 83 repeated words. This supports useful style retention while showing that recognisable identity does not imply identical letter geometry.

All 25 completed ODROID drawing requests meet four seconds per character. All 70 measured feature lengths are within 0.5 mm, with a largest error of 0.49 mm. Complete classification remains outside its time allowance on the recorded ODROID and laptop tests. A measured 2.76-fold network acceleration provides a credible improvement route, but complete embedded compliance requires further measurement and image-processing work.

6.3 Contribution

The project demonstrates a complete path from static handwriting samples to physical output in multiple styles. Its first-principles network implementation, explicit glyph geometry and direct motor control make the processing steps inspectable. Classifier scores, natural-variation comparisons and physical dimensions provide complementary evidence of the resulting performance.

6.4 Suggestions for future work

The next priorities are output-preserving batching and compiled processing, followed by complete ODROID timing tests. Contextual glyphs and multi-character units should improve connected styles. Repeated bed coordinates and word overlays would extend dimensional verification to placement and repeatability. Improved motor protection, acceleration control and pen restraint would strengthen the next hardware revision.

7. References

[1] A. Chen, “Handwritten text classification based on convolutional neural network,” *Highlights in Science, Engineering and Technology*, vol. 34, pp. 39–44, 2023.

[2] J. Ma, “A study on a hybrid CNN-RNN model for handwritten recognition based on deep learning,” in *Proc. 1st Int. Conf. Data Analysis and Machine Learning (DAML)*, 2023, pp. 268–273.

[3] A. Graves, “Generating sequences with recurrent neural networks,” arXiv preprint arXiv:1308.0850, 2013. [Online]. Available: https://arxiv.org/pdf/1308.0850

[4] U.-V. Marti and H. Bunke, “The IAM-database: an English sentence database for offline handwriting recognition,” *Int. J. Document Analysis and Recognition*, vol. 5, no. 1, pp. 39–46, 2002.

[5] G. R. Hemanth, M. Jayasree, S. K. Venii, P. Akshaya, and R. Saranya, “CNN-RNN based handwritten text recognition,” *ICTACT Journal on Soft Computing*, vol. 12, no. 1, pp. 2457–2463, Oct 2021.

[6] N. Otsu, “A threshold selection method from gray-level histograms,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62–66, 1979.

[7] D. Bradley and G. Roth, “Adaptive thresholding using the integral image,” *Journal of Graphics Tools*, vol. 12, no. 2, pp. 13–21, 2007. [Online]. Available: https://people.scs.carleton.ca/~roth/iit-publications-iti/docs/gerh-50002.pdf

[8] W. Postl, “Detection of linear oblique structures and skew scan in digitized documents,” in *Proc. 8th Int. Conf. Pattern Recognition (ICPR)*, 1986.

[9] A. Vinciarelli and J. Luettin, “A new normalization technique for cursive handwritten words,” *Pattern Recognition Letters*, vol. 22, no. 9, 2001.

[10] S. Avidan and A. Shamir, “Seam carving for content-aware image resizing,” *ACM Transactions on Graphics*, vol. 26, no. 3, pp. 10:1–10:9, 2007. [Online]. Available: https://www.cs.jhu.edu/~misha/ReadingSeminar/Papers/Avidan07.pdf

[11] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998. [Online]. Available: https://leon.bottou.org/papers/lecun-98h

[12] F. Kizilirmak and B. Yanikoglu, “CNN-BiLSTM model for English handwriting recognition: comprehensive evaluation on the IAM dataset,” arXiv preprint arXiv:2307.00664, 2023. [Online]. Available: https://arxiv.org/pdf/2307.00664

[13] B. Shi, X. Bai, and C. Yao, “An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 11, pp. 2298–2304, 2017.

[14] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, “Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks,” in *Proc. 23rd Int. Conf. Machine Learning (ICML)*, 2006, pp. 369–376. [Online]. Available: https://www.cs.toronto.edu/~graves/icml_2006.pdf

[15] S. Ioffe and C. Szegedy, “Batch normalization: accelerating deep network training by reducing internal covariate shift,” in *Proc. 32nd Int. Conf. Machine Learning (ICML)*, 2015, pp. 448–456. [Online]. Available: https://proceedings.mlr.press/v37/ioffe15.pdf

[16] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: a simple way to prevent neural networks from overfitting,” *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014. [Online]. Available: https://jmlr.org/papers/volume15/srivastava14a/srivastava14a.pdf

[17] C. Wigington, S. Stewart, B. Davis, B. Barrett, B. Price, and S. Cohen, “Data augmentation for recognition of handwritten words and lines using a CNN-LSTM network,” in *Proc. 14th IAPR Int. Conf. Document Analysis and Recognition (ICDAR)*, vol. 1, 2017, pp. 639–645.

[18] M. Bulacu and L. Schomaker, “Text-independent writer identification and verification using textural and allographic features,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 29, no. 4, pp. 701–717, 2007.

[19] S. He and L. Schomaker, “FragNet: writer identification using deep fragment networks,” *IEEE Transactions on Information Forensics and Security*, vol. 15, pp. 3013–3022, 2020.

[20] A. A. Brink, J. Smit, M. L. Bulacu, and L. R. B. Schomaker, “Writer identification using directional ink-trace width measurements,” *Pattern Recognition*, vol. 45, no. 1, pp. 162–171, 2012. [Online]. Available: https://www.ai.rug.nl/~mbulacu/brink-quill-pr2012.pdf

[21] L. Kang, P. Riba, Y. Wang, M. Rusiñol, A. Fornés, and M. Villegas, “GANwriting: content-conditioned generation of styled handwritten word images,” in *Computer Vision – ECCV 2020*. Springer, 2020, pp. 273–289.

[22] G. Dai, Y. Zhang, Q. Ke, Q. Guo, and S. Huang, “One-DM: one-shot diffusion mimicker for handwritten text generation,” in *Computer Vision – ECCV 2024*, ser. Lecture Notes in Computer Science. Springer, 2024. [Online]. Available: https://www.ecva.net/papers/eccv_2024/papers_ECCV/papers/07598.pdf

[23] T. S. F. Haines, O. Mac Aodha, and G. J. Brostow, “My text in your handwriting,” *ACM Transactions on Graphics*, vol. 35, no. 3, pp. 26:1–26:18, 2016. [Online]. Available: https://reality.cs.ucl.ac.uk/projects/haines/haines16handwriting.pdf

[24] O. Teikari, “Reproducing handwriting with position-controlled robots,” Master’s thesis, Tampere University of Technology, Tampere, Finland, 2016.

[25] D. H. Douglas and T. K. Peucker, “Algorithms for the reduction of the number of points required to represent a digitized line or its caricature,” *Cartographica*, vol. 10, no. 2, pp. 112–122, 1973.

[26] T. Y. Zhang and C. Y. Suen, “A fast parallel algorithm for thinning digital patterns,” *Communications of the ACM*, vol. 27, no. 3, pp. 236–239, 1984. [Online]. Available: https://agcggs680.pbworks.com/f/Zhan-Suen_algorithm.pdf

[27] IQS Directory, “Types of linear actuators,” IQS Directory Technical Library, 2024, [Online]. Available: https://www.iqsdirectory.com/articles/linear-actuator/types-of-linear-actuators.html.

[28] "NEMA 17 bipolar stepper motor, 1.68 A per phase, 200 full steps per revolution," Manufacturer datasheet.

[29] T. Dziedzic, A. Sen Yilmaz, and A. Szafrańska, "How fast is fluent handwriting?" *Forensic Science International*, vol. 383, p. 112896, Jun 2026.

[30] P. P. Acarnley, *Stepping Motors: A Guide to Theory and Practice*, 4th ed. London, U.K.: Institution of Electrical Engineers, 2002.

[31] Allegro MicroSystems, "A4988 DMOS microstepping driver with translator and overcurrent protection," Datasheet, rev. 8, 2022, [Online]. Available: https://www.allegromicro.com/-/media/files/datasheets/a4988-datasheet.pdf.

[32] Phipps Electronics, "GA12-N20-30 3 V, 150 rpm metal geared motor specifications," Product specification, 2024, [Online]. Available: https://www.phippselectronics.com/support/ga12-n20-metal-geared-motor-specifications/?seq_no=2 [Accessed: Oct. 31, 2026].

[33] Hardkernel, "ODROID-N2+ single-board computer," Hardware technical reference, [Online]. Available: https://www.hardkernel.com/shop/odroid-n2-with-4gbyte-ram-2/ [Accessed: Oct. 31, 2026].

[34] ON Semiconductor, "BC337/BC337-40 NPN general-purpose amplifier transistor," Datasheet, [Online]. Available: https://www.onsemi.com/download/data-sheet/pdf/bc337-fsc-d.pdf.

[35] Communica, "DML31P sub-miniature microswitch," Product listing, [Online]. Available: https://www.communica.co.za/products/dml31p [Accessed: Oct. 31, 2026].

[36] J. E. Bresenham, "Algorithm for computer control of a digital plotter," *IBM Systems Journal*, vol. 4, no. 1, pp. 25–30, 1965. [Online]. Available: https://janmr.com/files/papers/bresenham65.pdf

[37] R. M. Haralick, S. R. Sternberg, and X. Zhuang, "Image analysis using mathematical morphology," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 9, no. 4, pp. 532–550, 1987. [Online]. Available: https://www.haralick.org/journals/04767941.pdf

[38] A. Rosenfeld and J. L. Pfaltz, "Sequential operations in digital picture processing," *Journal of the ACM*, vol. 13, no. 4, pp. 471–494, 1966. [Online]. Available: https://www.cs.virginia.edu/~jlp/66.sequential.op.pdf

[39] K. Chellapilla, S. Puri, and P. Simard, "High performance convolutional neural networks for document processing," in *Proc. 10th Int. Workshop on Frontiers in Handwriting Recognition (ICFHR)*, 2006.

[40] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.

[41] Tanks and Temples, "Evaluation tutorial," Online, accessed 2 November 2026. [Online]. Available: https://tanksandtemples.org/tutorial/

[42] Z. Chen, D. Yang, J. Liang, X. Liu, Y. Wang, Z. Peng, and S. Huang, "Complex handwriting trajectory recovery: Evaluation metrics and algorithm," in *Asian Conference on Computer Vision*, 2022. [Online]. Available: https://arxiv.org/abs/2210.15879

[43] ROHM Semiconductor, “NPN transistor: Low-side switch fundamentals,” TechWeb, Transistors Basic, 2026, 27 February 2026; accessed 1 November 2026. [Online]. Available: https://techweb.rohm.com/product/transistors-diodes/transistors/27354/

[44] Republic of South Africa, “Protection of personal information act 4 of 2013,” 2013, official consolidated text. [Online]. Available: https://www.justice.gov.za/legislation/acts/2013-004.pdf [Accessed: 1 November 2026].

A. Extended design calculations and connections

Numerical substitutions extend the calculations referenced in Design.

A.1 Worked values from the implemented system

A.1.1 Page conditioning and line segmentation

The recorded Owen01 trace uses a stored photograph reduced before analysis, producing an 800×600 pre-deskew working image. At pixel $(317, 343)$, $G = 100$. Its 41×41 background window has prefix values 20696850, 18404173, 18359673 and 16338366. Equation 12 gives $\Sigma = 271370$, hence $B_r = 271370/1681 = 161.43367$ and $R = 100/161.43367 = 0.619449$.

For that image, Otsu's criterion selects bin 171. Its class masses are 0.07069375 and 0.92930625, with mean bin indices 124.98356 and 217.71913, giving between-class variance approximately 564.97942. The corresponding ratio threshold is $(171 + 0.5)(1.2/256) = 0.80390625$.

In the Owen01 trace, the scores at +0.5°, +1.0° and +1.5° are 442973, 464105 and 439403. The largest score across all seventeen angles is 464105, so the selected rotation is +1.0°.

In the recorded boundary between centres 38 and 61, $y_m = 49.5$ and $h_g = 11.5$. At row 49 the closed mask is zero, giving $C = 0.012[(49 - 49.5)/11.5]^2 = 0.0000226843$. At column 283, the selected predecessor is row 50 with stored cost 0.00641966. Its one-row change adds 0.02, so the accumulated cost is $0.00641966 + 0.02 + 0.0000226843 \simeq 0.02644234$.

At the implemented 0.012/0.02 setting, mean midpoint offset is 0.363 working pixels; removing midpoint attraction raises it to 4.055 pixels.

A.1.2 Network calculations

For the first Owen01 line, channel 0 of the first convolution at canvas row 34, column 462 gives $\sum WX = -1.150897$ with learned bias $b = -0.000435125$. Equation 19 therefore produces $z = -1.151332$. The nine actual input and learned-weight values are retained in Appendix A.2.

At the same activation, the checkpoint stores $\mu = 0.2163021$, $\sigma^2 = 0.02585394$, $\gamma = 1.0193636$ and $\beta = 0.1324657$. Using $\varepsilon = 10^{-5}$ in Equation 20 gives $y = 1.0193636(-1.1513321 - 0.2163021)/\sqrt{0.02585394 + 10^{-5}} + 0.1324657 \simeq -8.53619$.

In the first forward LSTM layer for that line, frame 10 and cell 0 have $c_{t-1} = -2.7277257$, $f_t = 0.7306111$, $i_t = 0.6666368$ and $g_t = -0.8560218$. Equation 21 gives $c_t \simeq -2.5635625$. With $o_t = 0.000145395$, the cell emits $h_t \simeq -0.000143680$.

For the complete labelled line "1.9. Vloggers using fame to support charities is a positive way", the recorded forward–backward total is $\log Z = -0.47033843$. Equation 22 gives $\mathcal{L}_{\mathrm{CTC}} = 0.47033843$ and $Z \simeq 0.6247908$.

At frame 11, the probability of V is $p_{11,V} = 0.98030665$ and the target-constrained occupancy is $q_{11,V} = 0.99999995$. The CTC logit derivative is therefore $p - q = -0.01969329$.

The same Owen crop has an estimated body-band height of 38 pixels. At the selected $\rho = 0.17$, the width calculation gives $w = \mathrm{clip}(\mathrm{round}(0.17 \times 38), 2, 9) = 6$, then $k = 2\lfloor 6/2 \rfloor + 1 = 7$.

A.1.3 Glyph extraction and reproduction calculations

For the `V` in that Owen line, frame 11 corresponds to state 11 of the blank-interleaved transcript. Its log emission is −0.01988991 and the three admissible predecessor scores are −13.91888, −10.01139 and −1.3624485. Equation 25 selects their maximum and gives $V = -0.01988991 - 1.3624485 = -1.3823384$. Backtracking assigns `V` the frame interval $[11, 12)$.

The refined right boundary of this `V` is source column 193. Although smoothing gives $\overline{p}_{193} = 2.8$ and the positive-column reference is 14.36, the raw ink count at column 193 is zero. The explicit blank-column rule therefore overrides 2.8/14.36 and assigns zero cut cost.

A traced point on this `V` is $(x, y) = (173, 59)$, with left cut $x_L = 146$, baseline $b = 107$, $h_x = 38$ and $\tan\phi = -0.0787017$. Equation 27 gives $v = 48/38 = 1.263158$ and $u_0 = 27/38 - 1.263158(-0.0787017) = 0.809939$. After the implementation's three-decimal intermediate rounding and measured post-trim shift $\delta = 0.4472489$, the stored point is $(u, v) = (0.363, 1.263)$ and the recorded lead is 0.447 x-heights.

Dylan's first stored `a` variant has recorded prior distance 0.114. Recomputing the pairwise rule on all nine final stored variants gives the smallest total distance, 33.3413, for variant index 4; the other totals range from 34.4306 to 42.9091.

In the generated Owen sentence "Writing with style.", the first placement call, after calibration and jitter, uses $(u, v) = (-0.01134565, 1.47270173)$, lead $l = 0.25117988$, zero shear and drift, and $h_x^{\mathrm{mm}} = 4$. With the initial placement origin $(0, 0)$, Equation 29 gives $x = 4(0.25117988 - 0.01134565) = 0.9593369$ mm and $y = 4(1.47270173) = 5.8908069$ mm.

The final Owen profile's stored glyph distances have median $\widetilde{d} = 0.18355$. Equation 30 gives $\lambda = \mathrm{clip}[(0.18355 - 0.24)/0.08, 0, 1] = 0$.

For the recorded Owen candidate 1, the 19-character target "Writing with style." differs from its reader output by one substitution, giving $A = 18/19 = 0.9473684$. The selected-writer posterior is $P = 0.5786133$, so Equation 31 gives $J = 0.7184359$.

In the worked Owen example, three pre-repair trials use the fixed text "Writing with style.", seeds 2026, 3003 and 3980, and the selected eleven-writer shape model with an explicit width fraction of 0.17. Candidate 1 has $(A, P, J) = (0.947, 0.579, 0.718)$, while candidate 3 has the same character score but $(P, J) = (0.464, 0.623)$. The combined criterion therefore selects candidate 1.

The generated Owen trajectory has $x_0 = 0.9593369$ mm and $y_1 = 6.6340616$ mm. A point $(2.9829210, 5.3303251)$ mm is placed using the configured origin $(0, 251.3)$ mm, margins $(15, 10)$ mm, unit fit scale and zero line shift. Equation 32 gives $(x_m, y_m) = (17.0235840, 239.9962635)$ mm. Three-decimal file emission writes this point as `X17.024 Y239.996`, connecting the trajectory calculation to the motion segment examined in Section 3.3.6.

For this same generated file, the unrounded distance accumulators are $D_d = 139.0263729$ mm and $D_u = 158.1555743$ mm. At $F_d = 900$ and $F_u = 2400$ mm/min, with 56 pen transitions and

two 0.35 s dwell settings, Equation 33 gives $t_{\mathrm{nom}} = 9.2684249 + 3.9538894 + 19.6 = 32.8223142$ s. The job metadata reports 32.8 s after rounding.

A.1.4 System integration calculation

Applying this vote to the first two archived Owen01 crops gives ink weights 20204 and 21750, with Owen probabilities 0.9996674 and 0.9999279 from the selected 0.17-width model. The recorded page-weight calculation gives $\overline{p}_{\mathrm{Owen}} = (20204 \times 0.9996674 + 21750 \times 0.9999279)/41954 \simeq 0.9998024$. This is a two-crop calculation trace, not the full-page evaluation reported in Results.

A.2 Additional recorded numerical checks

Recorded check	Values
Convolution patch storage	$368640 \times 4/2^{20} = 1.40625$ MiB
CTC finite-difference losses	0.4703364611173033, 0.4703403997759956
Perturbation and derivative	10^{-4}, −0.0196932934615
Analytic derivative and difference	−0.0196932934309, 3.06×10^{-11}
Switch confirmation intervals	6 + 6 = 12 ms

Table 17. Recorded calculation values.

The switch algorithm in Design uses the following recorded confirmation duration.

$$6\,\mathrm{ms} + 6\,\mathrm{ms} = 12\,\mathrm{ms}. \tag{36}$$

A.3 Extended training procedure

The following procedure is the one referenced in Design.

Algorithm 5 One numerical training batch and validation checkpoint decision

Require: Training batch, task, parameters θ and optimiser state.

$(X, y) \leftarrow$ task-specific preparation and training augmentation
$(F, \mathcal{K}) \leftarrow \text{BackboneForward}(X, \theta)$; retain caches
if task is reader **then**
    $(z, \mathcal{K}_h) \leftarrow \text{BiLSTMHead}(F)$; $(L, g) \leftarrow \text{CTC}(z, y)$
else
    $(z, \mathcal{K}_h) \leftarrow \text{WriterHead}(\text{MeanWidth}(F))$
    $(L, g) \leftarrow \text{CrossEntropy}(z, y)$
end if
$g_F \leftarrow \text{HeadBackward}(g, \mathcal{K}_h)$
$\nabla\theta \leftarrow \text{BackboneBackward}(g_F, \mathcal{K})$
if task is writer **then** clip total gradient norm to 5
$(\theta, \text{state}) \leftarrow \text{OptimiserStep}(\theta, \nabla\theta, \text{state})$
if epoch is complete **then**
    Validate with fixed preparation, stored normalisation and dropout disabled
    Update schedule; save progress; replace best only if its criterion improves
end if
return L, θ, state

A.4 Representation and connection records

Stage	Required input	Output and governing convention
Segmentation	Photographed or saved page	Ordered raw crops and full-page boxes; image y points down
Recognition	Line crops and model arrays	Text plus writer probabilities interpreted by the checkpoint mapping
Profile construction	Original lines, known transcripts and reader	Writer-keyed glyph paths in x-height units, baseline origin and y up
Synthesis	Text, stored profile, scale and search settings	Millimetre trajectory, source tags and internal selection scores
File conversion	Trajectory and workspace configuration	Absolute millimetre G-code, pen events, placement and timing budget
Physical execution	G-code, connected GPIO and completed calibration	Step-counter updates and job outcome; physical accuracy requires observation

Table 18. Representation boundaries that connect the three main processing stages to the gantry.

A.5 Controller pin and supply planning

Header pin / GPIO line	Connection	Direction and nominal level	Notes / verification
USB host	ODROID USB host → IMX179 camera	USB data and bus power	Camera current draw and captured resolution are not established by the sensor's advertised maximum.
Ethernet / network	ODROID RJ45 or Wi-Fi ↔ laptop/browser	Network data; no GPIO voltage	Operator request and response path.
ODROID DC jack	Supplied adapter → board power input	Actual adapter output unknown	The manufacturer's 12 V/2 A recommendation is not confirmation of the adapter in use [33].
External supply +	24 V supply → VMOT on both A4988 carriers	24 V DC; supply limit set to 2.11 A	A separate 0.35 A supply indication was recorded under one operating condition; neither value is the motor phase-current setting.
3.3 V header rail	ODROID dedicated 3.3 V output → both A4988 VDD inputs and pen-motor positive rail	3.3 V supply	Physical header pin and total rail loading are not recorded.
Ground return	ODROID GND, driver logic/motor returns, switch returns and BC337 emitter	0 V reference	A common signal reference is required; exact return routing and physical GND pin are not documented.

Table 19. Controller pin, signal-level and supply map for the installed system.

Header pin / GPIO line	Connection	Direction and nominal level	Notes / verification
7 / 64	`DIR1_PIN` → X-driver DIR	Output, nominal 0/3.3 V	Direction state; software inversion is configured for X.
11 / 68	`STEP1_PIN` → X-driver STEP	Output pulses, nominal 0/3.3 V	A4988 advances on STEP pulses; this is a logic input, not the motor supply.
12 / 81	`DIR2_PIN` → Y-driver DIR	Output, nominal 0/3.3 V	Direction state; software inversion is configured for Y.
13 / 69	`STEP2_PIN` → Y-driver STEP	Output pulses, nominal 0/3.3 V	A4988 advances on STEP pulses; this is a logic input, not the motor supply.
15 / 72	X-min switch → `X_MIN_PIN`	Input, active high, nominal 0/3.3 V	Reported 10 kΩ pull-down.
16 / 65	X-max switch → `X_MAX_PIN`	Input, active high, nominal 0/3.3 V	Reported 10 kΩ pull-down.
18 / 66	Y-min switch → `Y_MIN_PIN`	Input, active high, nominal 0/3.3 V	Reported 10 kΩ pull-down.
22 / 67	Y-max switch → `Y_MAX_PIN`	Input, active high, nominal 0/3.3 V	Reported 10 kΩ pull-down.
33 / 70	Pen-up switch → `PEN_SWITCH_PIN`	Input, active high, nominal 0/3.3 V	High indicates the raised state. Switch model and pull-down location are not recorded.
35 / 71	`PEN_MOTOR_PIN` → 1 kΩ base resistor → BC337	Output, nominal 0/3.3 V	GPIO controls the transistor only; motor current is supplied by the dedicated 3.3 V rail. A 10 kΩ base pull-down was reported; its connection is not documented. No flyback diode is fitted.
A4988 outputs	X/Y carrier 1A/1B and 2A/2B → motor coil pairs	Switched/chopped winding drive from VMOT	The coil voltage is not a fixed 24 V STEP signal. Phase-current limit depends on VREF and sense resistance; actual VREF remains unmeasured [31, pp. 4–5, 9].

Table 19. Controller pin, signal-level and supply map (continued).

B. Extended design figures and subsystem evidence

B.1 Recorded capture and seam checks

Page	Label lines	Reduced	Gradient	+3°	90°
Owen	20	19	20	11	19
Dylan	24	24	24	25	24
Yeukita	26	26	26	26	26

Table 20. Exploratory crop counts after extra input reduction and digital perturbation. Equal counts do not establish correct ink ownership.

Midpoint weight	Step penalty	Ink intersections	Row movement	Mean offset / px
0.000	0.020	2	35	4.0553
0.006	0.020	2	38	0.5517
0.012	0.020	2	42	0.3625
0.024	0.020	2	44	0.3261
0.048	0.020	2	47	0.2622
0.012	0.000	2	48	0.2622
0.012	0.010	2	44	0.3261
0.012	0.040	2	38	0.5517

Table 21. New coefficient-sensitivity check across six fixed real-page gaps; the implemented setting is 0.012/0.020.

B.2 Numerical verification records

Figure 38 compares analytic gradients with finite differences.

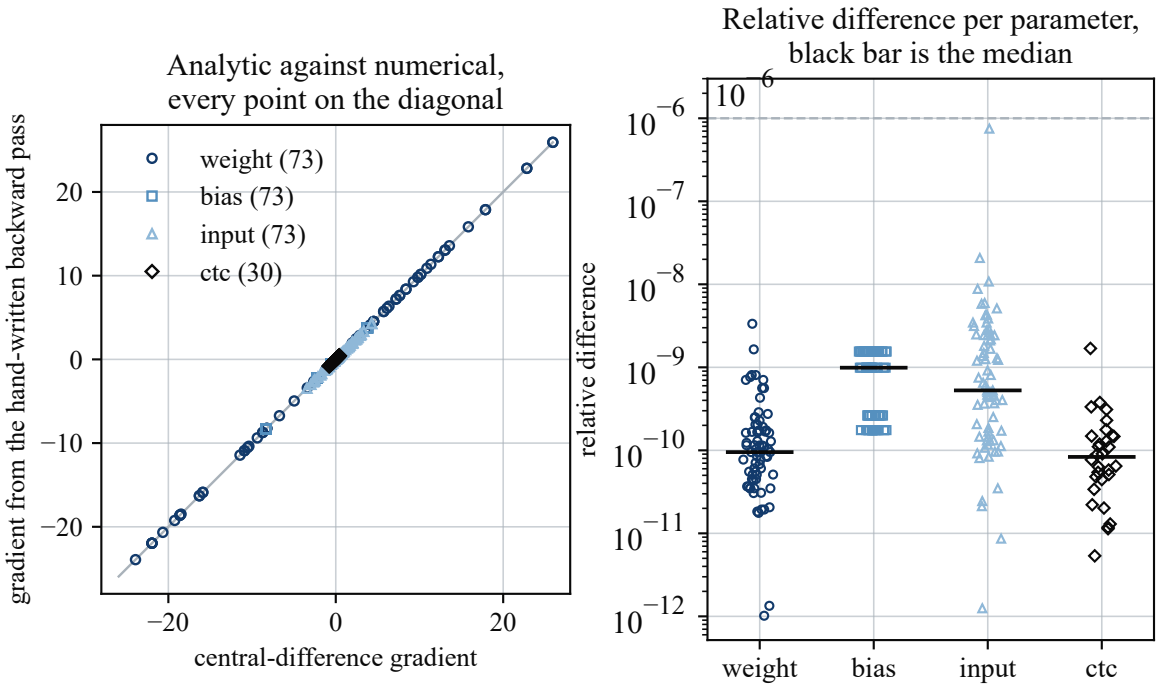


Figure 38. Analytic and finite-difference gradients.

Figure 39 extends the normalisation-width sweep.

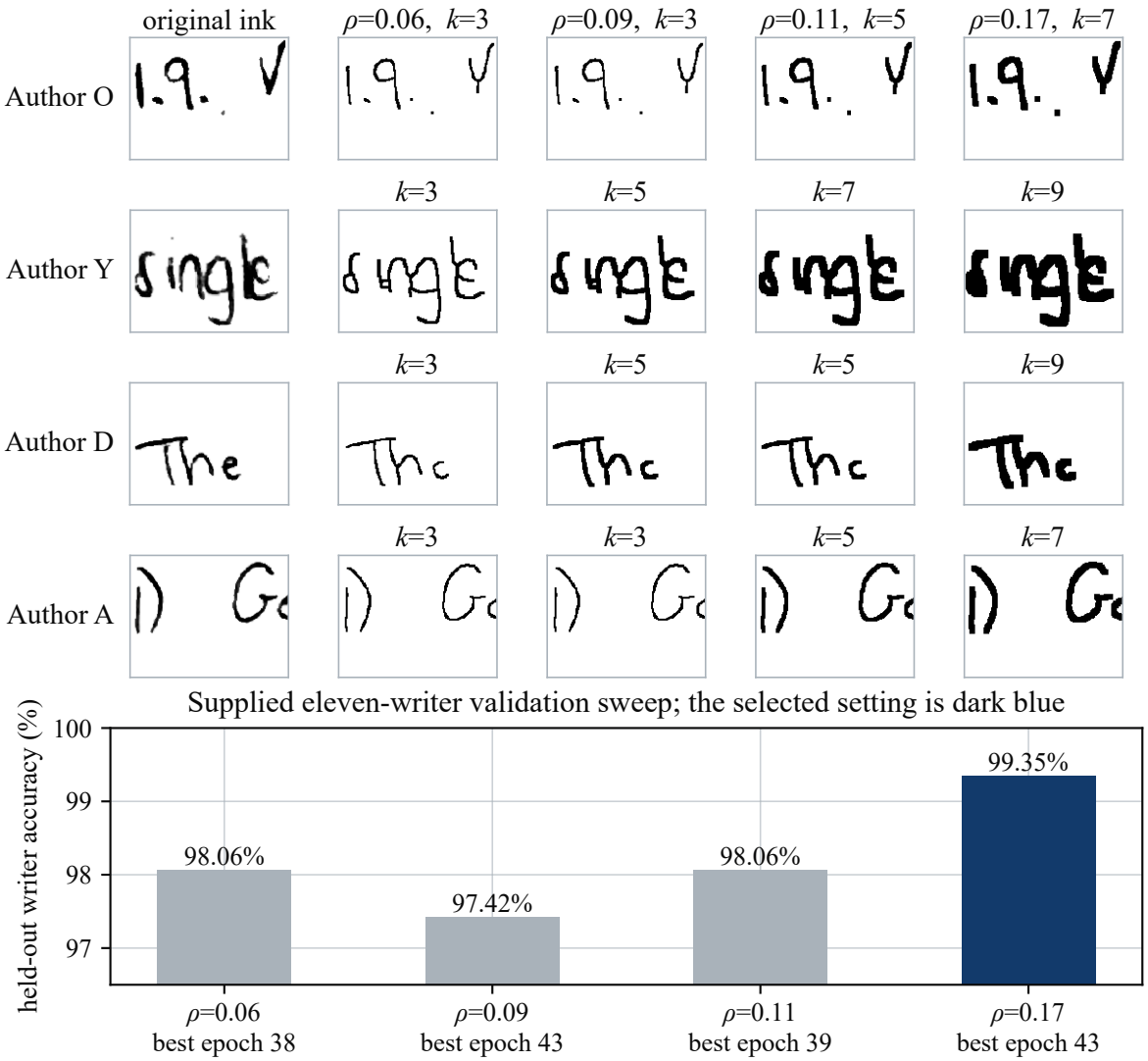
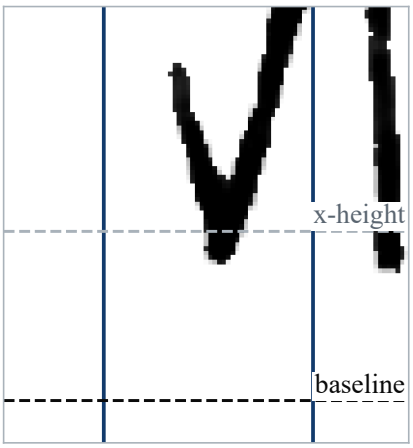


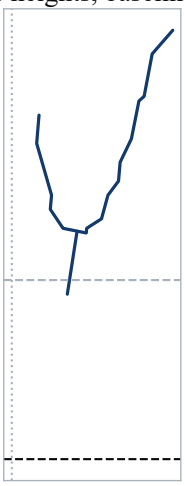
Figure 39. Stroke-width kernels and validation sweep.

Figure 40 shows the recorded glyph coordinate frames.

(a) the cell for 'V' in image pixels
baseline b=107 px, h_x=38 px
measured slant -4.50°



(b) stored, de-slanted
v in x-heights, baseline at 0



(c) the stored transform

$v = (b - y)/h_x$

$u_0 = (x - x_L)/h_x - v\tan\phi$

$u = u_0 - \delta$

(d) what is stored for this cell

x_L = 146.0 px
b = 107 px
h_x = 38 px
$\tan\phi$ = -0.0787017

stored width = 0.710 h_x
advance = 1.237 h_x
lead δ = 0.447 h_x
top = 2.395, bottom = 0.921

entry = (0.152, 1.921)
exit = (0.363, 1.263)
joined left = False
joined right = False

Figure 40. Image and stored glyph coordinate frames.

B.3 Extended figures from the Design section

Figure 41 extends the seam examples retained in Design.

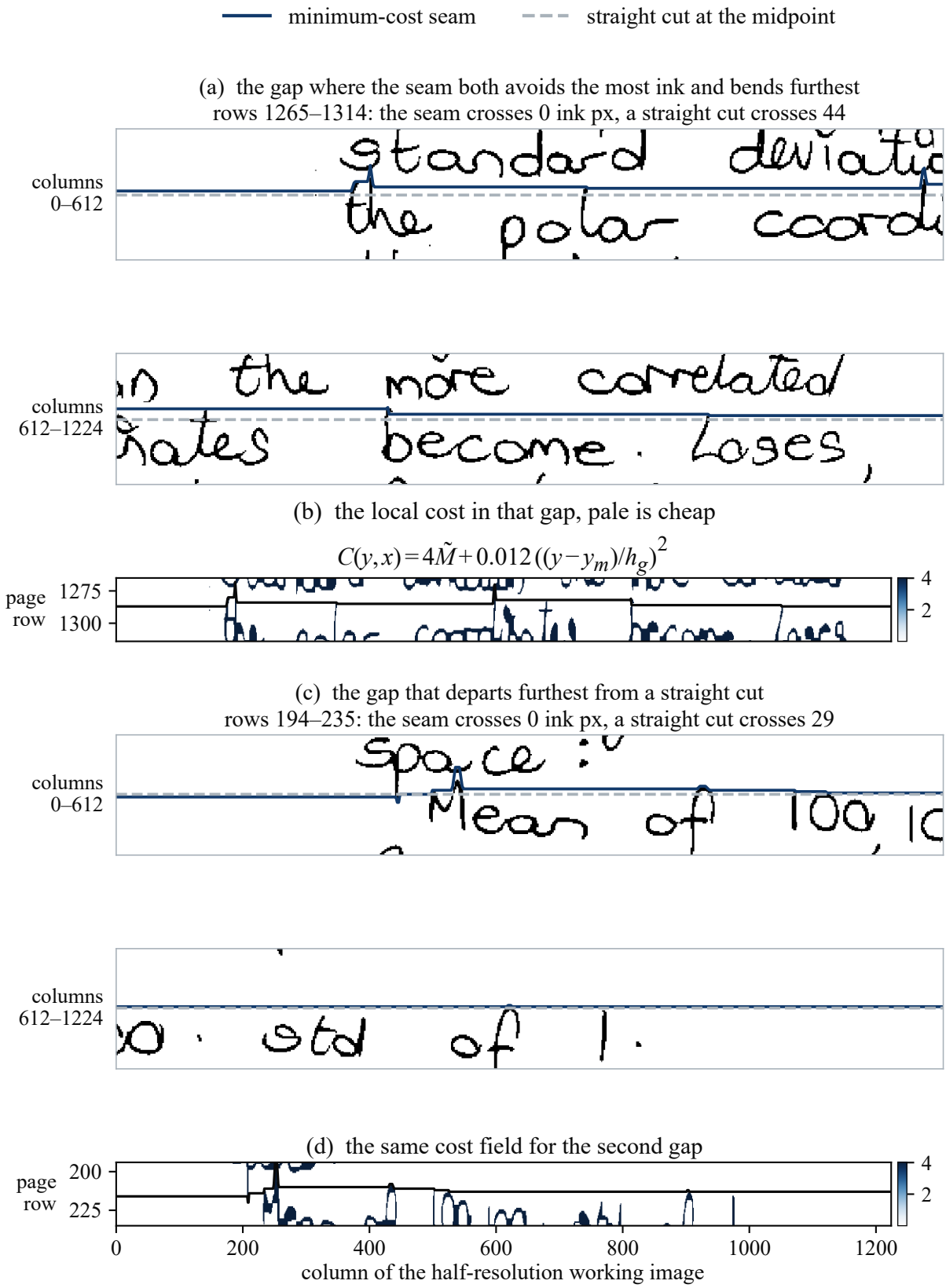


Figure 41. Complete seam and cost-field examples.

Figure 42 shows the complete feature sequence.

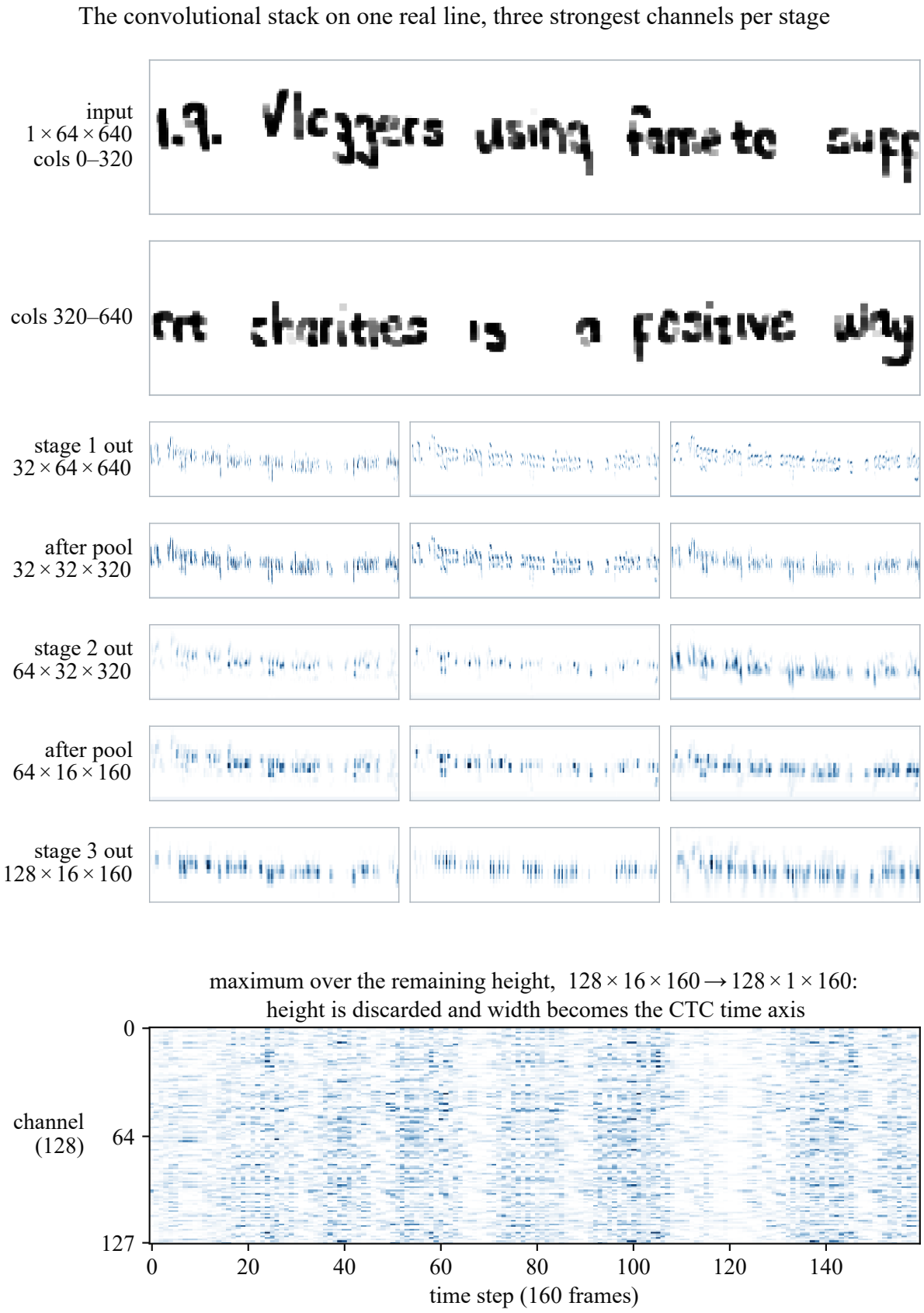


Figure 42. Complete convolutional feature sequence.

C. Qualification evidence

C.1 Measurement records

Record	Identity or scope
Physical classifier evidence	184 paired lines, 14 pages, 5471 reference characters
Evaluated reader	joint_best, SHA-256 prefix `336802d535992323`
Evaluated writer	11_pw170, `1884fb86467538eb`
Numerical comparison reader	scratch_best, `2add315ded078de5`
Numerical comparison writer	11w_pw170_scratch, `b23a709a30144f67`
Checkpoint training provenance	The evaluated joint/pw170 weights use the recorded library training route. Numerical comparison models are listed separately.
Execution records	Physical motion and request timings are ODROID records. Saved-image scoring and geometric analyses are offline verification on the development host.

Table 22. Identities and scope of the retained measurements.

C.2 Original-handwriting classifier evidence

Evaluation set	Lines	Characters	Joint / %	NumPy / %
Public IAM validation	976	42072	94.12	94.39
Photographed A	31	934	94.00	94.65
Photographed B	19	739	95.81	97.70
Photographed O	22	1017	94.40	94.79
Photographed T	11	345	85.51	89.28
Photographed Y	17	640	92.03	92.50
All	1076	45747	94.06	94.39

Table 23. Reader character accuracy by validation population.

The complete author confusion matrix is shown in QTP2.

C.3 Reproduced-page classifier evidence

The text and author classifier results are in QTP1. The following breakdown retains the three tested text blocks.

Text block	Lines	Characters	Character / %	Writer / %
Paragraph	119	4041	84.88	91.60
Pangram	21	463	78.40	76.19
Source text	44	967	85.32	77.27
All	184	5471	84.41	86.41

Table 24. Physical reproduction scores by text block.

C.4 Geometric comparison evidence

The following values support QTP1's natural-variation comparison. They use the same original reference banks for genuine and robot probes.

Writer	Words	Genuine F / %	Robot F / %	Relative / %	Nearest / %
150	8	90.01	87.73	97.47	97.26
152	8	91.61	74.99	81.86	83.63
153	6	89.24	74.12	83.05	82.90
384	6	80.14	72.34	90.27	88.99
552	8	79.92	61.95	77.51	83.54
588	5	75.66	64.16	84.80	91.36
A	12	72.98	60.40	82.76	82.40
B	10	74.11	69.70	94.05	93.83
O	5	76.03	70.22	92.36	97.50
T	4	66.24	49.19	74.26	70.34
Y	11	77.99	74.16	95.10	95.09

Table 25. All writers in the repeated-word comparison.

Figures 43 to 46 show further retained comparisons. Each left column compares two genuine samples. Each right column compares a robot word with its selected genuine reference. The printed F-scores are individual nearest-reference examples, rather than the ordinary-reference writer averages.

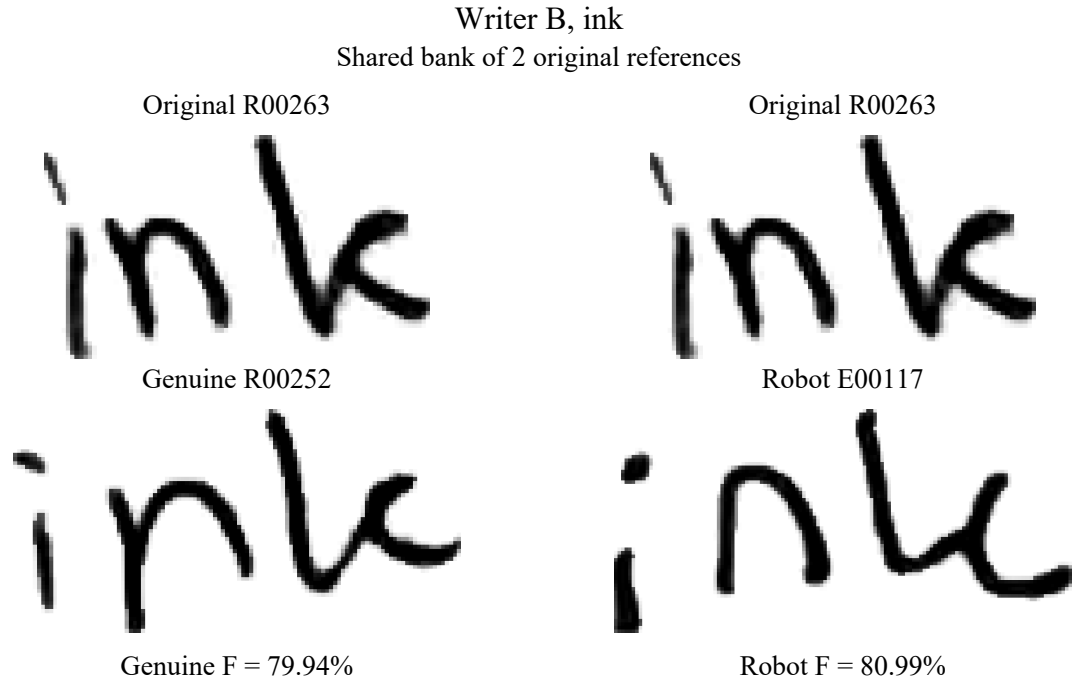


Figure 43. Genuine and robot word comparisons for writer B.

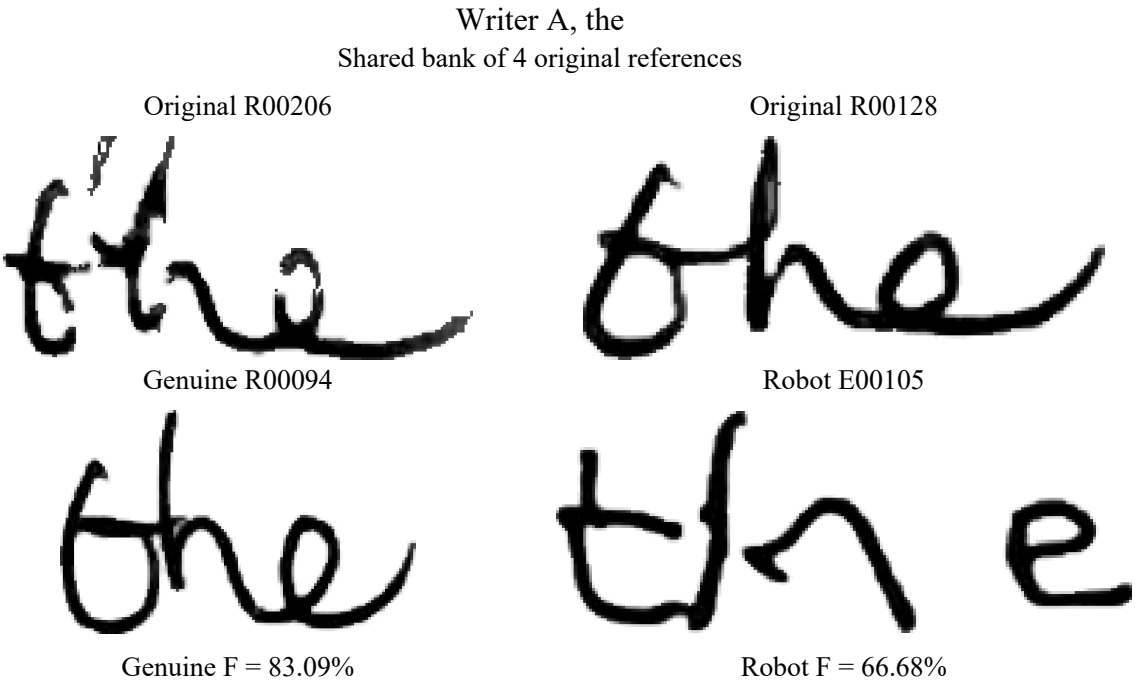


Figure 44. Genuine and robot word comparisons for writer A.

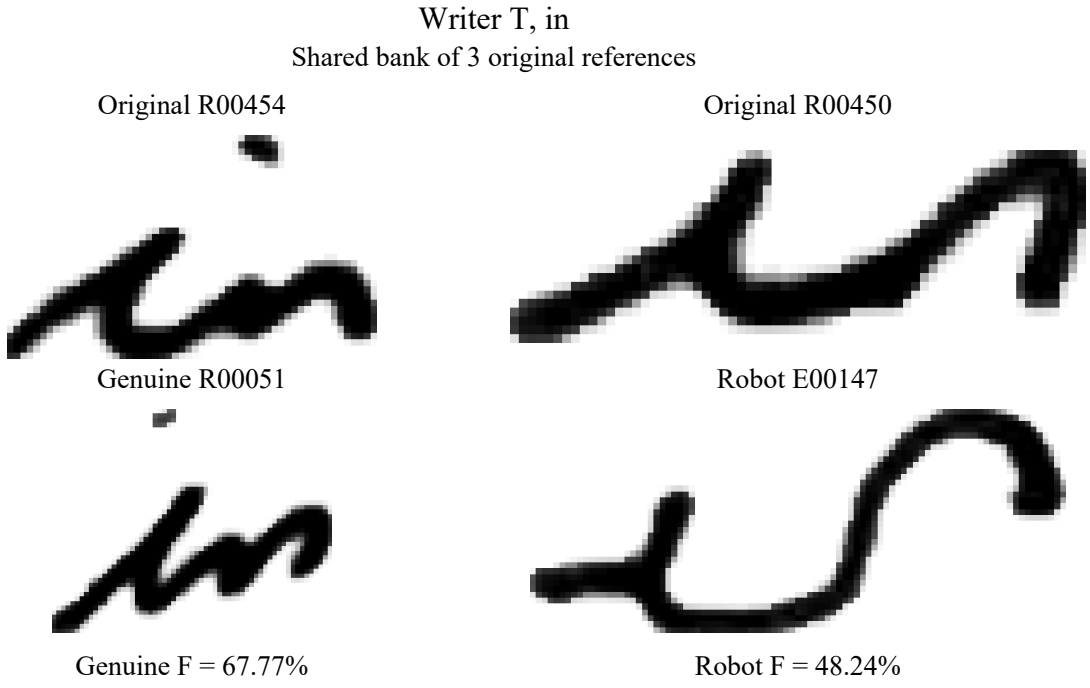


Figure 45. Genuine and robot word comparisons for writer T.

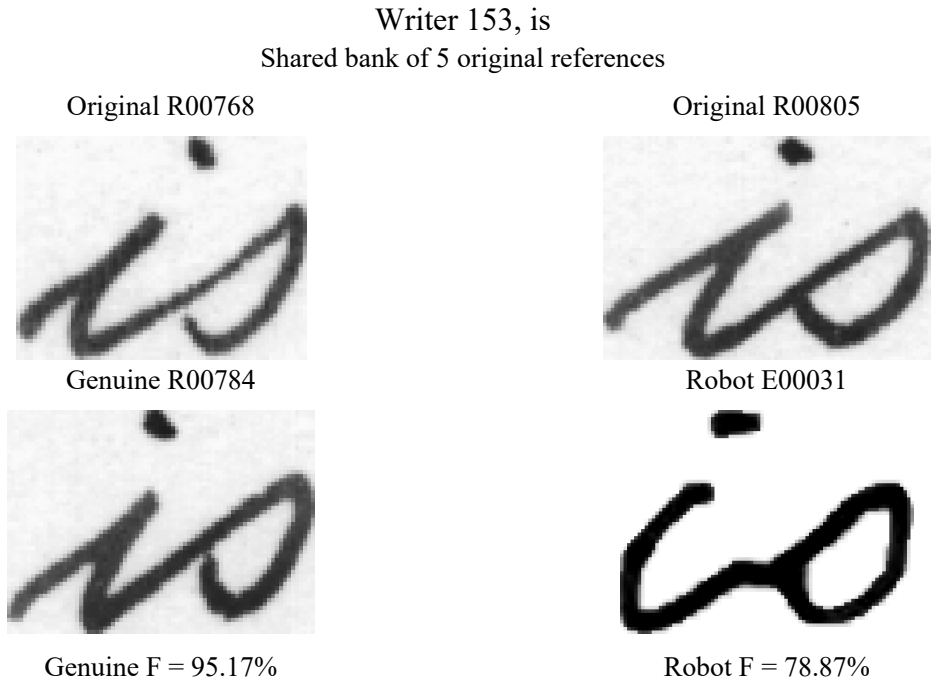


Figure 46. Genuine and robot word comparisons for writer 153.

C.5 Completed reproduction timing records

Table 26. All confirmed ODROID reproduction timings.

Log line	Characters	Synthesis / s	Drawing / s	Total / s	s per character
32	378	8.23	497.55	507.57	1.343
185	378	17.98	637.31	657.72	1.740
203	43	17.66	92.59	110.59	2.572
595	91	16.44	175.50	192.47	2.115
631	378	12.36	651.18	665.14	1.760
682	43	14.80	91.30	106.44	2.475
718	82	17.06	165.46	183.04	2.232
816	378	39.27	899.55	941.48	2.491
864	43	20.34	113.94	134.65	3.131
887	82	27.53	204.05	232.16	2.831
1004	378	58.78	1022.83	1085.01	2.870
1027	43	26.52	139.66	166.61	3.875
1050	82	33.67	225.88	260.33	3.175
1099	378	16.63	839.33	859.00	2.272
1122	43	18.81	106.99	126.19	2.935
1145	75	25.39	162.76	188.60	2.515
1189	378	12.11	663.01	676.64	1.790
1225	43	15.10	96.53	112.00	2.605
1260	43	15.22	96.44	112.03	2.605
1348	378	104.08	656.92	762.84	2.018
1384	43	15.65	81.40	97.28	2.262
1407	91	23.34	198.59	222.63	2.446
1495	378	14.17	726.40	742.72	1.965
1518	43	15.19	95.64	111.18	2.586
1541	132	22.88	254.16	277.90	2.105

C.6 Recorded classification stages

C.7 Dimensional measurement evidence

QTP5 shows all 70 signed errors. These statistics retain the ten readings per feature. The supplied readings are retained in the measurement archive.

Recorded quantity	500 characters	982 characters
Loading / s	0.63	0.53
Conditioning / s	6.61	6.04
Segmentation / s	4.29	5.59
Cropping / s	4.61	4.65
Image stages / s	16.14	16.81
Reading / s	19.02	24.66
Writer classification / s	11.09	14.01
Network stages / s	30.12	38.67
Total / s	46.26	55.49
R4 allowance / s	14.50	26.55
Overrun / s	31.76	28.94

Table 27. Recorded ODROID classification-stage timings.

Feature	Mean / mm	Sample SD / mm	Min / mm	Max / mm
Square bottom X	+0.150	0.315	-0.40	+0.48
Square top X	+0.112	0.277	-0.34	+0.42
Square left Y	-0.018	0.342	-0.46	+0.44
Square right Y	-0.006	0.304	-0.42	+0.40
Triangle base	+0.130	0.315	-0.42	+0.46
Triangle left slope	-0.182	0.327	-0.49	+0.43
Triangle right slope	+0.182	0.327	-0.43	+0.49

Table 28. Repeated feature-length error statistics, ten readings per feature.

C.8 Stored profile evidence

Writer	Character mappings	Physical pages
150	42	1
152	38	1
153	37	2
384	36	2
552	40	1
588	41	1
A	56	2
B	42	1
O	43	1
T	42	1
Y	58	1

Table 29. Stored profiles and physical-page coverage.

C.9 Training log extracts

Selected rows from the supplied first-principles logs are retained below. Full transcribed training records remain in the accompanying evidence archive.

Table 30. Selected writer training log records.

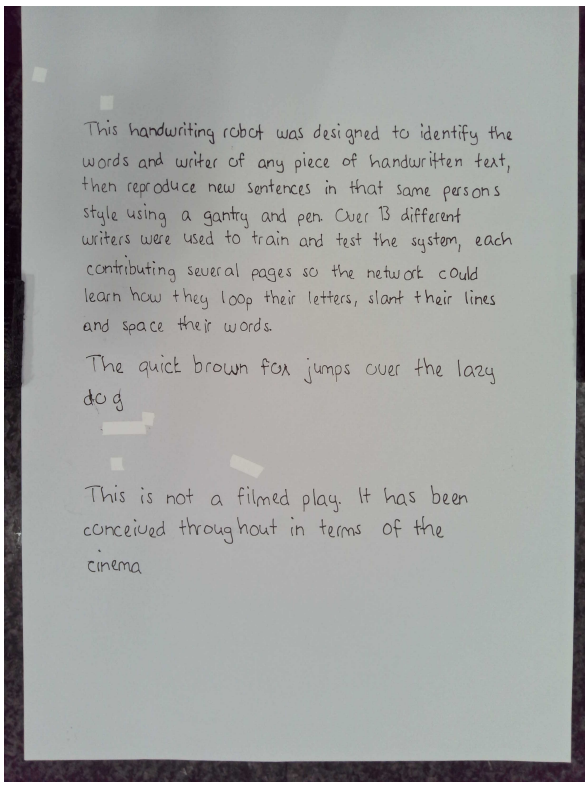
Epoch	Loss	Validation / %	Time / min
1	1.979	47.7	20.1
10	0.531	87.7	20.1
18	0.378	94.2	20.6
31	0.154	95.5	19.6
41	0.068	96.8	19.5
47	0.082	96.8	20.9

Table 31. Selected reader training log records.

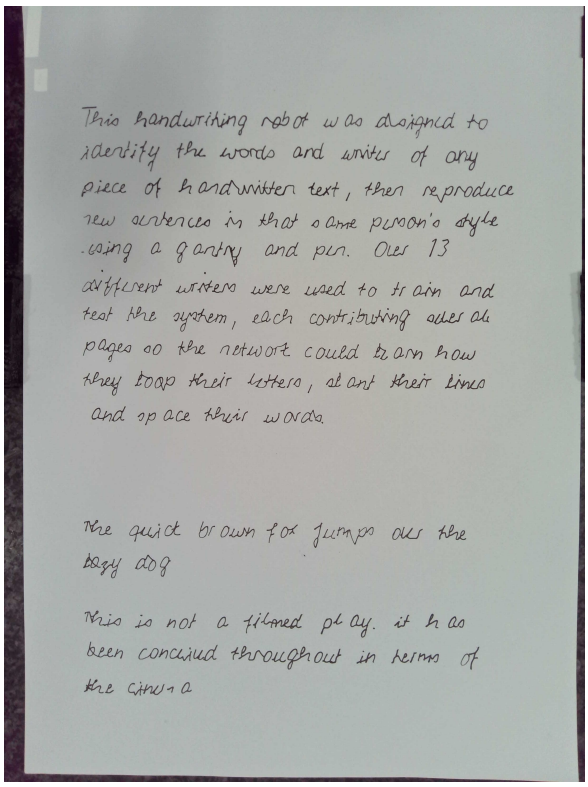
Epoch	Train loss	Val loss	Val char / %	Personal / %	Time / min
7	45.339	44.488	70.17	66.56	122.3
8	24.865	18.597	87.62	88.54	120.1
11	12.862	13.298	91.43	91.40	111.6
16	7.045	11.663	92.89	93.09	112.3

C.10 Complete physical-page photographs

Figures 47 to 53 retain all fourteen photographed output pages.

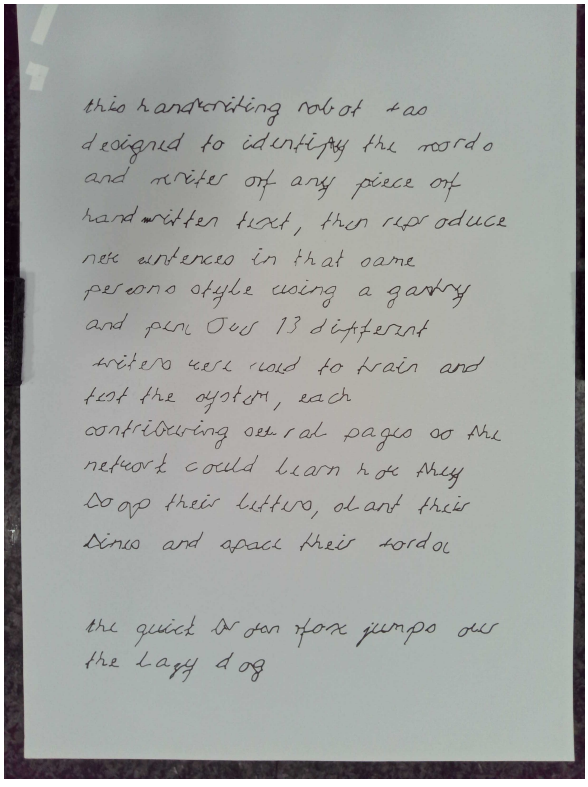


Author150

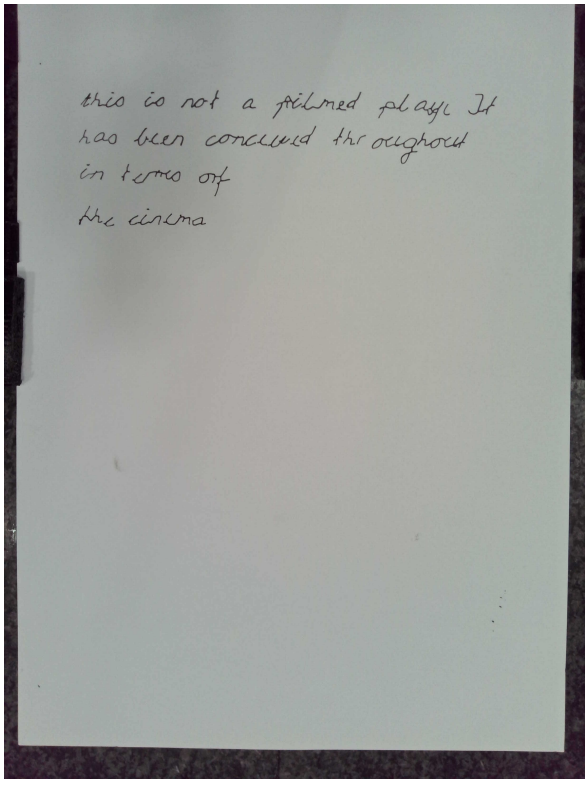


Author152

Figure 47. Physical pages Author150, Author152.

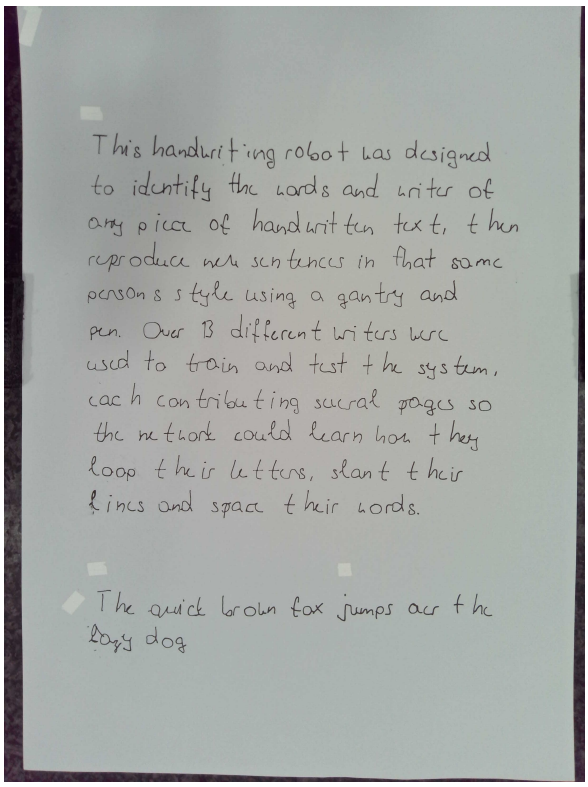


Author1531

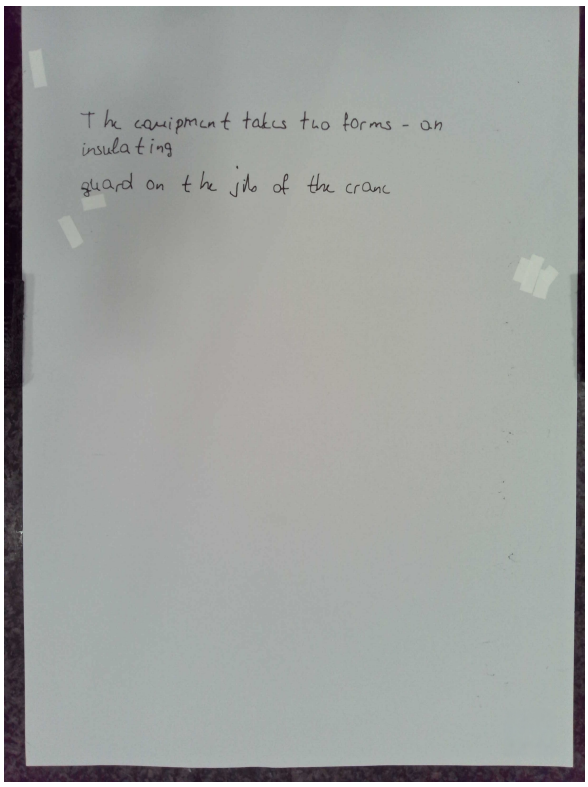


Author1532

Figure 48. Physical pages Author1531, Author1532.

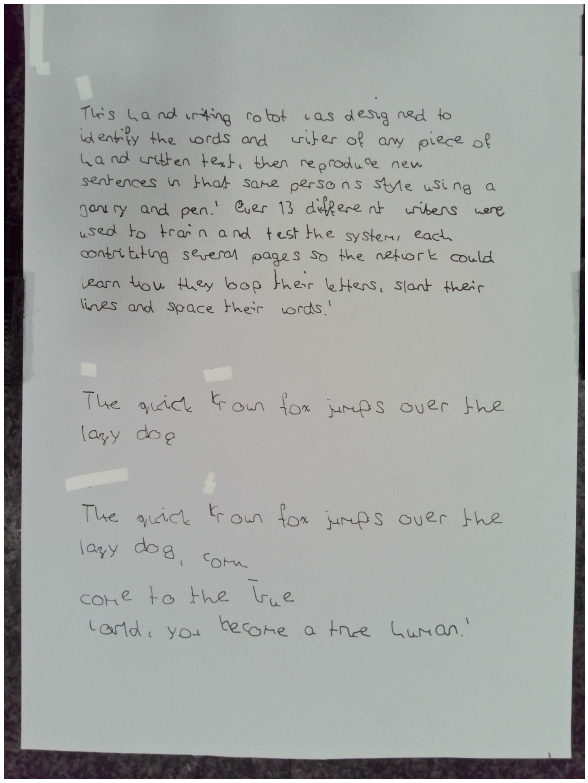


Author3841

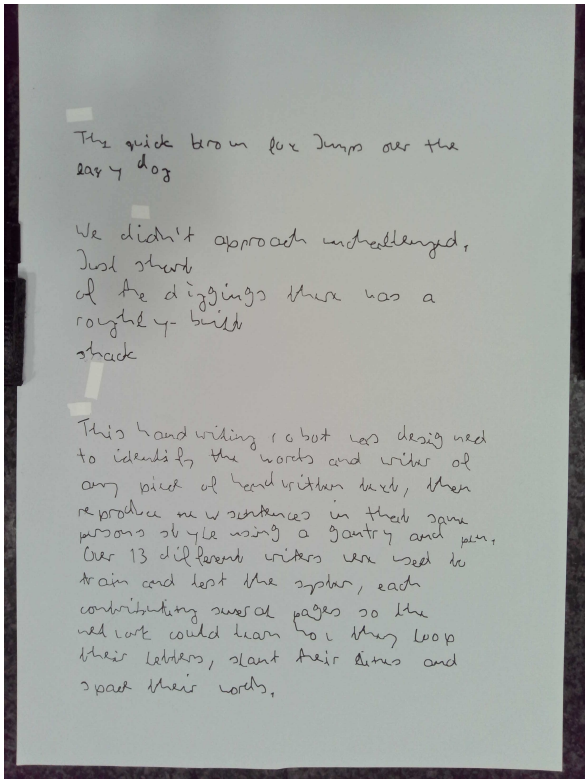


Author3842

Figure 49. Physical pages Author3841, Author3842.

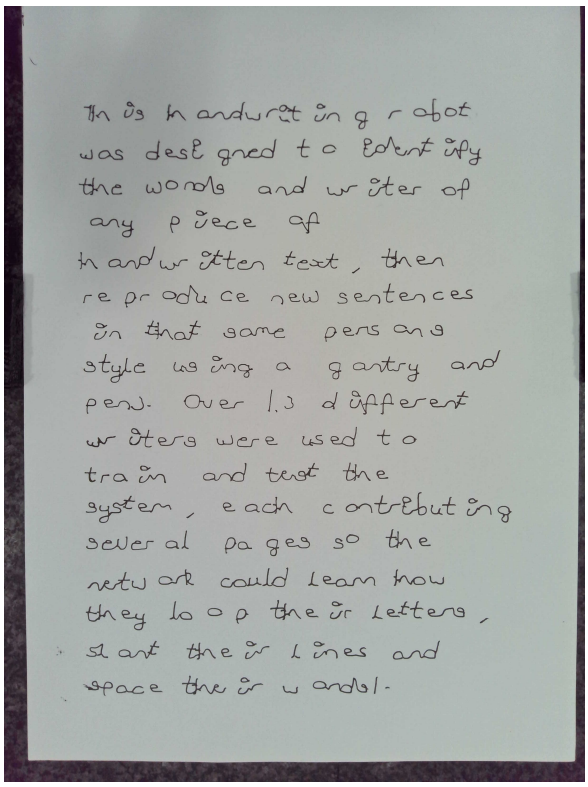


Author552

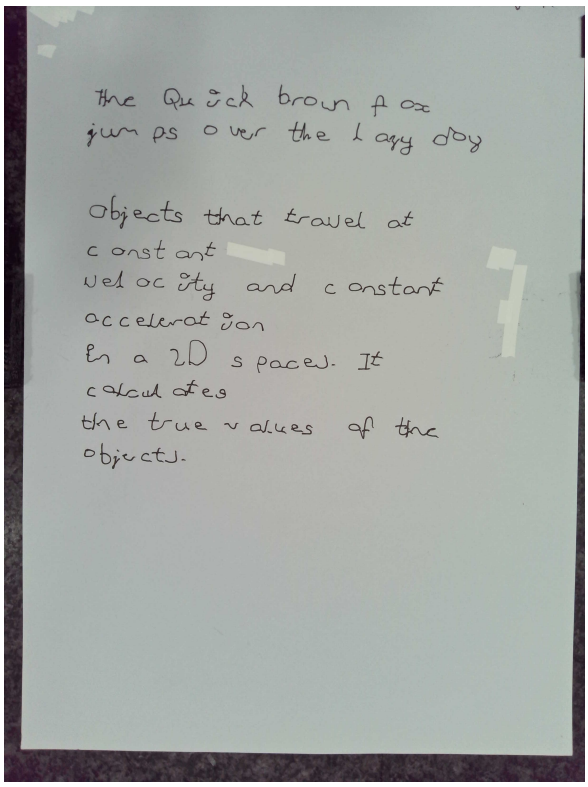


Author588

Figure 50. Physical pages Author552, Author588.

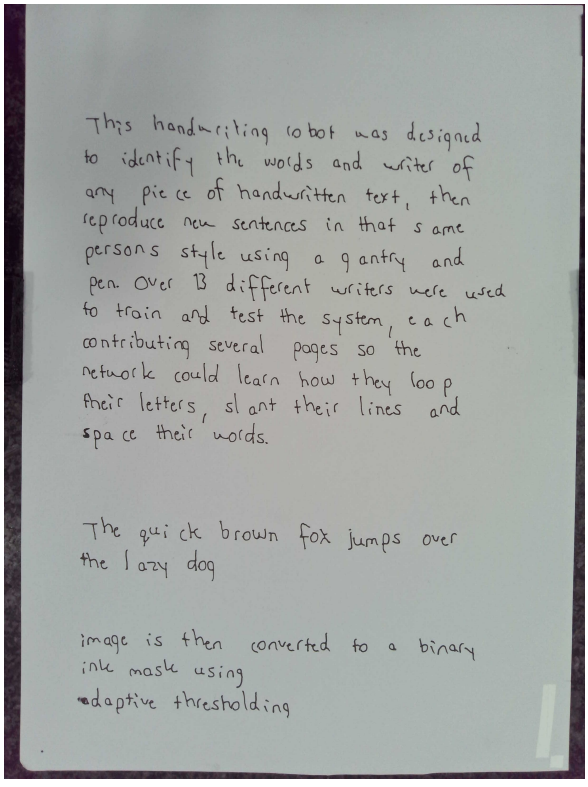


AuthorA1

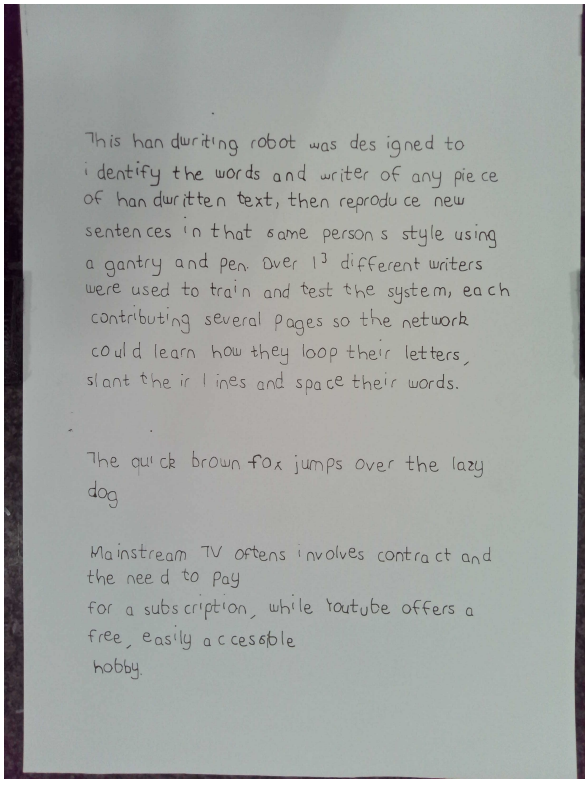


AuthorA2

Figure 51. Physical pages AuthorA1, AuthorA2.

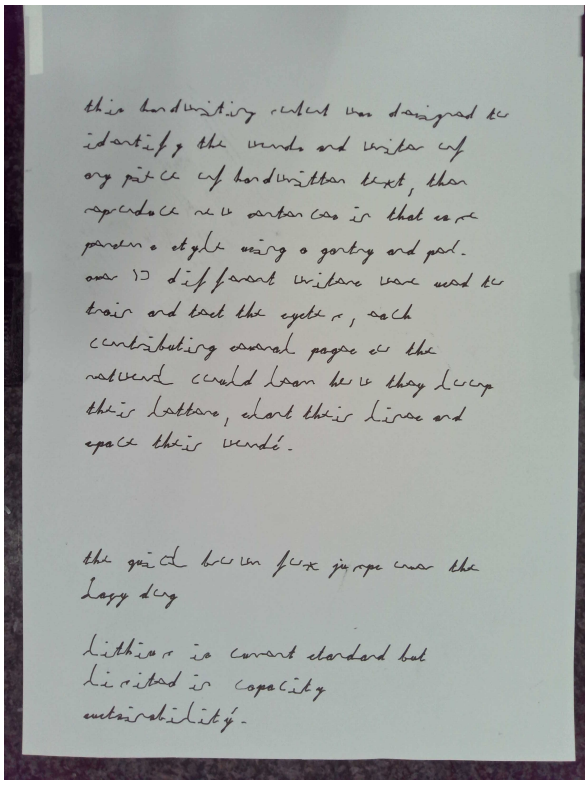


AuthorD



AuthorO

Figure 52. Physical pages AuthorD, AuthorO.



this handwriting robot was designed to identify the words and writer of any piece of handwritten text, then reproduce new sentences in that same persons style using a gantry and pen. over 13 different writers were used to train and test the system, each contributing several pages so the network could learn how they loop their letters, slant their lines and space their words.

the quick brown fox jumps over the lazy dog

in my setup, both filters are fed the exact same simulated measurement sequence in each run

AuthorT AuthorY

Figure 53. Physical pages AuthorT, AuthorY.